



TIA-102.BAAB-B-2005
APPROVED: MARCH 24, 2005
REAFFIRMED: JANUARY 29, 2013

TIA STANDARD

Project 25

Common Air Interface Conformance Test

TIA-102.BAAB-B

March 2005

**TELECOMMUNICATIONS
INDUSTRY ASSOCIATION**

tiaonline.org

NOTICE

TIA Engineering Standards and Publications are designed to serve the public interest through eliminating misunderstandings between manufacturers and purchasers, facilitating interchangeability and improvement of products, and assisting the purchaser in selecting and obtaining with minimum delay the proper product for their particular need. The existence of such Standards and Publications shall not in any respect preclude any member or non-member of TIA from manufacturing or selling products not conforming to such Standards and Publications. Neither shall the existence of such Standards and Publications preclude their voluntary use by Non-TIA members, either domestically or internationally.

Standards and Publications are adopted by TIA in accordance with the American National Standards Institute (ANSI) patent policy. By such action, TIA does not assume any liability to any patent owner, nor does it assume any obligation whatever to parties adopting the Standard or Publication.

This Standard does not purport to address all safety problems associated with its use or all applicable regulatory requirements. It is the responsibility of the user of this Standard to establish appropriate safety and health practices and to determine the applicability of regulatory limitations before its use.

(From Project No. 3-3585-RV2-RF1, formulated under the cognizance of the TIA TR-8 Mobile and Personal Private Radio Standards, TR-8.15 Subcommittee on Common Air Interface).

Published by
©TELECOMMUNICATIONS INDUSTRY ASSOCIATION
Standards and Technology Department
1320 N. Courthouse Road
Arlington, VA 22201 U.S.A.

**PRICE: Please refer to current Catalog of
TIA TELECOMMUNICATIONS INDUSTRY ASSOCIATION STANDARDS
AND ENGINEERING PUBLICATIONS
or call IHS, USA and Canada
(1-877-413-5187) International (303-397-2896)
or search online at <http://www.tiaonline.org/standards/catalog/>**

All rights reserved
Printed in U.S.A.

NOTICE OF COPYRIGHT

This document is copyrighted by the TIA.

Reproduction of these documents either in hard copy or soft copy (including posting on the web) is prohibited without copyright permission. For copyright permission to reproduce portions of this document, please contact the TIA Standards Department or go to the TIA website (www.tiaonline.org) for details on how to request permission. Details are located at:

<http://www.tiaonline.org/standards/catalog/info.cfm#copyright>

or

Telecommunications Industry Association
Technology & Standards Department
1320 N. Courthouse Road, Suite 200
Arlington, VA 22201 USA
+1.703.907.7700

Organizations may obtain permission to reproduce a limited number of copies by entering into a license agreement. For information, contact

IHS
15 Inverness Way East
Englewood, CO 80112-5704
or call
USA and Canada (1.800.525.7052)
International (303.790.0600)

Limited Use Only

NOTICE OF DISCLAIMER AND LIMITATION OF LIABILITY

The document to which this Notice is affixed (the "Document") has been prepared by one or more Engineering Committees or Formulating Groups of the Telecommunications Industry Association ("TIA"). TIA is not the author of the Document contents, but publishes and claims copyright to the Document pursuant to licenses and permission granted by the authors of the contents.

TIA Engineering Committees and Formulating Groups are expected to conduct their affairs in accordance with the TIA Engineering Manual ("Manual"), the current and predecessor versions of which are available at <http://www.tiaonline.org/standards/procedures/manuals>. TIA's function is to administer the process, but not the content, of document preparation in accordance with the Manual and, when appropriate, the policies and procedures of the American National Standards Institute ("ANSI"). TIA does not evaluate, test, verify or investigate the information, accuracy, soundness, or credibility of the contents of the Document. In publishing the Document, TIA disclaims any undertaking to perform any duty owed to or for anyone.

If the Document is identified or marked as a project number (PN) document, or as a standards proposal (SP) document, persons or parties reading or in any way interested in the Document are cautioned that: (a) the Document is a proposal; (b) there is no assurance that the Document will be approved by any Committee of TIA or any other body in its present or any other form; (c) the Document may be amended, modified or changed in the standards development or any editing process.

The use or practice of contents of this Document may involve the use of intellectual property rights ("IPR"), including pending or issued patents, or copyrights, owned by one or more parties. TIA makes no search or investigation for IPR. When IPR consisting of patents and published pending patent applications are claimed and called to TIA's attention, a statement from the holder thereof is requested, all in accordance with the Manual. TIA takes no position with reference to, and disclaims any obligation to investigate or inquire into, the scope or validity of any claims of IPR. TIA will neither be a party to discussions of any licensing terms or conditions, which are instead left to the parties involved, nor will TIA opine or judge whether proposed licensing terms or conditions are reasonable or non-discriminatory. TIA does not warrant or represent that procedures or practices suggested or provided in the Manual have been complied with as respects the Document or its contents.

If the Document contains one or more Normative References to a document published by another organization ("other SSO") engaged in the formulation, development or publication of standards (whether designated as a standard, specification, recommendation or otherwise), whether such reference consists of mandatory, alternate or optional elements (as defined in the TIA Engineering Manual, 4th edition) then (i) TIA disclaims any duty or obligation to search or investigate the records of any other SSO for IPR or letters of assurance relating to any such Normative Reference; (ii) TIA's policy of encouragement of voluntary disclosure (see Engineering Manual Section 6.5.1) of Essential Patent(s) and published pending patent applications shall apply; and (iii) Information as to claims of IPR in the records or publications of the other SSO shall not constitute identification to TIA of a claim of Essential Patent(s) or published pending patent applications.

TIA does not enforce or monitor compliance with the contents of the Document. TIA does not certify, inspect, test or otherwise investigate products, designs or services or any claims of compliance with the contents of the Document.

ALL WARRANTIES, EXPRESS OR IMPLIED, ARE DISCLAIMED, INCLUDING WITHOUT LIMITATION, ANY AND ALL WARRANTIES CONCERNING THE ACCURACY OF THE CONTENTS, ITS FITNESS OR APPROPRIATENESS FOR A PARTICULAR PURPOSE OR USE, ITS MERCHANTABILITY AND ITS NONINFRINGEMENT OF ANY THIRD PARTY'S INTELLECTUAL PROPERTY RIGHTS. TIA EXPRESSLY DISCLAIMS ANY AND ALL RESPONSIBILITIES FOR THE ACCURACY OF THE CONTENTS AND MAKES NO REPRESENTATIONS OR WARRANTIES REGARDING THE CONTENT'S COMPLIANCE WITH ANY APPLICABLE STATUTE, RULE OR REGULATION, OR THE SAFETY OR HEALTH EFFECTS OF THE CONTENTS OR ANY PRODUCT OR SERVICE REFERRED TO IN THE DOCUMENT OR PRODUCED OR RENDERED TO COMPLY WITH THE CONTENTS.

TIA SHALL NOT BE LIABLE FOR ANY AND ALL DAMAGES, DIRECT OR INDIRECT, ARISING FROM OR RELATING TO ANY USE OF THE CONTENTS CONTAINED HEREIN, INCLUDING WITHOUT LIMITATION ANY AND ALL INDIRECT, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES (INCLUDING DAMAGES FOR LOSS OF BUSINESS, LOSS OF PROFITS, LITIGATION, OR THE LIKE), WHETHER BASED UPON BREACH OF CONTRACT, BREACH OF WARRANTY, TORT (INCLUDING NEGLIGENCE), PRODUCT LIABILITY OR OTHERWISE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. THE FOREGOING NEGATION OF DAMAGES IS A FUNDAMENTAL ELEMENT OF THE USE OF THE CONTENTS HEREOF, AND THESE CONTENTS WOULD NOT BE PUBLISHED BY TIA WITHOUT SUCH LIMITATIONS.

This copy is provided to a TIA Member. It is not to be shared or posted to a web server accessible to others.

To purchase a copy of this standard go to <https://store.accuristech.com/tia> or contact TIA at standards@tiaonline.org.

Limited Use Only

Foreword

This foreword is not part of this document.

This standard is being promulgated and will be maintained by the TR-8.15 Common Air Interface Subcommittee as part of the effort of TR-8 Private Radio Technical Standards committee under the sponsorship of the Telecommunications Industry Association.

Generally, the Project 25 Common Air Interface Conformance Test and various elements thereof have been developed by TIA TR-8.15 to be consistent with the Statement of Requirements adopted by the Project 25 Steering committee. This Standards family includes and borrows heavily from the combined work of various industry and government agency representatives organized under the TIA Project 25 Ad Hoc Interface Committee.

This document presumes that APCO/NASTD/FED will establish an overall Project 25 System standard or specification. It further presumes that TIA will set standards based upon the APCO/NASTD/FED Project 25 standards or specifications.

This standard provides the Common Air Interface Conformance Test which provides information needed to subject the Project 25 FDMA Common Air Interface to a conformance test. While all reasonable efforts have been made to ensure the accuracy of this document, it should be understood that work continues in fully developing the Project 25 family of standards and that this document will be updated as necessary to ensure an accurate representation of Project 25 systems as other implementation requirements become available.

The reader's attention is called to the possibility that compliance with the APCO/NASTD/FED Project 25 Standard or any TIA standard for equipment conforming to the APCO/NASTD/FED Project 25 Standard may require the use of one or more inventions covered by patent rights. By publication of those standards, if any, no position is taken with respect to the validity of those claims or any patent rights in connection therewith. The patent holders so far identified have, however filed statements of willingness to grant licenses under those rights on reasonable and nondiscriminatory terms and conditions to applicants desiring to obtain such licenses. Details may be obtained from the publisher.

THIS PAGE INTENTIONALLY LEFT BLANK.

Limited Use Only

Table of Contents

1. Scope	1
2. History and References	2
3. Description.....	3
4. Test Parameters	5
4.1 Network Access Codes	5
4.2 Status Symbols	6
4.3 Link Control Fields	6
4.3.1 Talk Group IDs	6
4.3.2 Source and Destination IDs	6
4.3.3 Emergency Bit	6
4.3.4 Manufacturer's ID	6
4.4 Encryption Fields	7
4.5 Low Speed Data	7
4.6 IMBE Voice	7
4.7 Data Packet Fields	8
4.8 Null Bits	8
5. Transmit Voice Format Tests	9
5.1 Transmit Modulation Test	9
5.1.1 C4FM Transmit Polarity	9
5.1.2 Modulation Fidelity	10
5.1.3 CQPSK Transmit Polarity	10
5.2 Network Access Code Test	11
5.3 Status Symbol Test	11
5.4 Unencrypted Voice Header	12
5.5 Unencrypted Link Data Unit 1	12
5.6 Unencrypted Link Data Unit 2	13
5.7 Silent Audio	13
6. Transmit Data Format Tests	14
6.1 Confirmed Data Packets	14
6.2 Response Packets	14
6.3 Unconfirmed Data Packets	15
7. Receiver Tests	16
7.1 Voice Receiver Tests	16
7.2 Data Receiver Tests	18
Annex A. Test Modules	21
A.1 Conventions	21
A.2 Primitive Modules	21
A.2.1 BCH Code Module	22
A.2.2 CRC Code Module	24
A.2.3 Cyclic Code Module	26
A.2.4 Golay Code Module	28
A.2.5 Hamming Code Module	30
A.2.6 Reed-Solomon Code Module	32
A.2.7 Trellis Code Module	36
A.2.8 Voice Code Module	39
A.2.9 File Input Module	43
A.2.10 Vector Operation Module	45

A.3 Voice Data Unit Programs.....	47
A.3.1 Voice Header Data Unit.....	47
A.3.2 Voice Link Data Unit 1	50
A.3.3 Voice Link Data Unit 2	53
A.3.4 Voice Terminator	56
A.4 Data Packet Programs	58
A.4.1 Confirmed Data Packet	58
A.4.2 Response Data Packet.....	62
A.4.3 Unconfirmed Data Packet.....	65
Annex B. Test Vectors.....	69
B.1 Network Access Code Test Vectors.....	69
B.2 Status Symbol Test Vectors	72
B.3 Unencrypted Voice Header Test Vectors	78
B.4 Unencrypted Link Data Unit 1 Test Vectors	86
B.5 Unencrypted Link Data Unit 2 Test Vectors	116
B.6 Silent Audio Test Vectors	120
B.7 Confirmed Data Packet Test Vectors	122
B.8 Response Packet Test Vectors	130
B.9 Unconfirmed Data Packet Test Vectors	132
B.10 Voice Receiver Test Vectors	135
B.11 Data Receiver Test Vectors	137

Terminology

A	Acknowledgment request bit in a data packet. A=1 to request an ACK.
ALGID	Algorithm Identifier for an encryption algorithm
C4FM	Compatible 4-level FM version of QPSK-c modulation
CQPSK	Compatible QPSK version of QPSK-c modulation
CRC	Cyclic Redundancy Check; data packets have several CRCs
Dibit	Two bits grouped together to represent a 4-level symbol
DUID	Data Unit Identifier
ES	Encryption Synchronization information
FMF	Full Message Flag in data packet. FMF=1 for a full message.
FS	Frame Synchronization to mark the first information bit
FSNF	Fragment Sequence Number Field in a data packet.
Golay	Name of a standard error correction code
IMBE	Improved Multi-Band Excitation coder for voice
IO	Inbound / Outbound bit in a data packet. IO = 0 for inbound.
KID	Key Identifier
LC	Link Control information
LCF	Link Control Format identifier
LDU	Link Data Unit, there is an LDU 1 and LDU 2 for voice
LLID	Logical Link Identifier, can be either a source or destination ID.
LSD	Low Speed Data embedded in voice
MFID	Manufacturer's Identifier
MI	Message Indicator
NAC	Network Access Code
NID	Network Identifier, contains the NAC and DUID
N(S)	Sending Number for a data packet, to distinguish duplicate packets.
Octet	Eight bits grouped together, also called a byte
QPSK-c	Compatible Quadrature Phase Shift Keying family of modulations
RS	Reed-Solomon error correction code
SAP	Service Access Point, where a network provides a service
SS	Status Symbol to represent whether a channel is busy or idle
TGID	Talk Group Identifier
VC	Voice Code word, encodes 20 ms of speech with IMBE

This page intentionally left blank.

Limited Use Only

1. Scope

This document lists a series of conformance tests for the Common Air Interface, defined in reference [2]. These tests are intended to assure that equipment actually conforms to the formats specified in the Common Air Interface. The object of the conformance tests is to assure that equipment may be interoperable with other equipment conforming to the standard. These tests are different and distinct from performance tests, given in reference [5], which measure the actual limits of equipment performance. The performance and conformance tests are mutually complementary. These tests are also different and distinct from lock down tests, which are intended to demonstrate interoperability between different radios. These conformance tests are intended to precede lock down tests.

These tests are generally intended to be applied by a manufacturer to a product implementation to verify that it conforms to the Common Air Interface. These tests only apply to radio products, particularly portable and mobile radios and base stations or repeaters. Some of the tests are designed to apply to transmitters, by specifying the transmit signal under a representative set of transmitter conditions. Other tests are designed to apply to receivers, by defining a standard test signal, and then specifying the minimum response of the receiver to the test signal.

The specific test cases for the transmitter tests are selected to exercise the encoders for the various codes used in the Common Air Interface. In general the transmitter tests can be applied to a receiver by exciting the receiver with the test signals and then measuring the receiver's response. The required response of the receiver to the transmitter test signals is not defined in this document, but it may be inferred from the information content of the test signal, the conditions of the test signal, and the operating characteristics of the receiver. The specific test cases and conditions for the receiver tests are selected to exercise the error correcting and detecting mechanisms in the receiver.

2. History and References

February 25, 1993, P25.930225.1.0, contained the Scope and Description.

August 3, 1993, P25.930225.1.1, included test vectors.

September 17, 1993, P25.930225.1.2, corrected software defects.

March 7, 1994, P25.930225.1.3, added receiver tests.

May 3, 1994, P25.930225.1.4, moved test vectors into an appendix.

Version A, February 8, 1995, removed IMBE mute test vector and updated some software to be more independent of the computer hardware. This version was not published.

Version B, April 7, 1995, added modulation fidelity test, published as revision A.

Version B.2, July 28, 1998, revised for phase 2.

Version 2.0, June 11, 2003, distributed for promotion to a standard.

Version 2.1, October 3, 2003, revised the foreword.

The following documents contain provisions which, through reference in this text, constitute provisions of this document. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the standards indicated below. ANSI and TIA maintain registers of currently valid national standards published by them.

1. *Project 25 System and Standard Definition*, November 1995, TIA TSB-102-A.
This contains within it reference [1.1]
- 1.1. *APCO 25 General System Model*, APCO Project 25, 9/30/92, P25.920901.2.1.
2. *Project 25 FDMA Common Air Interface*, APCO Project 25, September 2003, TIA102.BAAA.
3. *Project 25 Vocoder Description*, TIA, May 1998, TIA102.BABA.
4. *DES Encryption Protocol*, TIA, February 2001, TIA102.AAAA.
5. *Digital C4FM/CQPSK Transceiver Measurement Methods*, TIA, November 2002, TIA102.CAAA.
6. *Common Air Interface Operational Description for Conventional Channels*, TIA, December 2003, TIA102.BAAD.
7. *Digital C4FM / CQPSK Transceiver Performance Recommendations*, TIA, September 2002, TIA102.CAAB.

3. Description

The voice formats are based on a number of code words that are described in detail in reference [2]. For the purposes of conformance testing these code words will be abbreviated as follows.

FS -- This is the Frame Synchronization word. It is a fixed 48 bit pattern.

NID -- This is the Network Identifier code word. It is 64 bits long with two information fields: a Network Access Code (NAC) and a Data Unit ID (DUID).

SS -- This is the Status Symbol. It is 2 bits long. It is periodically inserted in the voice message after every 70 bits of information.

HDR -- This is the Header Data Unit code word. It is 648 bits long with several information fields.

LC -- This is the Link Control word. There are two slightly different LC words. One is embedded within a voice message and it is 240 bits long. The other is defined for a Terminator and is 288 bits long. The Link Control word contains a Link Control Format (LCF) field, the Manufacturer's ID (MFID) field, and several other variable fields.

ES -- This is the Encryption Sync word. It is 240 bits long and contains the Message Indicator (MI), Algorithm ID (AlgID), and Key ID (KID) fields.

LSD -- This is the Low Speed Data word. It is a 16 bit code word that contains 8 bits of low speed data. There are 2 LSD words embedded in every Logical Link Data Unit of voice.

VC -- This is the voice code word. It is 144 bits long and contains one IMBE frame.

The construction of a voice message is diagrammed in Figure 3-1.

The conformance tests described here will test that the encoders in the diagram are operating properly and that the interleaving function for voice messages operates properly. In general, these tests consist of the specification of the various signal inputs and the resulting modulation output. The signal inputs are usually externally accessible for manipulation by programming a radio's operation. The modulation output is usually available at an antenna port, or optionally, within the radio before being applied to the modulator.

Each of the code words is constructed from one or more primitive code words. For example, the NID code is constructed from a primitive BCH code while the HDR code is constructed from a concatenation of a primitive Reed-Solomon (RS) code and a Golay code. Reference [2] provides two definitions for these codes, one based on the generator polynomial and one based on a generator matrix. In an implementation which uses the generator matrix, the transmitter performs a matrix multiplication to compute the transmit code word from the information vector. A test that is designed to validate this type of design must test enough code words to ensure that the generator matrix in the design is correct. This requires at least K test vectors, given an (N,K) code using a generator matrix with

K rows and N columns. Furthermore, the K test vectors must be linearly independent. The generator matrix, by definition, provides K rows that are linearly independent code words, and therefore suitable for a test. In this document, the test vectors will be constructed from standardized parameter values for the information fields where ever possible.

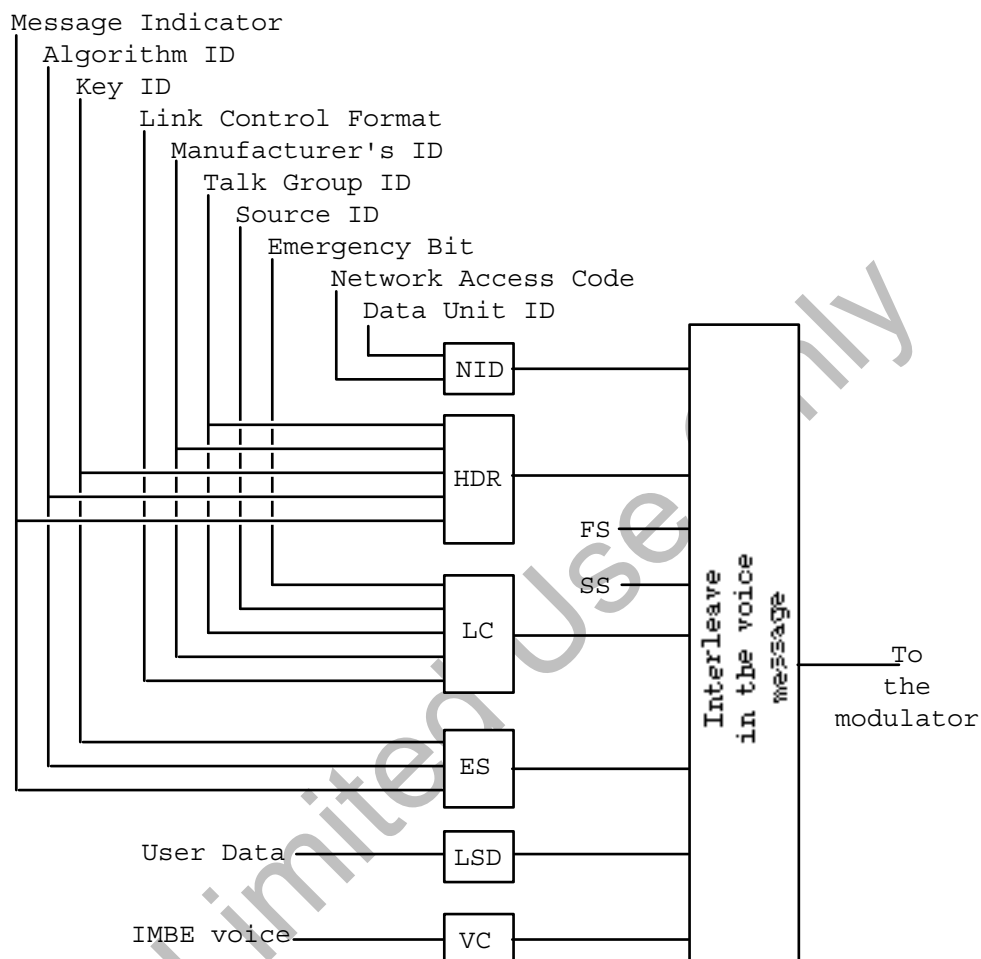


Figure 3-1 Voice Encoder Arrangement for a Transmitter

The corresponding encoder arrangement for data is given in reference [2]. Data packets are separated into blocks, which are each encoded and interleaved with a trellis encoder. The first block is called the header block and contains several information fields defined for the Common Air Interface. Preceding the header block the transmitter sends a Frame Sync and Network ID just as is done for voice data units. The tests for data define enough test cases to fully exercise the trellis encoders as well as the CRC encoders for the data packets.

The arrangement for decoders for a receiver for voice messages is shown in Figure 3-2. In general a voice receiver can be separated into decoder blocks just as the transmitter was separated into blocks. These blocks include decoders for the Frame Sync, NID, Status Symbols, IMBE Voice, Link Control, Encryption

Sync, Header, and Low Speed Data. The conformance tests described in this document will test that the decoders process the correct information and deliver results for use in the radio. The conformance test does not require the decoders to actually exist, but when they do exist in an implementation they should pass a minimum requirement.

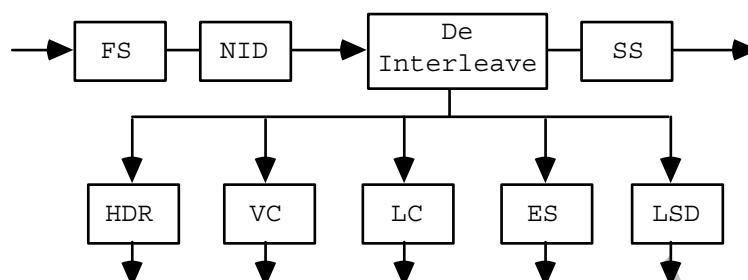


Figure 3-2. Voice Receiver Decoder Arrangement

4. Test Parameters

This section catalogs the parameters which are used for the voice and data tests in Sections 5 and 6 respectively. The parameters, and the resulting voice or data code words, are listed in hexadecimal notation. The parameters have various field lengths, some as short as 1 bit and others as long as 72 bits. The code words are frequently even longer. For convenience, these are represented as strings of hexadecimal digits that are right justified. The right hand digits are the least significant, following the usual convention. The convention of the Common Air Interface document, that is followed here, is that the right hand digits correspond to signals which are transmitted last, while the left hand digits are transmitted first.

In general the vectors of information are often grouped into 2 or 3 digit groups. The 2-digit groups are used where information naturally uses octets, usually for data packets. The 3-digit groups are used primarily for vectors of information which are transmitted over the Common Air Interface. These 3-digit groups contain 12 bits, or 6 dibit symbols, as defined in the Common Air Interface. This is done by selecting pairs of bits, and mapping them into the modulation as specified by the Common Air Interface. For example:

1b3 hexadecimal, becomes
 00 01 10 11 00 11 in dibits, which becomes
 +1 +3 -1 -3 +1 -3 in symbol amplitudes.

These symbols are transmitted starting from the left (+1) and finishing on the right (-3).

4.1 Network Access Codes

The following Network Access Code values are used.

Default value for most tests 293

For the Network Access Code Test

023	031	032	043	114	205
40b	411	464	516	585	811

The values for the Network Access Code Test are linearly independent.

4.2 Status Symbols

There are four possible values for the Status Symbols. They are used as follows.

01 binary	used as a default for repeater tests
00 binary	used as a default for subscriber radio tests
10 binary	used in the Status Symbol Test
11 binary	used in the Status Symbol Test

4.3 Link Control Fields

4.3.1 Talk Group IDs

A 16-bit value is used for Talk Group IDs. The values used in the tests are as follows.

Default used for most tests.	0001
Used to exercise the voice header and LDU 1 tests.	
0002 0004 0008 0010 ...	2000 4000 8000

4.3.2 Source and Destination IDs

These are exercised in the LDU 1 tests.

Default used for most tests	000001
Used to exercise LDU1.	
000002 000004 000008 000010 ...	400000 800000

4.3.3 Emergency Bit

The emergency bit has only two values. The routine value (0) is used as a default, and the emergency value (1) is used in the tests of LDU1.

4.3.4 Manufacturer's ID

Only the standard value (00) for the manufacturer's ID is used. Testing the transmitter for other specific values is left to the manufacturer.

4.4 Encryption Fields

The encryption fields consist of the Message Indicator (MI), Algorithm ID (ALGID), and Key ID (KID). The defaults used for the unencrypted tests in this document are as follows.

Default MI	00 0000 0000 0000 0000
Default ALGID	80
Default KID	0000

4.5 Low Speed Data

The null value for Low Speed Data is used in most tests. In the tests of LDU 2, the other eight values are used to exercise the cyclic code for Low Speed Data.

ASCII LSD

null	00	used as default in most tests
@	40	
A	41	
B	42	
D	44	
H	48	
P	50	
p	70	
	80	

4.6 IMBE Voice

There are two frames of IMBE information used for testing here. The 1011 Hz test tone is used as a default for most tests. The frame for silence is used to test the voice parts of LDU1 and LDU2.

1011 Hz test tone frame

u₀ u₁ u₂ u₃ u₄ u₅ u₆ u₇
 09b 088 0cc 621 7b4 02a 04c 00

Output frame of 144 bits:

3892 8490 d433 c0be 1b91 844f f058 a589 d839

Silence frame

u₀ u₁ u₂ u₃ u₄ u₅ u₆ u₇
 040 cfd 7bf b7d 793 6cf 33c 45

Output frame of 144 bits:

6c42 e85d e2e8 2693 63d9 81f9 be23 b18a e006

The least significant bit of u₇ is defined to alternate (0, 1, 0, 1, ...) on consecutive voice frames. This convention is followed in the tests for LDU1 and LDU2 in Section 5.

4.7 Data Packet Fields

The fields of information in the header block for data packets are exercised in section 6. The SAP values of 00 and 1f are tested for standard unencrypted data and enhanced addressing respectively.

4.8 Null Bits

In several test cases this document lists the null bits following the code words. According to reference [2] these null bits are not part of the message. Although the null bits are listed with zero values, the equipment under test shall not be required to make identical zero values for the null bits.

Limited Use Only

5. Transmit Voice Format Tests

This section contains tests for the voice formats and voice data units of the Common Air Interface. These tests apply to transmitters.

5.1 Transmit Modulation Test

5.1.1 C4FM Transmit Polarity

This tests that the transmitter has the correct polarity. The transmitter's polarity is tested by keying the transmitter and observing the frame synchronization pattern on a test receiver. A modulation analyzer and an oscilloscope are suitable for observing the frame synchronization.

The frame synchronization pattern for C4FM modulation is shown in Figure 5.1-1. A C4FM transmitter shall deviate from the nominal carrier frequency in the direction shown in Figure 5.1-1, starting at the time value of 0 and sampling 24 times every 208.33 microseconds.

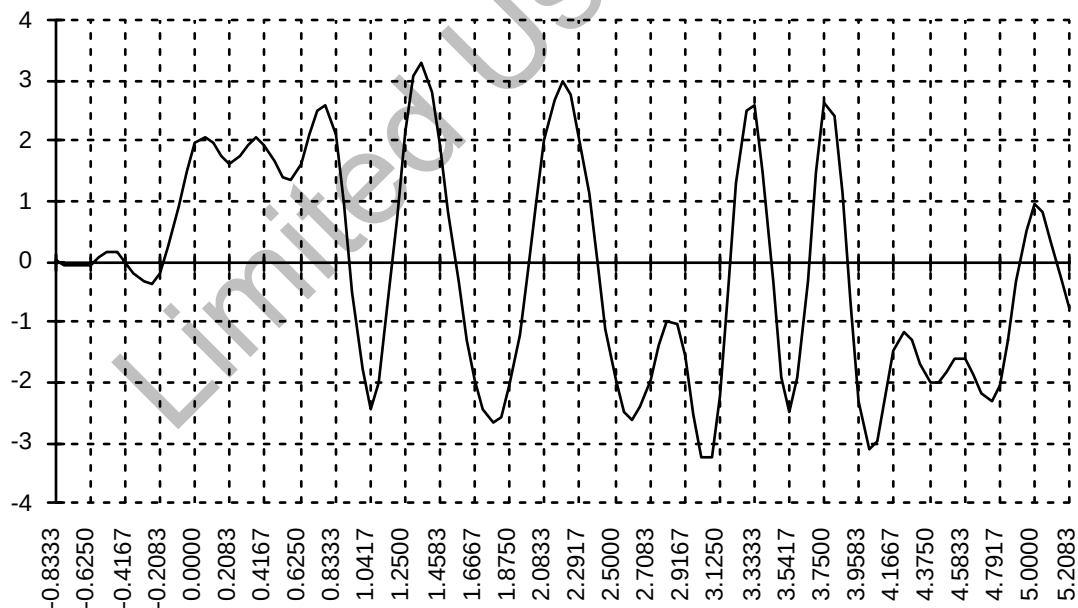


Figure 5.1-1 Frame Synchronization Pattern for C4FM

Vertical: Deviation in kHz

Horizontal: Time in ms

Time origin is the sample instant for the first symbol of frame sync.

Peak deviation is +3.30 kHz and -3.30 kHz.

5.1.2 Modulation Fidelity

The measurement of Modulation Fidelity for both C4FM and CQPSK is defined in reference [5]. The specified performance limits are given in reference [7]. If a transmitter implements C4FM or CQPSK modulation, then it shall conform to the modulation fidelity specified in reference [7] for the respective modulation that is implemented.

5.1.3 CQPSK Transmit Polarity

This tests that the transmitter has the correct polarity. The transmitter's polarity is tested by keying the transmitter and observing the frame synchronization pattern on a test receiver. A modulation analyzer and an oscilloscope are suitable for observing the frame synchronization.

The frame synchronization pattern for CQPSK modulation is shown in Figure 5.1-2. The instantaneous deviation is shown, consequently the peak deviation is unbounded. A CQPSK transmitter shall deviate from the nominal carrier frequency in the direction shown in Figure 5.1-2, starting at the time value of 0 and sampling 24 times every 208.33 microseconds.

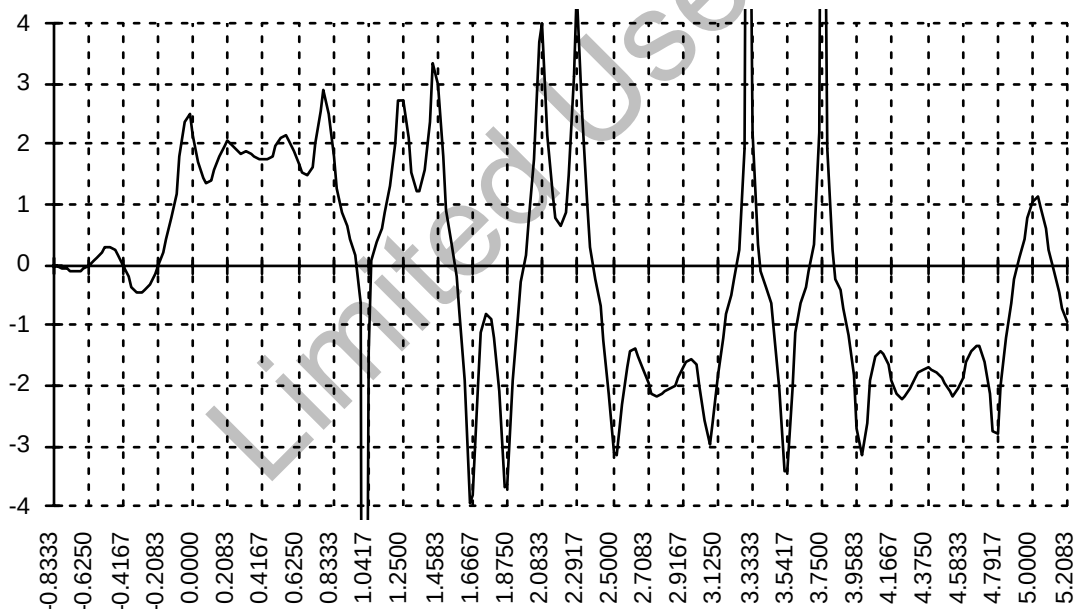


Figure 5.1-2 Frame Synchronization Pattern for CQPSK

Vertical: Deviation in kHz

Horizontal: Time in ms

Time origin is the sample instant in the middle of the first symbol of sync.

5.2 Network Access Code Test

This tests the BCH encoder for the Network Identifier. A linearly independent set of Network Access Codes are selected for this test. These codes are then applied to a voice message, and the voice terminator is recorded. There are two possible test cases; one for a subscriber radio and one for a repeater radio.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Terminator Data Unit test vectors as tabulated in Section B.1 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the Terminator Data Unit test vectors as tabulated in Section B.1 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver shall be excited with a generator producing the equivalent subscriber data unit test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

5.3 Status Symbol Test

This test determines that Status Symbols are inserted properly into messages. Each of the four Status Symbol cases is tested. There are two cases, one for subscriber radios and one for repeaters.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the data unit test vectors as tabulated in Section B.2 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the first 4 data unit test vectors as tabulated in Section B.2 for the Repeater Radio Case. Wireline controlled repeaters shall also be tested to produce the last 4 data unit test vectors as tabulated in Section B.2 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver shall be excited with a generator producing the equivalent subscriber data unit

test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

5.4 Unencrypted Voice Header

This tests the concatenated Reed-Solomon and Golay code word in the Header Data Unit. There are two cases given, one for a subscriber radio and one for a repeater.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Header Data Unit test vectors as tabulated in Section B.3 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the Header Data Unit test vectors as tabulated in Section B.3 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver shall be excited with a generator producing the equivalent subscriber data unit test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

5.5 Unencrypted Link Data Unit 1

This tests the concatenated Reed-Solomon and Golay code word in Link Data Unit 1. There are two cases given, one for a subscriber radio and one for a repeater.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Link Data Unit 1 test vectors as tabulated in Section B.4 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the Link Data Unit 1 test vectors as tabulated in Section B.4 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver shall be excited with a generator producing the equivalent subscriber data unit

test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

5.6 Unencrypted Link Data Unit 2

This tests the concatenated Reed-Solomon and Golay code word in Link Data Unit 2. An unencrypted message only has a single instance for this code word, and this is all that is given for each of the cases. In addition, the Low Speed Data words are tested here. There are two cases given, one for a subscriber radio and one for a repeater.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Link Data Unit 2 test vectors as tabulated in Section B.5 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the Link Data Unit 2 test vectors as tabulated in Section B.5 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver shall be excited with a generator producing the equivalent subscriber data unit test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

5.7 Silent Audio

This tests the encoding and interleaving functions for IMBE frames using silent audio. There are two cases given, one for a subscriber radio and one for a repeater.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Link Data Unit test vectors as tabulated in Section B.6 for the Subscriber Radio Case.

Case 2: Repeater Radio

For each of the indicated information fields, the repeater radio shall produce the Link Data Unit test vectors as tabulated in Section B.6 for the Repeater Radio Case. For cases where a transparent repeater is to be tested, i.e., a repeater transmitter which does not actually encode the transmitted code words, but simply repeats bit-for-bit the code words that are received, the repeater receiver

shall be excited with a generator producing the equivalent subscriber data unit test vectors, and the repeater transmitter shall produce the data unit test vectors as tabulated in Section B.

6. Transmit Data Format Tests

This section contains test vectors for data packets. The packets themselves are arranged into microslots just like the voice data units. In addition, the CRC values are displayed for the header block, the data message, and (in the case of confirmed data) the selectable data blocks. These tests apply to transmitters.

6.1 Confirmed Data Packets

These tests exercise the trellis codes and interleave functions for confirmed data packets. Some representative values for the fields are given in various cases. In addition there is one partial packet retry, and a packet with enhanced addressing. As in the case of voice, there are two separable cases. One case is for subscriber radios and one is for base station radios. The base station radio formats are constructed for outbound transmissions originating from the fixed network equipment. Consequently the address is a destination address in the header block. Subscriber radios shall produce the sample packets given in Case 1, while base station radios shall produce the sample packets given in Case 2.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Packet Data Unit test vectors as tabulated in Section B.7 for the Subscriber Radio Case.

Case 2: Base Station Radio

For each of the indicated information fields, the base station radio shall produce the Packet Data Unit test vectors as tabulated in Section B.7 for the Base Station Radio Case.

6.2 Response Packets

These tests are representative response packets. One case is for subscriber radios and one is for base station radios. The base station radio formats are constructed for outbound transmissions originating from the fixed network equipment. Consequently the address is a destination address in the header block. Subscriber radios shall produce the sample packets given in Case 1, while base station radios shall produce the sample packets given in Case 2.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Packet Data Unit test vectors as tabulated in Section B.8 for the Subscriber Radio Case.

Case 2: Base Station Radio

For each of the indicated information fields, the base station radio shall produce the Packet Data Unit test vectors as tabulated in Section B.8 for the Base Station Radio Case.

6.3 Unconfirmed Data Packets

These tests exercise the trellis codes and interleave functions for unconfirmed data packets. Some representative values for the fields are given in various cases. There is one packet with enhanced addressing. As in the case of voice, there are two separable cases. One case is for subscriber radios and one is for base station radios. The base station radio formats are constructed for outbound transmissions originating from the fixed network equipment. Consequently the address is a destination address in the header block. Subscriber radios shall produce the sample packets given in Case 1, while base station radios shall produce the sample packets given in Case 2.

Case 1: Subscriber Radio

For each of the indicated information fields, the subscriber radio shall produce the Packet Data Unit test vectors as tabulated in Section B.9 for the Subscriber Radio Case.

Case 2: Base Station Radio

For each of the indicated information fields, the base station radio shall produce the Packet Data Unit test vectors as tabulated in Section B.9 for the Base Station Radio Case.

7. Receiver Tests

This section contains test data units for voice or data that are used to test the receiver response. In all cases the test data units correspond to test cases used in Section 6 or 7. The ability of the receiver to de-interleave, perform error correction, and extract data from the transmitted information should be verified by these tests. A receiver shall be excited

- with a standard input signal,
- with the standard tone test pattern (in the case of voice),
- at the reference sensitivity (either static or faded),
- as defined in reference [5].

The receiver shall then be verified that it receives correctly. The ability to decode frame sync, NID, LDUs, data frames, etc. ... should be verified by these tests.

7.1 Voice Receiver Tests

The standard tone test pattern shall consist of the header, LDU1, LDU2, and terminator data units given in section B.10. The data units shall be applied to the receiver under test in the order given. The data units LDU1 and LDU2 may be optionally repeated as often as necessary to conduct the test.

The signal applied to the receiver shall be adjusted to the reference sensitivity as given in reference [5]. The reference sensitivity is intended to deliver the standard bit error rate in reference [5] of 5% BER. While this condition is measured in reference [5], verification of the measurement shall not be required for conformance. Instead, this test shall be performed with the level measured by the measurement procedure given in reference [5] for reference sensitivity in static conditions and again for faded reference sensitivity in fading conditions.

The subscriber radio response to the test signal and test conditions shall be determined by the presence or absence of the various decoders for the signal as depicted in Figure 3-2. An error in one or more of the fields of the decoder output for each instance of a data unit shall count as a single decoder error when computing the average.

If a Status Symbol decoder is present then the decoded status output shall be zero (0) for at least 78% of the symbols over an interval of at least 100 symbols (750 ms) for the static condition or a time interval of at least t seconds for the faded condition, where t is given by the following formula.

$$t = \frac{180000}{F_{\text{MHz}} S_{\text{km/hr}}} \quad \begin{array}{l} F_{\text{MHz}} = \text{receiver operating frequency in MHz} \\ S_{\text{km/hr}} = \text{vehicle speed in km/hr} \end{array}$$

A receiver is able to decode the header code word on a voice message if it can decode the Frame Sync, NID, and the header code word itself. A test of the

header code word decoder reliability is therefore a test of the Frame Sync and NID decoders as well. If the Header block decoder is present then the decoder shall decode the following values at least 96% of the time for the static condition and 67% of the time for the faded condition when averaged over at least 100 header data units.

```

MI      = 00 0000 0000 0000 0000
MFID    = 00
ALGID   = 80
KID     = 0000
TGID    = 0001

```

If the receiver is equipped with a Normal Squelch as defined in reference [6], and the squelch is programmed for the default NAC value of 293, and a Voice decoder is present, then the speaker output should approximate a 1011 Hz tone. While this is easily checked by listening to the receiver, this is not a normative requirement of this section, because the tone is typically distorted in a complex way at 5% BER. The normative requirement for the voice coder, in the conditions previously mentioned, is that the voice decoder (VC in Figure 3-2) shall decode the coset code words c_0 , c_1 , ... c_6 , to present the vectors u_0 , u_1 , ... u_6 as given in Section 4.6 for the 1011 Hz tone a minimum percentage of the time tabulated in Table 7.1-1 when averaged over at least 100 voice frames (20 ms each). This test measures the Golay and Hamming decoders, the coset code generation, and the de-interleaving function for the voice frame; but it specifically excludes the detection of uncorrectable errors as defined in reference [3]. The u_7 value is not tested here because it is not protected from errors by an error correction code.

Table 7.1-1 Minimum Voice Decoder Success Rates at 5% BER

Value	Static	Faded	Value	Static	Faded
u_0	90%	77%	u_4	65%	77%
u_1	85%	77%	u_5	65%	77%
u_2	85%	77%	u_6	65%	77%
u_3	85%	77%			

A receiver is normally able to decode the Link Control without decoding the Frame Sync and NID of the LDU1. A receiver will normally "flywheel" through a channel fade which prevents the detection of the FS or NID. Consequently, the test of the Link Control word is conditioned on the receiver being already synchronized to the signal. If the Link Control decoder is present the output shall decode:

```

LCF     = 00
MFID    = 00
TGID    = 0001
Source= 000001

```

at least 91% of the time for the static condition and 71% of the time for the faded condition when averaged over at least 100 LDU1 data units, while the receiver maintains sync. If the radio displays the source ID, then a displayed value of 000001 shall be sufficient to pass the test. If the radio displays the talk group ID, then a displayed value of 0001 shall be sufficient to pass the test.

If the Encryption Sync decoder is present then the output shall decode:

```
MI      = 00 0000 0000 0000 0000
ALGID   = 80
KID     = 0000
```

at least 91% of the time for the static condition and 67% of the time for the faded condition when averaged over at least 100 LDU2 data units, while the receiver maintains sync.

If the Low Speed Data decoder is present then the output shall decode the repeating sequence:

```
00 00 40 41 00 00 40 41 ...
```

with at least 84% of the octets correct for the static condition and 63% of the octets correct for the faded condition when averaged over at least 100 octets, equivalent to 25 superframes, while the receiver maintains sync.

7.2 Data Receiver Tests

The standard data test pattern shall consist of the data packets, one confirmed and one unconfirmed, given in Section B.11.

The signal applied to the receiver shall be adjusted to the reference sensitivity as given in reference [5]. The reference sensitivity is intended to deliver the standard bit error rate in reference [5] of 5% BER. While this condition is measured in reference [5], verification of the measurement shall not be required for conformance. Instead, this test shall be performed with the level measured by the measurement procedure given in reference [5] for reference sensitivity in static conditions and again for faded reference sensitivity in fading conditions.

The receiver response to the test signal and test conditions shall be determined by the presence or absence of the decoders for confirmed and unconfirmed messages.

If the receiver is equipped to decode either type of data packet, then it must possess decoders for the Frame Sync, NID, and at least the Header Block of the packet. In this case the receiver shall decode the contents of the Header Block, as well as the FS and NID, for at least 84% of the packets in a sample of at least 100 packets (confirmed or unconfirmed) for the static condition, and at least 63% of the packets in a sample of at least 100 packets (confirmed or unconfirmed) for the faded conditions.

If a decoder for confirmed data is present then the data blocks shall pass the 9-bit CRC check for at least 22% of the data blocks for a sample of at least 100 data blocks (20 test packets) for the static condition, or at least 32% of the data blocks for a sample of at least 100 data blocks (20 test packets) for the faded condition. Only data blocks following decoded header blocks shall be counted.

If a decoder for unconfirmed data is present then the data block shall pass the 32-bit CRC check for at least 84% of the blocks for a sample of at least 100

unconfirmed data packets for the static condition or at least 75% of the blocks for a sample of at least 100 unconfirmed data packets for the faded condition. Only data blocks following decoded header blocks shall be counted.

Limited Use Only

This page is intentionally left blank.

Limited Use Only

Annex A. Test Modules

This annex is informative only. It contains a list of modules and programs written in C that may be used to generate test vectors. As such, the modules are representative working functions simulating a working transmitter conforming to the Common Air Interface standard.

A.1 Conventions

These C programs are unavoidably machine dependent. The assumptions of the machine are as follows.

1. These modules perform extensive binary shifting, to move individual bits around within integers. The shifts are represented in the C language as the >> and << operators. These must be equivalent to logical shifts, which is to say, they must shift binary bits by integer numbers, and shift zeroes into the integer.
2. Integers are treated as 16 bit values. In places where the programs shift information beyond 16 bits, a masking operation has been performed to limit the information to the 16 least significant places. As such, these programs should operate with a 32 bit value for integers.
3. Long integers are treated as 32 bit values. These programs will not work if the long integer cannot store 32 bits.
4. A modular design has been followed. The program is divided up into primitive functions, which are individually listed as separate files. The files are then independently compiled. After the step of compilation, there is a linking step which combines the compiled binary objects into useful executable programs. This design allows the individual objects to be tested with module tests. Consequently, the testing requirement on the final executable programs is greatly reduced.
5. In some places an octal representation for integers is used, and in others, a hexadecimal representation is used. The octal representation is used in places to be compatible with the notation in the Common Air Interface document (reference [2]). The hexadecimal representation is used to be compatible with this document, which benefits from a more compact representation.

A.2 Primitive Modules

This section contains the primitive modules which implement the low level codes in the Common Air Interface.

A.2.1 BCH Code Module

This module consists of two files:

bch.h which is an include file, and
bch.c which is the function definition for the BCH encoder.

```

/*****
    Name:   bch.h
    Author: Al Wilson
    Date:   7/28/93
    History: 7/28/93  New
  *****/

```

This module declares one function:

bch_64_encode (in, out) encodes (64,16,23) primitive BCH code

```

/*****
#ifndef BCH_H
#define BCH_H

void bch_64_encode ( int, int * );

#endif

```

```

/*****
    Name:   bch.c
    Author: Al Wilson
    Date:   7/28/93
    History: 7/28/93  New
           2/15/95  revised to be more machine independent
  *****/

```

This module contains one functions:

bch_64_encode (in, out) encodes (64,16,23) primitive BCH code

The functions takes a 16 bit integer as an argument, and returns four 16-bit values for the corresponding code word.

```

/*****
#include "bch.h"

static int matrix[16][4] = {0x8000, 0xcd93, 0x0bdd, 0x3b2a,
                           0x4000, 0xab5a, 0x8e33, 0xa6be,
                           0x2000, 0x983e, 0x4cc4, 0xe874,
                           0x1000, 0x4c1f, 0x2662, 0x743a,
                           0x0800, 0xeb9c, 0x98ec, 0x0136,
                           0x0400, 0xb85d, 0x47ab, 0x3bb0,
                           0x0200, 0x5c2e, 0xa3d5, 0x9dd8,
                           0x0100, 0x2e17, 0x51ea, 0xceec,
                           0x0080, 0x170b, 0xa8f5, 0x6776,
                           0x0040, 0xc616, 0xdfa7, 0x8890,
                           0x0020, 0x630b, 0x6fd3, 0xc448,
                           0x0010, 0x3185, 0xb7e9, 0xe224,
                           0x0008, 0x18c2, 0xdbf4, 0xf112,
                           0x0004, 0xc1f2, 0x6627, 0x43a2,
                           0x0002, 0xad6a, 0x38ce, 0x9afb,
                           0x0001, 0x9b26, 0x17ba, 0x7657};

static int code_word_mask[16] = {0x8000,
                                0x4000,
                                0x2000,
                                0x1000,

```

```

                                0x0800,
                                0x0400,
                                0x0200,
                                0x0100,
                                0x0080,
                                0x0040,
                                0x0020,
                                0x0010,
                                0x0008,
                                0x0004,
                                0x0002,
                                0x0001};

void bch_64_encode( int in_word, int * code_word_out )
{
    unsigned int temp_word;
    char i, j;

    for(j=0;j<4;j++)
    {
        code_word_out[j] = 0;
        for(i=0;i<16;i++)
        {
            temp_word = in_word & code_word_mask[i];
            if(temp_word >= 1)
                code_word_out[j] = code_word_out[j] ^ matrix[i][j];
        }
    }
}

```

A.2.2 CRC Code Module

This module consists of two files:

crc.h which is an include file, and

crc.c which is the function definition for the cyclic redundancy encoder.

```

/*****
    Name:   crc.h
    Author: Al Wilson
    Date:   8/4/93
    History: 8/4/93  New
  *****/

```

This module declares three functions:

```

    crc_ccitt ( pointer )   encodes the header parity check
    crc_9 ( num, pointer )  encodes the confirmed data block parity check
    crc_32 ( length, pointer ) encodes the data parity check

```

The pointer points to a list of integers. The 32 bit pointer is a variable length list. The first two functions return a 16 bit and a 9 bit result respectively. The last function returns a 32 bit long integer result.

```

*****/

#ifndef CRC_H
#define CRC_H

int crc_ccitt ( int * );
int crc_9 ( int, int * );
long int crc_32 ( int, int * );

#endif

```

```

/*****
    Name:   crc.c
    Author: Al Wilson
    Date:   8/4/93
    History: 8/4/93  New
  *****/

```

This module contains three functions:

```

    crc_ccitt ( pointer )   encodes the header parity check
    crc_9 ( num, pointer )  encodes the confirmed data block parity check
    crc_32 ( length, pointer ) encodes the data parity check

```

The pointer points to a list of integers. The 32 bit pointer is a variable length list. The first two functions return a 16 bit and a 9 bit result respectively. The last function returns a 32 bit long integer result.

The arguments must follow the following rules.

```

    crc_ccitt  pointer  -- must point to an array of 10 octets

    crc_9      num      -- is a 7-bit serial number for the data block
                       pointer -- must point to an array of 16 octets

    crc_32     length   -- indicates the length of the array, 1..512
                       pointer -- must point to an array of (length) octets

```

```

*****/

#include "crc.h"

#define G_16 ((1 << 12) | (1 << 5) | 1)
#define G_9  ((1 << 6) | (1 << 4) | (1 << 3) | 1)
#define G_32 0x04c11db7

```

```

int crc_ccitt ( int * ptr )
{
    int temp;
    int i, j;

    temp = 0;
    for ( i=0; i<10; i++ )
    {
        for ( j=7; j>=0; j-- )
        {
            if ( ( (temp >> 15) ^ ((*ptr+i) >> j) ) & 1 )
                temp = (temp << 1) ^ G_16;
            else
                temp = temp << 1;
        }
    }
    temp = ( temp & 0xffff ) ^ 0xffff;
    return ( temp );
}

int crc_9 ( int num, int * ptr )
{
    int temp;
    int i, j;

    temp = 0;
    for ( j=6; j>=0; j-- )
    {
        if ( ( (temp >> 8) ^ (num >> j) ) & 1 )
            temp = (temp << 1) ^ G_9;
        else
            temp = temp << 1;
    }
    for ( i=0; i<16; i++ )
    {
        for ( j=7; j>=0; j-- )
        {
            if ( ( (temp >> 8) ^ ((*ptr+i) >> j) ) & 1 )
                temp = (temp << 1) ^ G_9;
            else
                temp = temp << 1;
        }
    }
    temp = ( temp & 0x1fff ) ^ 0x1fff;
    return ( temp );
}

long int crc_32 ( int length, int * ptr )
{
    long int temp;
    int i, j;

    temp = 0;
    for ( i=0; i<length; i++ )
    {
        for ( j=7; j>=0; j-- )
        {
            if ( ( (temp >> 31) ^ ((*ptr+i) >> j) ) & 1 )
                temp = ( temp << 1 ) ^ G_32;
            else
                temp = temp << 1;
        }
    }
    temp = temp ^ 0xffffffff;
    return ( temp );
}

```

A.2.3 Cyclic Code Module

This module consists of two files:

cyclic.h	which is an include file, and
cyclic.c	which is the function definition for the cyclic encoder.

```

/*****

    Name:  cyclic.h
    Author: Al Wilson
    Date:  7/28/93
    History: 7/28/93  New

This module declares one function:
    cyclic_encode ( in, out )    encodes (16,8,5) shortened cyclic code

*****/

#ifndef CYCLIC_H
#define CYCLIC_H

int cyclic_encode ( int );

#endif

/*****

    Name:  cyclic.c
    Author: Al Wilson
    Date:  7/28/93
    History: 7/28/93  New

This module contains one functions:
    cyclic_encode ( in )    encodes (16,8,5) shortened cyclic code
The function takes an 8-bit integer as an argument, and returns a 16-bit
result.

*****/

#include "cyclic.h"

static int matrix[8] = {0x804e,
                        0x4027,
                        0x208f,
                        0x10db,
                        0x08f1,
                        0x04e4,
                        0x0272,
                        0x0139};

static int code_word_mask[8] = {0x80,
                                0x40,
                                0x20,
                                0x10,
                                0x08,
                                0x04,
                                0x02,
                                0x01};

int cyclic_encode( int in_word )
{
    int temp_word;

```

```
int code_word_out;  
char i;  
  
code_word_out = 0;  
for(i=0;i<8;i++)  
{  
    temp_word = in_word & code_word_mask[i];  
    if(temp_word >= 1)  
        code_word_out = code_word_out ^ matrix[i];  
}  
return ( code_word_out );  
}
```

Limited Use Only

A.2.4 Golay Code Module

This module consists of two files:

golay.h which is an include file, and
golay.c which is the function definition for the Golay encoders.

```

/*****

    Name:   golay.h
    Author: LeCocq
    Date:   7/28/93
    History: 6/28/93  New
            7/28/93  Revised by ALW to compile on HP

```

This module declares three functions:

```

    golay_18_encode ( word )    encodes (18, 6,8) shortened Golay code
    golay_23_encode ( word )    encodes (23,12,7) standard Golay code
    golay_24_encode ( word )    encodes (24,12,8) extended Golay code

```

The functions return a long int value that is 18, 23, or 24 bits long for the respective function.

```

*****/

#ifndef GOLAY_H
#define GOLAY_H

long int golay_18_encode ( int );
long int golay_23_encode ( int );
long int golay_24_encode ( int );

#endif

```

```

/*****

    Name:   golay.c
    Author: LeCocq
    Date:   7/28/93
    History: 6/28/93  New
            7/28/93  Revised by ALW to compile on HP

```

This module contains three functions:

```

    golay_18_encode ( word )    encodes (18, 6,8) shortened Golay code
    golay_23_encode ( word )    encodes (23,12,7) standard Golay code
    golay_24_encode ( word )    encodes (24,12,8) extended Golay code

```

The functions return a long int value that is 18, 23, or 24 bits long for the respective function.

```

*****/

#include "golay.h"

static long int matrix[12] = {040006165,
                              020003073,
                              010007550,
                              04003664,
                              02001732,
                              01006631,
                              0403315,
                              0201547,
                              0106706,
                              045227,
                              024476,
                              014353};

```



```

static int code_word_mask[12] = {04000,
                                02000,
                                01000,
                                0400,
                                0200,
                                0100,
                                040,
                                020,
                                010,
                                04,
                                02,
                                01};

long int golay_18_encode(int code_word_in)
{
    return ( golay_24_encode ( code_word_in & 077 ) );
}

long int golay_23_encode(int code_word_in)
{
    return ( golay_24_encode ( code_word_in ) >> 1 );
}

long int golay_24_encode(int code_word_in)
{
    long int code_word_out = 0;
    long int temp_word;
    char i;

    for(i=0;i<12;i++)
    {
        temp_word = code_word_in & code_word_mask[i];
        if(temp_word >= 1)
            code_word_out = code_word_out ^ matrix[i];
    }

    return(code_word_out);
}

```

A.2.5 Hamming Code Module

This module consists of two files:

ham.h which is an include file, and

ham.c which is the function definition for the Hamming encoders.

```

/*****
    Name:    ham.h
    Author:  LeCocq
    Date:    7/28/93
    History: 6/28/93  New
            7/28/93  Revised by ALW to compile on HP
  *****/

```

This module declares two functions:

```

    hamming_10_encode ( word )    encodes (10, 6,3) shortened Hamming code
    hamming_15_encode ( word )    encodes (15,11,3) standard Hamming code

```

The functions return a long int value that is 18, 23, or 24 bits long for the respective function.

```

*****/

#ifndef HAM_H
#define HAM_H

long int hamming_10_encode ( int );
long int hamming_15_encode ( int );

#endif

```

```

/*****
    Name:    ham.c
    Author:  LeCocq
    Date:    7/28/93
    History: 6/28/93  New
            7/28/93  Revised by ALW to compile on HP
  *****/

```

This module contains two functions:

```

    hamming_10_encode ( word )    encodes (10, 6,3) shortened Hamming code
    hamming_15_encode ( word )    encodes (15,11,3) standard Hamming code

```

The functions return a long int value that is 18, 23, or 24 bits long for the respective function.

```

*****/

#include "ham.h"

static long int matrix_10[6]    = {0x20e,0x10d,0x8b,0x47,0x23,0x1c};
static int code_word_mask_10[6] = {0x20 ,0x10 ,0x8, 0x4, 0x2, 0x1};

long int hamming_10_encode(int code_word_in)
{
    long int code_word_out = 0;
    long int temp_word;
    char i;

    for(i=0;i<6;i++)
    {
        temp_word = code_word_in & code_word_mask_10[i];
        if(temp_word >= 1)
            code_word_out = code_word_out ^ matrix_10[i];
    }
}

```

```

    }

    return(code_word_out);
}

static long int matrix_15[11] = {0x400f,0x200e,0x100d,0x80c,0x40b,0x20a,0x109,
                                0x87,0x46,0x25,0x13};
static int code_word_mask_15[11] = {0x400,0x200,0x100,0x80,0x40,0x20,0x10,
                                    0x08,0x04,0x02,0x01};

long int hamming_15_encode(int code_word_in)
{
    long int code_word_out = 0;
    long int temp_word;
    char i;

    for(i=0;i<11;i++)
    {
        temp_word = code_word_in & code_word_mask_15[i];
        if(temp_word >= 1)
            code_word_out = code_word_out ^ matrix_15[i];
    }

    return(code_word_out);
}

```

A.2.6 Reed-Solomon Code Module

This module consists of two files:

reed_solomon.h which is an include file, and
reed_solomon.c which is the function definition for the RS encoders.

```
/******
```

```
    Name:  reed_solomon.h
    Author: Jim Holthaus
    Date:   7/28/93
    History: 6/11/93 Original by Jim Holthaus
            7/28/93 Revised by ALW to compile on HP
```

This module declares three functions:

```
    reed_solomon_24_12 ( in, out )    encodes (24,12,13) Reed-Solomon code
    reed_solomon_24_16 ( in, out )    encodes (24,16, 9) Reed-Solomon code
    reed_solomon_36_20 ( in, out )    encodes (36,20,17) Reed-Solomon code
```

The functions require a pointer to an array for the in and out variables.
The input variable is a pointer to a vector of length K and the output
variable is a pointer to a vector of length N, where N & K are the length
and dimension of the Reed-Solomon code.

```
*****/
```

```
#ifndef REED_SOLOMON_H
```

```
#define REED_SOLOMON_H
```

```
void reed_solomon_24_12 ( int *, int * );
void reed_solomon_24_16 ( int *, int * );
void reed_solomon_36_20 ( int *, int * );
```

```
#endif
```

```
/******
```

```
    Name:  reed_solomon.c
    Author: Jim Holthaus
    Date:   7/28/93
    History: 6/11/93 Original by Jim Holthaus
            7/28/93 Revised by ALW to be a module
```

This module contains three functions:

```
    reed_solomon_24_12 ( in, out )    encodes (24,12,13) Reed-Solomon code
    reed_solomon_24_16 ( in, out )    encodes (24,16, 9) Reed-Solomon code
    reed_solomon_36_20 ( in, out )    encodes (36,20,17) Reed-Solomon code
```

The functions require a pointer to an array for the in and out variables.
The input variable is a pointer to a vector of length K and the output
variable is a pointer to a vector of length N, where N & K are the length
and dimension of the Reed-Solomon code.

```
*****/
```

```
#include "reed_solomon.h"
```

```
static int RS_24_12[12][24] = { /*generator matrix for RS (24,12,13)*/
1,0,0,0, 0,0,0,0, 0,0,0,0, 062,044,003,025, 014,016,027,003, 053,004,036,047,
0,1,0,0, 0,0,0,0, 0,0,0,0, 011,012,011,011, 016,064,067,055, 001,076,026,073,
0,0,1,0, 0,0,0,0, 0,0,0,0, 003,001,005,075, 014,006,020,044, 066,006,070,066,
0,0,0,1, 0,0,0,0, 0,0,0,0, 021,070,027,045, 016,067,023,064, 073,033,044,021,
0,0,0,0, 1,0,0,0, 0,0,0,0, 030,022,003,075, 015,015,033,015, 051,003,053,050,
0,0,0,0, 0,1,0,0, 0,0,0,0, 001,041,027,056, 076,064,021,053, 004,025,001,012,
0,0,0,0, 0,0,1,0, 0,0,0,0, 061,076,021,055, 076,001,063,035, 030,013,064,070,
0,0,0,0, 0,0,0,1, 0,0,0,0, 024,022,071,056, 021,035,073,042, 057,074,043,076,
```

```

0,0,0,0, 0,0,0,0, 1,0,0,0, 072,042,005,020, 043,047,033,056, 001,016,013,076,
0,0,0,0, 0,0,0,0, 0,1,0,0, 072,014,065,054, 035,025,041,016, 015,040,071,026,
0,0,0,0, 0,0,0,0, 0,0,1,0, 073,065,036,061, 042,022,017,004, 044,020,025,005,
0,0,0,0, 0,0,0,0, 0,0,0,1, 071,005,055,003, 071,034,060,011, 074,002,041,050};

```

```

static int RS_24_16[16][24] = { /*generator matrix for RS (24,16,9)*/
1,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 051,045,067,015, 064,067,052,012,
0,1,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 057,025,063,073, 071,022,040,015,
0,0,1,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 005,001,031,004, 016,054,025,076,
0,0,0,1, 0,0,0,0, 0,0,0,0, 0,0,0,0, 073,007,047,014, 041,077,047,011,
0,0,0,0, 1,0,0,0, 0,0,0,0, 0,0,0,0, 075,015,051,051, 017,067,017,057,
0,0,0,0, 0,1,0,0, 0,0,0,0, 0,0,0,0, 020,032,014,042, 075,042,070,054,
0,0,0,0, 0,0,1,0, 0,0,0,0, 0,0,0,0, 002,075,043,005, 001,040,012,064,
0,0,0,0, 0,0,0,1, 0,0,0,0, 0,0,0,0, 024,074,015,072, 024,026,074,061,
0,0,0,0, 0,0,0,0, 1,0,0,0, 0,0,0,0, 042,064,007,022, 061,020,040,065,
0,0,0,0, 0,0,0,0, 0,1,0,0, 0,0,0,0, 032,032,055,041, 057,066,021,077,
0,0,0,0, 0,0,0,0, 0,0,1,0, 0,0,0,0, 065,036,025,007, 050,016,040,051,
0,0,0,0, 0,0,0,0, 0,0,0,1, 0,0,0,0, 064,006,054,032, 076,046,014,036,
0,0,0,0, 0,0,0,0, 0,0,0,0, 1,0,0,0, 062,063,074,070, 005,027,037,046,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,1,0,0, 055,043,034,071, 057,076,050,064,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,1,0, 024,023,023,005, 050,070,042,023,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,1, 067,075,045,060, 057,024,006,026};

```

```

static int RS_36_20[20][36] = { /*generator matrix for RS (36,20,17)*/
1,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 074,037,034,006, 002,007,044,064,
026,014,026,044, 054,013,077,005,
0,1,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 004,017,050,024, 011,005,030,057,
033,003,002,002, 015,016,025,026,
0,0,1,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 007,023,037,046, 056,075,043,045,
055,021,050,031, 045,027,071,062,
0,0,0,1, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 026,005,007,063, 063,027,063,040,
006,004,040,045, 047,030,075,007,
0,0,0,0, 1,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 023,073,073,041, 072,034,021,051,
067,016,031,074, 011,021,012,021,
0,0,0,0, 0,1,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 024,051,025,023, 022,041,074,066,
074,065,070,036, 067,045,064,001,
0,0,0,0, 0,0,1,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 052,033,014,002, 020,006,014,025,
052,023,035,074, 075,075,043,027,
0,0,0,0, 0,0,0,1, 0,0,0,0, 0,0,0,0, 0,0,0,0, 055,062,056,025, 073,060,015,030,
013,017,020,002, 070,055,014,047,
0,0,0,0, 0,0,0,0, 1,0,0,0, 0,0,0,0, 0,0,0,0, 054,051,032,065, 077,012,054,013,
035,032,056,012, 075,001,072,063,
0,0,0,0, 0,0,0,0, 0,1,0,0, 0,0,0,0, 0,0,0,0, 074,041,030,041, 043,022,051,006,
064,033,003,047, 027,012,055,047,
0,0,0,0, 0,0,0,0, 0,0,1,0, 0,0,0,0, 0,0,0,0, 054,070,011,003, 013,022,016,057,
003,045,072,031, 030,056,035,022,
0,0,0,0, 0,0,0,0, 0,0,0,1, 0,0,0,0, 0,0,0,0, 051,007,072,030, 065,054,006,021,
036,063,050,061, 064,052,001,060,
0,0,0,0, 0,0,0,0, 0,0,0,0, 1,0,0,0, 0,0,0,0, 001,065,032,070, 013,044,073,024,
012,052,021,055, 012,035,014,072,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,1,0,0, 0,0,0,0, 011,070,005,010, 065,024,015,077,
022,024,024,074, 007,044,007,046,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,1,0, 0,0,0,0, 006,002,065,011, 041,020,045,042,
046,054,035,012, 040,064,065,033,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,1, 0,0,0,0, 034,031,001,015, 044,064,016,024,
052,016,006,062, 020,013,055,057,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 1,0,0,0, 063,043,025,044, 077,063,017,017,
064,014,040,074, 031,072,054,006,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,1,0,0, 071,021,070,044, 056,004,030,074,
004,023,071,070, 063,045,056,043,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,1,0, 002,001,053,074, 002,014,052,074,
012,057,024,063, 015,042,052,033,
0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,0, 0,0,0,1, 034,035,002,023, 021,027,022,033,
064,042,005,073, 051,046,073,060};

```

```
/* Function Prototypes */
```

```
int multgf26( int b, int c);
```

```
void reed_solomon_24_12 ( int *in_word, int *code_word )
```

```
{
    int temp, i, j;
```

```

    for(i=0; i<24; i++)
    {
        temp = 0;
        for(j=0; j<12; j++)
        {
            temp ^= multgf26 ( in_word[j], RS_24_12[j][i] );
        }
        code_word[i] = temp&0x03f;
    }
}

void reed_solomon_24_16 ( int *in_word, int *code_word )
{
    int temp, i, j;

    for(i=0; i<24; i++)
    {
        temp = 0;
        for(j=0; j<16; j++)
        {
            temp ^= multgf26 ( in_word[j], RS_24_16[j][i] );
        }
        code_word[i] = temp&0x03f;
    }
}

void reed_solomon_36_20 ( int *in_word, int *code_word )
{
    int temp, i, j;

    for(i=0; i<36; i++)
    {
        temp = 0;
        for(j=0; j<20; j++)
        {
            temp ^= multgf26 ( in_word[j], RS_36_20[j][i] );
        }
        code_word[i] = temp&0x03f;
    }
}

/* Function multgf26                                     */
/* This function performs the Galois (2^6) multiply      */
/*                                                        */

int multgf26(int b, int c)
{
    int aout,a5temp;
    int tempa,tempb,tempc;
    int t[6], a[6];
    int i,j;

    a[0]=a[1]=a[2]=a[3]=a[4]=a[5]=0;
    for( i=0; i<6; i++)
    {
        tempc = ( c>>(5-i) ) & 0x01;
        for( j=0; j<6; j++)
        {
            tempb = ( b>>j ) & 0x01;
            t[j] = tempc & tempb;
        }
        a5temp = a[5];
        a[5] = t[5] ^ a[4];
        a[4] = t[4] ^ a[3];
        a[3] = t[3] ^ a[2];
        a[2] = t[2] ^ a[1];
        a[1] = t[1] ^ a[0] ^ a5temp;
        a[0] = t[0] ^ a5temp;
    }
    aout = 0;
    for( i=0; i<6; i++ )
        aout += (a[i]<<i);
}

```

```
    return(aout);  
}
```

Limited Use Only

A.2.7 Trellis Code Module

This module consists of two files:

trellis.h	which is an include file, and
trellis.c	which is the function definition for the trellis encoders.

```

/*****

    Name:   trellis.h
    Author: Al Wilson
    Date:   8/5/93
    History: 8/5/93  New

This module declares two functions:
    trellis_3_4_encode ( in, out )    encodes the rate 3/4 trellis code
    trellis_1_2_encode ( in, out )    encodes the rate 1/2 trellis code
The input and output arguments are pointers a integer vectors.  The input
arguments must point to octets while the output arguments point to an array
of 14 integers with 14 bits in each (196 bits in total).

*****/

#ifndef TRELLIS_H
#define TRELLIS_H

void trellis_3_4_encode(int *, int *);
void trellis_1_2_encode(int *, int *);

# endif

/*****

    Name:   trellis.c
    Author: Al Wilson
    Date:   9/5/93
    History: 8/5/93  New
            9/5/93  Fixed the initialization of dibits & tribits arrays.

This module defines two functions:
    trellis_3_4_encode ( in, out )    encodes the rate 3/4 trellis code
    trellis_1_2_encode ( in, out )    encodes the rate 1/2 trellis code
The input and output arguments are pointers a integer vectors.  The input
arguments must point to octets while the output arguments point to an array
of 14 integers with 14 bits in each (196 bits in total).

*****/

#include "trellis.h"
#include "vector_operations.h"

static int sigIndx_1_2[4][4] = {
    0, 15, 12,  3,
    4, 11,  8,  7,
    13, 2,  1, 14,
    9,  6,  5, 10};

static int sigIndx_3_4[8][8] = {
    0,  8,  4, 12,  2, 10,  6, 14,
    4, 12,  2, 10,  6, 14,  0,  8,
    1,  9,  5, 13,  3, 11,  7, 15,
    5, 13,  3, 11,  7, 15,  1,  9,
    3, 11,  7, 15,  1,  9,  5, 13,
    7, 15,  1,  9,  5, 13,  3, 11,
    2, 10,  6, 14,  0,  8,  4, 12,

```



```

    6, 14, 0, 8, 4, 12, 2, 10};

static int signalP[16][2] = {
    0, 2, 2, 2,
    1, 3, 3, 3,
    3, 2, 1, 2,
    2, 3, 0, 3,
    3, 1, 1, 1,
    2, 0, 0, 0,
    0, 1, 2, 1,
    1, 0, 3, 0};

static int intlvRule[98] = {
    0, 1, 8, 9, 16, 17, 24, 25, 32, 33, 40, 41,
    48, 49, 56, 57, 64, 65, 72, 73, 80, 81, 88, 89, 96, 97,
    2, 3, 10, 11, 18, 19, 26, 27, 34, 35, 42, 43,
    50, 51, 58, 59, 66, 67, 74, 75, 82, 83, 90, 91,
    4, 5, 12, 13, 20, 21, 28, 29, 36, 37, 44, 45,
    52, 53, 60, 61, 68, 69, 76, 77, 84, 85, 92, 93,
    6, 7, 14, 15, 22, 23, 30, 31, 38, 39, 46, 47,
    54, 55, 62, 63, 70, 71, 78, 79, 86, 87, 94, 95 };

void trellis_3_4_encode(int * inBlock, int * outBlock)
{
    int i, k;
    int state, sig;
    int tri_bits[49];
    int di_bits[98];

    for ( k=0; k<49; k++ ) tri_bits[k] = 0;
    for ( k=0; k<18; k++ ) tri_bits[k] = inBlock[k];
    for ( k=0; k<49; k++ ) shift_vector_right( 49-k, 8, 5, &tri_bits[k]);

    state = 0;
    for ( k=0; k<49; k++ )
    {
        sig = sigIndx_3_4[state][ tri_bits[k] ];
        di_bits[2*k] = signalP[sig][0];
        di_bits[2*k + 1] = signalP[sig][1];
        state = tri_bits[k];
    }

    for ( k=0; k<14; k++ )
    {
        sig = 0;
        for ( i=0; i<7; i++ )
        {
            sig = (sig << 2) | di_bits[intlvRule[7*k + i]];
        }
        *(outBlock + k) = sig;
    }
}

void trellis_1_2_encode(int * inBlock, int * outBlock)
{
    int i, k;
    int state, sig;
    int di_bits_in[49];
    int di_bits[98];

    for ( k=0; k<49; k++ ) di_bits_in[k] = 0;
    for ( k=0; k<12; k++ ) di_bits_in[k] = inBlock[k];
    for ( k=0; k<49; k++ ) shift_vector_right( 49-k, 8, 6, &di_bits_in[k]);

    state = 0;
    for ( k=0; k<49; k++ )
    {
        sig = sigIndx_1_2[state][ di_bits_in[k] ];
        di_bits[2*k] = signalP[sig][0];
        di_bits[2*k + 1] = signalP[sig][1];
        state = di_bits_in[k];
    }
}

```

```
for ( k=0; k<14; k++ )
{
    sig = 0;
    for ( i=0; i<7; i++ )
    {
        sig = (sig << 2) | di_bits[intlvRule[7*k + i]];
    }
    *(outBlock + k) = sig;
}
}
```

Limited Use Only

A.2.8 Voice Code Module

This module consists of two files:

voice_word.h which is an include file, and

voice_word.c which is the function definition for the voice encoders.

These are useful functions to perform the elaborate interleaving for IMBE voice words.

```

/*****

```

```

    Name:  voice_word.h
    Author: Al Wilson
    Date:   7/29/93
    History: 7/29/93  New

```

This module declares two functions:

```

    voice_encode ( u_vector, s_vector )
    voice_decode ( s_vector, u_vector )

```

The functions take arguments that are pointers to arrays.

The u_vector points to eight elements (called u_0, u_1, ... u_7 in the CAI). Some of the elements of the u_vector are 12 bits, some are 11 bits, and the last is 7 bits.

The s_vector points to the 144 bit interleaved code word, in a vector consisting of nine elements, of 16 bits each.

```

*****/

```

```

#ifndef VOICE_WORD_H

```

```

#define VOICE_WORD_H

```

```

void voice_encode(int * , int * );
void voice_decode(int * , int * );

```

```

#endif

```

```

/*****

```

```

    Name:  voice_word.c
    Author: Al Wilson
    Date:   7/29/93
    History: 7/29/93  New
           2/8/95   revised for machine independence

```

This module contains two functions:

```

    voice_encode ( u_vector, s_vector )
    voice_decode ( s_vector, u_vector )

```

The functions take arguments that are pointers to arrays.

The u_vector points to eight elements (called u_0, u_1, ... u_7 in the CAI). Some of the elements of the u_vector are 12 bits, some are 11 bits, and the last is 7 bits.

The s_vector points to the 144 bit interleaved code word, in a vector consisting of nine elements, of 16 bits each.

The voice_encode function reads in the u_vector and writes out the s_vector. The voice_decode function reads in the s_vector and writes out the u_vector.

The voice_decode function does NOT correct errors. If merely de-interleaves the vector, computes and removes the coset codes, then discards the parities and produces the u_vector result.

```

*****/

#include "voice_word.h"
#include "golay.h"
#include "ham.h"

/* This array contains the interleave schedule for IMBE. */
static int interleave_schedule[72][4] =
{
    0, 22, 1, 21,
    2, 20, 3, 19,
/* 2*/ 4, 10, 5, 1,
    1, 20, 0, 21,
    3, 18, 2, 19,
/* 5*/ 5, 0, 4, 9,
    0, 20, 1, 19,
    2, 18, 3, 17,
/* 8*/ 4, 8, 6, 14,
    1, 18, 0, 19,
    3, 16, 2, 17,
/* 11*/ 6, 13, 4, 7,
    0, 18, 1, 17,
    2, 16, 3, 15,
/* 14*/ 4, 6, 6, 12,
    1, 16, 0, 17,
    3, 14, 2, 15,
/* 17*/ 6, 11, 4, 5,
    0, 16, 1, 15,
    2, 14, 3, 13,
/* 20*/ 4, 4, 6, 10,
    1, 14, 0, 15,
    3, 12, 2, 13,
/* 23*/ 6, 9, 4, 3,
    0, 14, 1, 13,
    2, 12, 3, 11,
/* 26*/ 4, 2, 6, 8,
    1, 12, 0, 13,
    3, 10, 2, 11,
/* 29*/ 6, 7, 4, 1,
    0, 12, 1, 11,
    2, 10, 3, 9,
/* 32*/ 4, 0, 6, 6,
    1, 10, 0, 11,
    3, 8, 2, 9,
/* 35*/ 6, 5, 5, 14,
    0, 10, 1, 9,
    2, 8, 3, 7,
/* 38*/ 5, 13, 6, 4,
    1, 8, 0, 9,
    3, 6, 2, 7,
/* 41*/ 6, 3, 5, 12,
    0, 8, 1, 7,
    2, 6, 3, 5,
/* 44*/ 5, 11, 6, 2,
    1, 6, 0, 7,
    3, 4, 2, 5,
/* 47*/ 6, 1, 5, 10,
    0, 6, 1, 5,
    2, 4, 3, 3,
/* 50*/ 5, 9, 6, 0,
    1, 4, 0, 5,
    3, 2, 2, 3,
/* 53*/ 7, 6, 5, 8,
    0, 4, 1, 3,
    2, 2, 3, 1,
/* 56*/ 5, 7, 7, 5,
    1, 2, 0, 3,
    3, 0, 2, 1,
/* 59*/ 7, 4, 5, 6,
    0, 2, 1, 1,
    2, 0, 4, 14,
/* 62*/ 5, 5, 7, 3,

```

```

        1,  0,  0,  1,
        4, 13,  3, 22,
/* 65*/ 7,  2,  5,  4,
        0,  0,  2, 22,
        3, 21,  4, 12,
/* 68*/ 5,  3,  7,  1,
        2, 21,  1, 22,
        4, 11,  3, 20,
/* 71*/ 7,  0,  5,  2};

void voice_encode(int * u, int * s)
{
    long int c[8]; /* coset code words */
    long int m[8]; /* PN vectors      */
    long int p;
    int i, j, symbol;

    c[0] = golay_23_encode ( u[0] );
    c[1] = golay_23_encode ( u[1] );
    c[2] = golay_23_encode ( u[2] );
    c[3] = golay_23_encode ( u[3] );
    c[4] = hamming_15_encode ( u[4] );
    c[5] = hamming_15_encode ( u[5] );
    c[6] = hamming_15_encode ( u[6] );
    c[7] = u[7];

    p = 16 * u[0];
    for ( i=1; i<4; i++ )
    {
        m[i] = 0;
        for ( j=0; j<23; j++ )
        {
            p = (173 * p + 13849) & 0xffff;
            m[i] = m[i] << 1;
            if ( p & 0x8000 ) m[i]++;
        }
        c[i] = c[i] ^ m[i];
    }
    for ( i=4; i<7; i++ )
    {
        m[i] = 0;
        for ( j=0; j<15; j++ )
        {
            p = (173 * p + 13849) & 0xffff;
            m[i] = m[i] << 1;
            if ( p & 0x8000 ) m[i]++;
        }
        c[i] = c[i] ^ m[i];
    }

    symbol = 0;
    for ( i=0; i<9; i++ )
    {
        s[i] = 0;
        for ( j=0; j<8; j++ )
        {
            s[i] = s[i] << 2;
            s[i] += ( ( c[interleave_schedule[symbol][0]] >>
                        interleave_schedule[symbol][1]
                        ) << 1 ) & 2
                    +
                    ( ( c[interleave_schedule[symbol][2]] >>
                        interleave_schedule[symbol][3]
                        ) & 1
                    );
            symbol++;
        }
    }
}

```

```

void voice_decode(int * s, int * u)
{
    long int c[8];    /* coset code words */
    long int m[8];    /* PN vectors      */
    long int p;
    int i, j, symbol;
    long int t;

    for ( i=0; i<8; i++ ) c[i] = 0;
    symbol = 0;
    for ( i=0; i<9; i++ )
    {
        for ( j=7; j>=0; j-- )
        {
            t = ( s[i] >> (2 * j) ) & 3; /* dibit symbol */
            c[interleave_schedule[symbol][0]] +=
                (t >> 1) << interleave_schedule[symbol][1];
            c[interleave_schedule[symbol][2]] +=
                (t & 1) << interleave_schedule[symbol][3];
            symbol++;
        }
    }

    u[0] = (c[0] >> 11) & 0xfff;
    u[7] = c[7];

    p = 16 * u[0];
    for ( i=1; i<4; i++ )
    {
        m[i] = 0;
        for ( j=0; j<23; j++ )
        {
            p = (173 * p + 13849) & 0xffff;
            m[i] = m[i] << 1;
            if ( p & 0x8000 ) m[i]++;
        }
        c[i] = c[i] ^ m[i];
    }
    for ( i=4; i<7; i++ )
    {
        m[i] = 0;
        for ( j=0; j<15; j++ )
        {
            p = (173 * p + 13849) & 0xffff;
            m[i] = m[i] << 1;
            if ( p & 0x8000 ) m[i]++;
        }
        c[i] = c[i] ^ m[i];
    }

    u[1] = (c[1] >> 11) & 0xfff;
    u[2] = (c[2] >> 11) & 0xfff;
    u[3] = (c[3] >> 11) & 0xfff;
    u[4] = (c[4] >> 4) & 0x7ff;
    u[5] = (c[5] >> 4) & 0x7ff;
    u[6] = (c[6] >> 4) & 0x7ff;
}

```

A.2.9 File Input Module

This module consists of two files:

 get_file_io.h which is an include file, and
 get_file_io.c which is the function definition for simple file I/O.

```

/*****

    Name:  get_file_io.h
    Author: Al Wilson
    Date:  8/3/93
    History: 8/3/93 New
            2/8/95 revised for machine independence

This module declares three functions for reading in files:
    get_file_name()    which returns a I/O error if unable to open a file
    get_file_word(leng) which takes an integer argument and returns an integer
    get_file_word_long(leng) which also takes an arg, and returns a long int

The arguments to the last two functions indicate the number of hex digits to
read for the integer.  For get_file_word the argument should be in the range
of 1..4.  For get_file_word_long the argument should be 1..8.

*****/

#ifndef GET_FILE_IO_H
#define GET_FILE_IO_H

FILE *in,*out;

int get_file_name ( void );
int get_file_word ( int );
long int get_file_word_long ( int );

#endif

/*****

    Name:  get_file_io.c
    Author: Al Wilson
    Date:  8/3/93
    History: 8/3/93 New

This module defines three functions for reading in files:
    get_file_name()    which returns a I/O error if unable to open a file
    get_file_word(leng) which takes an integer argument and returns an integer
    get_file_word_long(leng) which also takes an arg, and returns a long int

The arguments to the last two functions indicate the number of hex digits to
read for the integer.  For get_file_word the argument should be in the range
of 1..4.  For get_file_word_long the argument should be 1..8.

*****/

#include <stdio.h>
#include "get_file_io.h"

int get_file_name(void)
{
    char in_file[20],out_file[20];

    printf("\nEnter input file name:\t");
    scanf("%20s",in_file);

```

```

    if((in = fopen(in_file,"rt")) == 0)
    {
        fprintf(stderr,"Cannot open input file\n");
        return (0);
    }

    printf("\nEnter output file name:\t");
    scanf("%20s",out_file);
    out = fopen(out_file,"w+t");
    return(1);
}

int get_file_word(int field_size)
{
    int word;

    switch ( field_size )
    {
        case 1: fscanf(in, "%1x", &word); break;
        case 2: fscanf(in, "%2x", &word); break;
        case 3: fscanf(in, "%3x", &word); break;
        case 4: fscanf(in, "%4x", &word); break;
    }
    return(word);
}

long int get_file_word_long(int field_size)
{
    long int word;

    switch ( field_size )
    {
        case 1:
        case 2:
        case 3:
        case 4: word = get_file_word(field_size); break;
        case 5: fscanf(in, "%5lx", &word); break;
        case 6: fscanf(in, "%6lx", &word); break;
        case 7: fscanf(in, "%7lx", &word); break;
        case 8: fscanf(in, "%8lx", &word); break;
    }
    return(word);
}

```


A.2.10 Vector Operation Module

This module consists of two files:

vector_operations.h which is an include file, and

vector_operations.c which defines simple vector operations.

These vector operations are useful for manipulating long strings of bits which are created by the codes in the Common Air Interface.

```

/*****
    Name:  vector_operations.h
    Author: Al Wilson
    Date:  8/3/93
    History: 8/3/93  New
  *****/

```

This module declares two functions for operations on vectors:

```

    shift_vector_right ( int length, int element_length, int shift, int * ptr )
    extract_dibit ( int length, int element_length, int * ptr )

```

```

*****/

#ifndef VECTOR_OPERATIONS_H
#define VECTOR_OPERATIONS_H

void shift_vector_right(int, int, int, int * );
int extract_dibit(int, int, int * );

#endif

```

```

/*****
    Name:  vector_operations.c
    Author: Al Wilson
    Date:  8/3/93
    History: 8/3/93  New
           5/3/94  revised for machine independence
  *****/

```

This module defines two functions for operations on vectors:

```

    shift_vector_right ( int length, int element_length, int shift, int * ptr )
    extract_dibit ( int length, int element_length, int * ptr )

```

The arguments to the two functions are as follows:

```

length      -- indicates the number of elements in the vector
element_length -- indicates the number of bits in each element
shift       -- indicates the number of bits to shift [ to the right ]
ptr         -- pointer to the first element of the vector

```

The shift_vector_right function has the following restrictions:

```

length      -- must be >= 1
element_length -- must be 1..16
shift       -- must be < element_length

```

The function operates by shifting each element to the right, starting with the first element pointed to by the pointer. It takes the LSBs which shift out of the element, and shifts them into the next element, starting at the MSB indicated by the element_length argument. The bits shifted out of the last element in the vector are discarded.

The extract_dibit function uses the shift_vector_right function to extract the 2 MSBs of the first vector element (pointed to by the ptr argument). This is done by shifting to the right (element_length - 2) bits. The result in the first element is simply the 2 MSBs, in the LSB positions. This is then returned as the extracted dibit. The vector is then left shifted by moving each element from position i+1 to position i.

To prevent the last element in the vector from losing 2 bits, an additional element is assumed to exist in the vector beyond the indicated length. Consequently, the actually allocated length of the vector must be at least (length + 1).

```

*****/

#include "vector_operations.h"

void shift_vector_right(int vector_length, /* should be >= 1 */
                       int element_length, /* should be 1..16 */
                       int shift, /* should be < element_length */
                       int * pointer)

{
    int i, temp;

    for ( i=vector_length-1; i>=0; i-- )
    {
        if ( i > 0 ) temp = *(pointer+i-1);
        else temp = 0;
        temp &= (1 << shift) - 1;
        *(pointer+i) = (unsigned int)(*(pointer+i)) >> shift;
        *(pointer+i) = (*(pointer+i)) | (temp << (element_length - shift));
    }
}

int extract_dibit ( int vector_length, /* should be >= 1 */
                   int element_length, /* should be 1..16 */
                   int * pointer )

{
    int temp, i;

    *(pointer+vector_length) = 0; /* vector should have one extra element */
    shift_vector_right ( vector_length+1, element_length,
                        element_length-2, pointer );
    temp = *pointer;
    for ( i=0; i<vector_length; i++ )
        *(pointer + i) = *(pointer + i + 1);
    *(pointer+vector_length) = 0; /* clean up the residue */
    return ( temp & 0x3 );
}

```

A.3 Voice Data Unit Programs

These are a collection of programs which use the primitive modules in Section A.2 to encode complete voice data units.

A.3.1 Voice Header Data Unit

This is a program to encode the Header Data Unit for a voice message.

```

/*****
    Name:  voice_header.c
    Author: Al Wilson
    Date:  9/5/93
    History: 7/30/93  New
            9/5/93  Fixed initialization of hex_bits_in array.

This program is for testing the voice formats for the CAI document.  It
accepts as input a file of values, in hexadecimal, in the following order:
    Network Access Code ( 1  3 digits)
    Status Symbols      (11  1 digit )
    Message Indicator   ( 1  5 values, 4 digits each, except the first with 2)
    Manufacturer ID     ( 1  2 digits)
    Algorithm ID        ( 1  2 digits)
    Key ID              ( 1  4 digits)
    Talk Group ID       ( 1  4 digits)

The program then generates a header data unit with the appropriate code
words.  The output is formatted in hexadecimal, with 3 digit groups.

*****/

#include "golay.h"
#include "bch.h"
#include "reed_solomon.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

#define DUID 0x0 /* Data Unit ID for Header */

int main(void)
{
    int nac, ss[11], mi[5], algid, kid, mfid, tgid;
    int header[11][6]; /* 11 microslots, 6 ints of 12 bits each per slot */
    int error_state, i, j, k;
    int fs[4]; /* Frame Sync */
    int nid[5]; /* Network ID code word */
    int hex_bits_in[20]; /* input to the Reed-Solomon encoder */
    int code_word[55]; /* header code word, constrained to 16-bit elem. */
    long int header_code_word[36];
    int microslot, word;

    error_state = get_file_name();
    if(error_state == 0)
        return (1);

    fprintf(out, "Voice Header Data Unit");
    while(!feof(in) == 0)
    {
        nac = get_file_word(3);
        for ( i=0; i<11; i++ ) ss[i] = get_file_word(1) & 0x3;
        mi[0] = get_file_word(2);

```

```

for ( i=1; i<5; i++ ) mi[i] = get_file_word(4);
mi[0] &= 0xff;
mfid = get_file_word(2);
algid = get_file_word(2);
kid = get_file_word(4);
tgid = get_file_word(4);

for ( i=0; i<11; i++ ) for ( j=0; j<6; j++ ) header[i][j] = 0;

/* Do the Frame Sync */
for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

/* Do the Network ID */
for ( i=0; i<5; i++ ) nid[i] = 0;
bch_64_encode ( (nac << 4) | DUID, nid );

/* To construct the hex bits for input to the Reed-Solomon code
   first load all the arguments into a vector. Note that mi[0]
   has only 8 bits (the LSBs) used. The rest are 16 bits.
*/
for ( i=0; i<5; i++ ) hex_bits_in[i] = mi[i];
hex_bits_in[5] = (mfid << 8) | algid;
hex_bits_in[6] = kid;
hex_bits_in[7] = tgid;
for ( i=8; i<20; i++ ) hex_bits_in[i] = 0;
/* Now we space these out to 6 bits in 20 elements. */
shift_vector_right ( 20, 16, 2, &hex_bits_in[0] );
for ( i=1; i<20; i++ )
    shift_vector_right ( 20-i, 16, 10, &hex_bits_in[i] );
reed_solomon_36_20 ( hex_bits_in, code_word );
for ( i=0; i<36; i++ )
    header_code_word[i] = golay_18_encode(code_word[i]);
/* We now place 2 x 18-bit golay words into 3 x 12-bit code words.
   This uses 54 elements of 12-bits each in the code_word array.
*/
for ( i=0; i<18; i++ )
{
    code_word[3*i] = header_code_word[2*i] >> 6;
    code_word[3*i + 1] = ((header_code_word[2*i] & 0x3f) << 6) |
        (header_code_word[2*i+1] >> 12);
    code_word[3*i + 2] = header_code_word[2*i+1] & 0xfff;
}

microslot = 0; word = 0;
for ( i=0; i<396; i++ )
{
    header[microslot][word] = header[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        header[microslot][word] |= ss[(i-35)/36];
    else if ( i < 24 ) /* Frame Sync */
        header[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        header[microslot][word] |= extract_dibit(4,16,nid);
    else if ( i < 390 ) /* Header Code Word */
        header[microslot][word] |= extract_dibit(54,12,code_word);
    else /* trailing nulls */
        header[microslot][word] |= 0;
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if(feof(in) == 0)
{
    fprintf(out, "\n\nNAC = %03x\nSSym =", nac);
    for ( i=0; i<11; i++ ) fprintf(out, " %1x", ss[i]);
    fprintf(out, "\n\nMI = %02x", mi[0] );
    for ( i=1; i<5; i++ ) fprintf(out, " %04x", mi[i]);
    fprintf(out, "\n\nMFID = %02x", mfid);
    fprintf(out, "\n\nALGID= %02x", algid);
    fprintf(out, "\n\nKID = %04x", kid);
    fprintf(out, "\n\nTGID = %04x", tgid);
    fprintf(out, "\n\nHeader Data Unit:");
}

```

```

        fprintf(out,
            "\nMicroslot _____0_____1_____");
        for ( i=0; i<11; i++ )
        {
            if ( i % 2 == 0) fprintf(out, "\n          %2i", i);
            fprintf(out, " ");
            for ( j=0; j<6; j++ ) fprintf(out, " %03x", header[i][j]);
        }
    }

    fclose(in);
    fclose(out);
    return (0);
}

```

Limited Use Only

A.3.2 Voice Link Data Unit 1

This is a program to encode the Link Data Unit 1 for a voice message.

```
/******
```

```
    Name:  voice_ldu1.c
    Author: Al Wilson
    Date:   9/5/93
    History: 7/30/93  New
            9/5/93   Fixed initialization of hex_bits_in array.
```

This program is for testing the voice formats for the CAI document. It accepts as input a file of values, in hexadecimal, in the following order:

```
    Network Access Code ( 1   3 digits)
    Status Symbols      (24   1 digit )
    Link Control Format  ( 1   2 digits)
    Manufacturer ID     ( 1   2 digits)
    Emergency bit       ( 1   1 digit )
    Talk Group ID       ( 1   4 digits)
    Source ID           ( 1   6 digits)
    Destination ID      ( 1   6 digits)
    Low Speed Data      ( 2   2 digits)
    IMBE u_vectors      ( 9   8 values, 3 digits each, except 2 in the last)
```

The program then generates a Link Data Unit (LDU) 1 with the appropriate code words. The output is formatted in hexadecimal, with 3 digit groups.

```
*****/
```

```
#include "voice_word.h"
#include "ham.h"
#include "bch.h"
#include "cyclic.h"
#include "reed_solomon.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

#define DUID          0x5   /* Data Unit ID for LDU 1 */

int main(void)
{
    int nac, ss[61], mfid, lcf, emergency, tgid, lsd[2];
    long int source, destination;
    int u_values[9][8]; /* 9 frames, 8 vectors per frame */
    int ldu[24][6]; /* 24 microslots, 6 ints of 12 bits each per slot */
    int error_state, i, j, k;
    int fs[4]; /* Frame Sync */
    int nid[5]; /* Network ID code word */
    int encoded_imbe[10];
    int hex_bits_in[12]; /* input to the Reed-Solomon encoder */
    int code_word[25]; /* long enough for the LC word */
    int lsd_encode[3];
    int microslot, word;

    error_state = get_file_name();
    if(error_state == 0)
        return (1);

    for ( i=0; i<24; i++ ) for ( j=0; j<6; j++ ) ldu[i][j] = 0;

    fprintf(out, "Voice LDU 1");
    while(feof(in) == 0)
    {
```

```

nac = get_file_word(3);
for ( i=0; i<24; i++ ) ss[i] = get_file_word(1) & 0x3;
lcf = get_file_word(2);
mfid = get_file_word(2);
emergency = get_file_word(1) & 0x1;
tgid = get_file_word(4);
source = get_file_word_long(6);
destination = get_file_word_long(6);
for ( i=0; i<2; i++ ) lsd[i] = get_file_word(2);
for ( i=0; i<9; i++ )
{
    for ( j=0; j<7; j++ )
        u_values[i][j] = get_file_word(3);
    u_values[i][7] = get_file_word(2);
}

for ( i=0; i<24; i++ ) for ( j=0; j<6; j++ ) ldu[i][j] = 0;

/* Do the Frame Sync */
for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

/* Do the Network ID */
for ( i=0; i<5; i++ ) nid[i] = 0;
bch_64_encode ( (nac << 4) | DUID, nid );

/* To construct the hex bits for input to the Reed-Solomon code
   first load all the arguments into a vector. Note that mi[0]
   has only 8 bits (the LSBs) used. The rest are 16 bits.
*/
hex_bits_in[0] = (lcf << 8) | mfid;
if ( lcf == 0x03 )
{
    /* Individual call case */
    hex_bits_in[1] = destination >> 16;
    hex_bits_in[2] = destination & 0xffff;
}
else
{
    /* Group call case */
    hex_bits_in[1] = emergency << 15;
    hex_bits_in[2] = tgid;
}
hex_bits_in[3] = source >> 8;
hex_bits_in[4] = (source & 0xff) << 8;
for ( i=5; i<12; i++ ) hex_bits_in[i] = 0;
for ( i=0; i<12; i++ )
    shift_vector_right ( 12-i, 16, 10, &hex_bits_in[i] );
reed_solomon_24_12 ( hex_bits_in, code_word );
for ( i=0; i<24; i++ )
    code_word[i] = hamming_10_encode(code_word[i]);

/* Do the low speed data. */
for ( i=0; i<2; i++ ) lsd_encode[i] = cyclic_encode(lsd[i]);

/* Do the first voice frame. */
voice_encode ( &u_values[0][0], encoded_imbe );

microslot = 0; word = 0;
for ( i=0; i<864; i++ )
{
    ldu[microslot][word] = ldu[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        ldu[microslot][word] |= ss[(i-35)/36];
    else if ( i < 24 ) /* Frame Sync */
        ldu[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        ldu[microslot][word] |= extract_dibit(4,16,nid);
    else if ( i < 205 ) /* IMBE frame 1 & 2 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 226 ) /* LC words 1-4 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 300 ) /* IMBE frame 3 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 320 ) /* LC words 5-8 */

```

```

        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 394 ) /* IMBE frame 4 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 415 ) /* LC words 9-12 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 489 ) /* IMBE frame 5 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 510 ) /* LC words 13-16 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 584 ) /* IMBE frame 6 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 604 ) /* LC words 17-20 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 678 ) /* IMBE frame 7 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 699 ) /* LC words 21-24 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 773 ) /* IMBE frame 8 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 789 ) /* LSD words */
        ldu[microslot][word] |= extract_dibit(2,16,lsd_encode);
    else /* IMBE frame 9 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    if ( i == 130 ) voice_encode ( &u_values[1][0], encoded_imbe );
    if ( i == 225 ) voice_encode ( &u_values[2][0], encoded_imbe );
    if ( i == 319 ) voice_encode ( &u_values[3][0], encoded_imbe );
    if ( i == 414 ) voice_encode ( &u_values[4][0], encoded_imbe );
    if ( i == 509 ) voice_encode ( &u_values[5][0], encoded_imbe );
    if ( i == 603 ) voice_encode ( &u_values[6][0], encoded_imbe );
    if ( i == 698 ) voice_encode ( &u_values[7][0], encoded_imbe );
    if ( i == 788 ) voice_encode ( &u_values[8][0], encoded_imbe );
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if(feof(in) == 0)
{
    fprintf(out, "\n\nNAC = %03x\nSSym =", nac);
    for ( i=0; i<24; i++ ) fprintf(out, " %1x", ss[i]);
    fprintf(out, "\nLCF = %02x", lcf);
    fprintf(out, "\nMFID = %02x", mfid);
    fprintf(out, "\nEmerg= %01x", emergency);
    fprintf(out, "\nTGID = %04x", tgid);
    fprintf(out, "\nSourc= %06x", source);
    fprintf(out, "\nDest = %06x", destination);
    fprintf(out, "\nLSD = %02x %02x", lsd[0], lsd[1]);
    fprintf(out, "\nIMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7");
    for ( i=0; i<9; i++ )
    {
        fprintf(out, "\n %1i: ", i+1);
        for ( j=0; j<7; j++ ) fprintf(out, " %03x", u_values[i][j]);
        fprintf(out, " %02x", u_values[i][7]);
    }
    fprintf(out, "\nLink Data Unit 1:");
    fprintf(out,
        "\nMicroslot _____0_____1_____");
    for ( i=0; i<24; i++ )
    {
        if ( i % 2 == 0 ) fprintf(out, "\n %2i", i);
        fprintf(out, " ");
        for ( j=0; j<6; j++ ) fprintf(out, " %03x", ldu[i][j]);
    }
}

fclose(in);
fclose(out);
return (0);
}

```


A.3.3 Voice Link Data Unit 2

This is a program to encode the Link Data Unit 2 for a voice message.

```

/*****
    Name:  voice_ldu2.c
    Author: Al Wilson
    Date:  9/5/93
    History: 7/30/93  New
            9/5/93  Fixed initialization of hex_bits_in array.

This program is for testing the voice formats for the CAI document.  It
accepts as input a file of values, in hexadecimal, in the following order:
    Network Access Code ( 1   3 digits)
    Status Symbols      (24   1 digit )
    Message Indicator   ( 1   5 values, 4 digits each, except 2 in the first)
    Algorithm ID        ( 1   2 digits)
    Key ID              ( 1   4 digits)
    Low Speed Data      ( 2   2 digits)
    IMBE u_vectors      ( 9   8 values, 3 digits each, except 2 in the last)

The program then generates a Link Data Unit (LDU) 1 with the appropriate code
words.  The output is formatted in hexadecimal, with 3 digit groups.

*****/

#include "voice_word.h"
#include "ham.h"
#include "bch.h"
#include "cyclic.h"
#include "reed_solomon.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

#define DUID          0xa /* Data Unit ID for LDU 2 */

int main(void)
{
    int nac, ss[24], mi[5], algid, kid, lsd[2];
    int u_values[9][8]; /* 9 frames, 8 vectors per frame */
    int ldu[24][6]; /* 24 microslots, 6 ints of 12 bits each per slot */
    int error_state, i, j, k;
    int fs[4]; /* Frame Sync */
    int nid[5]; /* Network ID code word */
    int encoded_imbe[10];
    int hex_bits_in[16]; /* input to the Reed-Solomon encoder */
    int code_word[25]; /* long enough for the ES word */
    int lsd_encode[3];
    int microslot, word;

    error_state = get_file_name();
    if(error_state == 0)
        return (1);

    for ( i=0; i<24; i++ ) for ( j=0; j<6; j++ ) ldu[i][j] = 0;

    fprintf(out, "Voice LDU 2");
    while(!feof(in) == 0)
    {
        nac = get_file_word(3);
        for ( i=0; i<24; i++ ) ss[i] = get_file_word(1) & 0x3;
        mi[0] = get_file_word(2); mi[0] &= 0xff;
        for ( i=1; i<5; i++ ) mi[i] = get_file_word(4);

```

```

algid = get_file_word(2);
kid   = get_file_word(4);
for ( i=0; i<2; i++ ) lsd[i] = get_file_word(2);
for ( i=0; i<9; i++ )
{
    for ( j=0; j<7; j++ )
        u_values[i][j] = get_file_word(3);
    u_values[i][7] = get_file_word(2);
}

for ( i=0; i<24; i++ ) for ( j=0; j<6; j++ ) ldu[i][j] = 0;

/* Do the Frame Sync */
for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

/* Do the Network ID */
for ( i=0; i<5; i++ ) nid[i] = 0;
bch_64_encode ( (nac << 4) | DUID, nid );

/* To construct the hex bits for input to the Reed-Solomon code
   first load all the arguments into a vector. Note that mi[0]
   has only 8 bits (the LSBs) used. The rest are 16 bits.
*/
for ( i=0; i<5; i++ ) hex_bits_in[i] = mi[i];
hex_bits_in[5] = (algid << 8) | (kid >> 8);
hex_bits_in[6] = (kid & 0xff) << 8;
for ( i=7; i<16; i++ ) hex_bits_in[i] = 0;
/* Now we space these out to 6 bits in 16 elements. */
shift_vector_right ( 16, 16, 2, &hex_bits_in[0] );
for ( i=1; i<16; i++ )
    shift_vector_right ( 16-i, 16, 10, &hex_bits_in[i] );
reed_solomon_24_16 ( hex_bits_in, code_word );
for ( i=0; i<24; i++ )
    code_word[i] = hamming_10_encode(code_word[i]);

/* Do the low speed data. */
for ( i=0; i<2; i++ ) lsd_encode[i] = cyclic_encode(lsd[i]);

/* Do the first voice frame. */
voice_encode ( &u_values[0][0], encoded_imbe );

microslot = 0; word = 0;
for ( i=0; i<864; i++ )
{
    ldu[microslot][word] = ldu[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        ldu[microslot][word] |= ss[(i-35)/36];
    else if ( i < 24 ) /* Frame Sync */
        ldu[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        ldu[microslot][word] |= extract_dibit(4,16,nid);
    else if ( i < 205 ) /* IMBE frame 1 & 2 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 226 ) /* LC words 1-4 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 300 ) /* IMBE frame 3 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 320 ) /* LC words 5-8 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 394 ) /* IMBE frame 4 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 415 ) /* LC words 9-12 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 489 ) /* IMBE frame 5 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 510 ) /* LC words 13-16 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 584 ) /* IMBE frame 6 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 604 ) /* LC words 17-20 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 678 ) /* IMBE frame 7 */

```

```

        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 699 ) /* LC words 21-24 */
        ldu[microslot][word] |= extract_dibit(24,10,code_word);
    else if ( i < 773 ) /* IMBE frame 8 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    else if ( i < 789 ) /* LSD words */
        ldu[microslot][word] |= extract_dibit(2,16,lsd_encode);
    else /* IMBE frame 9 */
        ldu[microslot][word] |= extract_dibit(9,16,encoded_imbe);
    if ( i == 130 ) voice_encode ( &u_values[1][0], encoded_imbe );
    if ( i == 225 ) voice_encode ( &u_values[2][0], encoded_imbe );
    if ( i == 319 ) voice_encode ( &u_values[3][0], encoded_imbe );
    if ( i == 414 ) voice_encode ( &u_values[4][0], encoded_imbe );
    if ( i == 509 ) voice_encode ( &u_values[5][0], encoded_imbe );
    if ( i == 603 ) voice_encode ( &u_values[6][0], encoded_imbe );
    if ( i == 698 ) voice_encode ( &u_values[7][0], encoded_imbe );
    if ( i == 788 ) voice_encode ( &u_values[8][0], encoded_imbe );
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if(feof(in) == 0)
{
    fprintf(out, "\n\nNAC = %03x\nSSym =", nac);
    for ( i=0; i<24; i++ ) fprintf(out, " %1x", ss[i]);
    fprintf(out, "\n\nMI = %02x", mi[0]);
    for ( i=1; i<5; i++ ) fprintf(out, " %04x", mi[i]);
    fprintf(out, "\n\nALGID= %02x", algid);
    fprintf(out, "\n\nKID = %04x", kid);
    fprintf(out, "\n\nLSD = %02x %02x", lsd[0], lsd[1]);
    fprintf(out, "\n\nIMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7");
    for ( i=0; i<9; i++ )
    {
        fprintf(out, "\n %1i:", i+1);
        for ( j=0; j<7; j++ ) fprintf(out, " %03x", u_values[i][j]);
        fprintf(out, " %02x", u_values[i][7]);
    }
    fprintf(out, "\n\nLink Data Unit 2:" );
    fprintf(out,
        "\n\nMicroslot _____0_____1_____");
    for ( i=0; i<24; i++ )
    {
        if ( i % 2 == 0 ) fprintf(out, "\n %2i", i);
        fprintf(out, " ");
        for ( j=0; j<6; j++ ) fprintf(out, " %03x", ldu[i][j]);
    }
}

fclose(in);
fclose(out);
return (0);
}

```

A.3.4 Voice Terminator

This is a program to encode the simple Terminator data unit for a voice message.

```

/*****

```

```

    Name:  voice_term.c
    Author: Al Wilson
    Date:  8/3/93
    History: 8/3/93  New

```

This program is for testing the voice formats for the CAI document. It accepts as input a file of values, in hexadecimal, in the following order:

```

    Network Access Code ( 1   3 digits)
    Status Symbols      ( 2   1 digit )

```

The program then generates a simple terminator data unit with the appropriate code words. The output is formatted in hexadecimal, with 3 digit groups.

```

*****/

```

```

#include "bch.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

#define DUID          0x3    /* Data Unit ID for simple Terminator */

int main(void)
{
    int nac, ss[2];
    int error_state, i, j, k;
    int fs[4];
    int nid[5];
    int term[2][6];
    int microslot, word;

    error_state = get_file_name();
    if(error_state == 0)
        return (1);

    fprintf(out, "Voice Terminator Data Unit");
    while(feof(in) == 0)
    {
        nac = get_file_word(3);
        for ( i=0; i<2; i++ )  ss[i] = get_file_word(1) & 0x3;

        for ( i=0; i<2; i++ )  for ( j=0; j<6; j++ )  term[i][j] = 0;

        /* Do the Frame Sync */
        for ( i=0; i<4; i++ )  fs[i] = frame_sync_const[i];

        /* Do the Network ID */
        for ( i=0; i<5; i++ )  nid[i] = 0;
        bch_64_encode ( (nac << 4) | DUID, nid );

        microslot = 0;  word = 0;
        for ( i=0; i<72; i++ )
        {
            term[microslot][word] = term[microslot][word] << 2;
            if ( i%36 == 35 )      /* Status Symbol */
                term[microslot][word] |= ss[(i-35)/36];
            else if ( i < 24 )      /* Frame Sync */
                term[microslot][word] |= extract_dibit(3,16,fs);
            else if ( i < 57 )      /* Network ID */

```

```

        term[microslot][word] |= extract_dibit(4,16,nid);
    else
        /* trailing nulls */
        term[microslot][word] |= 0;
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if(feof(in) == 0)
{
    fprintf(out, "\n\nNAC = %03x\nSSym =", nac);
    for ( i=0; i<2; i++ ) fprintf(out, " %1x", ss[i]);
    fprintf(out, "\nTerminator Data Unit:" );
    fprintf(out,
        "\nMicroslot _____0_____1_____");
    for ( i=0; i<2; i++ )
    {
        if ( i % 2 == 0 ) fprintf(out, "\n          %2i", i);
        fprintf(out, " ");
        for ( j=0; j<6; j++ ) fprintf(out, " %03x", term[i][j]);
    }
}

fclose(in);
fclose(out);
return (0);
}

```

A.4 Data Packet Programs

These are a collection of programs which use the primitive modules in Section A.2 to encode complete data packets.

A.4.1 Confirmed Data Packet

This is a program to encode Confirmed Data Packets.

```

/*****

```

```

    Name:  data_confirmed.c
    Author: Al Wilson
    Date:  9/5/93
    History: 8/5/93  New
            9/5/93  Fixed exception for zero data blocks.

```

This program is for testing the voice formats for the CAI document. It accepts as input a file of values, in hexadecimal, in the following order:

```

Network Access Code ( 1   3 digits)
Status Symbols      ( 1   1 digit )   to be repeated as required
io_bit              ( 1   1 digit )
SAP                 ( 1   2 digits)
Manufacturer ID     ( 1   2 digits)
Logical Link Addr   ( 1   6 digits)
fmf                 ( 1   1 digit )
Blocks_to_follow    ( 1   2 digits)    = B
Pad_octets          ( 1   2 digits)    = P
sync                ( 1   1 digit )
n(s)                ( 1   1 digit )
fsnf                ( 1   1 digit )
Data_header_offset  ( 1   2 digits)
Serial_number       ( B   2 digits)    ser. no. precedes every 16 data octets
Data_octets         ( N   2 digits)    N = 16 x B - 4 - P

```

The program then generates a header block and B data blocks. The output is formatted in hexadecimal, with 3 digit groups.

```

*****/

```

```

#include "bch.h"
#include "crc.h"
#include "trellis.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

```

```

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

```

```

#define DUID          0xc    /* Data Unit ID for Data Packet */

```

```

int main(void)
{

```

```

    int nac, ss, io_bit, sap, mfid, fmf, blocks_to_follow, pad_octet_count;
    int sync, n_s, fsnf, header_offset;
    long int logical_link_id, data_crc;
    int data_length, header_crc;
    int data_octets[528];
    int serial_number[34], sarq_crc[34];
    int data_block[15]; /* enough for 1 block + 1 extra */
    int header_block[12];
    int sarq_block[18]; /* 18 octets with the ser. no. + CRC9 */
    int data[116][6]; /* 116 microslots, 6 ints of 12 bits per slot */
    int error_state, i, j, k;

```

```

int fs[4];          /* Frame Sync */
int nid[5];         /* Network ID code word */
int microslot, word;
int total_di_bits;  /* The number of dibits, not counting busy bits */
int total_microslots; /* The number of status symbols. */
int data_block_di_bit;
int data_block_number;

error_state = get_file_name();
if(error_state == 0)
    return (1);

fprintf(out, "Confirmed Data Packet");
while(feof(in) == 0)
{
    nac = get_file_word(3);
    ss = get_file_word(1) & 0x3;
    io_bit = get_file_word(1) & 0x1;
    sap = get_file_word(2) & 0x3f;
    mfid = get_file_word(2);
    logical_link_id = get_file_word_long(6);
    fmf = get_file_word(1) & 0x1;
    blocks_to_follow = get_file_word(2) & 0x7f;
    pad_octet_count = get_file_word(2) & 0x1f;
    if ( fmf == 0 ) pad_octet_count = 0;
    sync = get_file_word(1) & 0x1;
    n_s = get_file_word(1) & 0x7;
    fsnf = get_file_word(1);
    header_offset = get_file_word(2) & 0x3f;
    data_length = 16 * blocks_to_follow - 4 - pad_octet_count;
    if ( fmf == 0 ) data_length += 4; /* No CRC32 octets */
    if ( feof(in) != 0 ) break;
    for ( i=0; i<data_length; i++ )
    {
        if ( i % 16 == 0 ) serial_number[i/16] = get_file_word(2) & 0x7f;
        data_octets[i] = get_file_word(2);
    }
    if ( feof(in) != 0 ) break;

    for ( i=0; i<116; i++ ) for ( j=0; j<6; j++ ) data[i][j] = 0;

    /* Do the Frame Sync */
    for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

    /* Do the Network ID */
    for ( i=0; i<5; i++ ) nid[i] = 0;
    bch_64_encode ( (nac << 4) | DUID, nid );

    /* Do the header block */
    header_block[0] = 0x56 | (io_bit << 5);
    header_block[1] = 0xc0 | sap;
    header_block[2] = mfid;
    header_block[3] = 0xff & (logical_link_id >> 16);
    header_block[4] = 0xff & (logical_link_id >> 8);
    header_block[5] = 0xff & logical_link_id;
    header_block[6] = (fmf << 7) | blocks_to_follow;
    header_block[7] = pad_octet_count;
    header_block[8] = (sync << 7) | (n_s << 4) | fsnf;
    header_block[9] = header_offset;
    header_crc = crc_ccitt ( header_block );
    header_block[10] = 0xff & (header_crc >> 8);
    header_block[11] = 0xff & header_crc;
    trellis_1_2_encode ( header_block, data_block );
    data_block_di_bit = 98; /* number of dibits in array */

    /* Get data ready to turn into blocks. */
    if ( blocks_to_follow > 0 )
    {
        for ( i=data_length; i<(data_length+pad_octet_count); i++ )
        {
            data_octets[i] = 0;
            if ( i % 16 == 0 )

```

```

        serial_number[i/16] = serial_number[(i/16)-1] + 1;
    }
    if ( fmf == 1 )
    { /* Only do CRC32 if its a full message, not a selective retry. */
        data_crc = crc_32 ( data_length+pad_octet_count, data_octets );
        for ( i=data_length+pad_octet_count+3;
              i>=(data_length+pad_octet_count); i-- )
        {
            data_octets[i] = 0xff & data_crc;
            data_crc = data_crc >> 8;
        }
    }
    for ( i=0; i<blocks_to_follow; i++ )
        sarq_crc[i] = crc_9 ( serial_number[i], &data_octets[16*i] );
}
data_block_number = 0;

/* There are 24 frame sync dibits
   32 NID dibits
   98 dibits for each block (add 1 for header block)
*/
total_di_bits = 24 + 32 + 98 * (1 + blocks_to_follow);
total_microslots = (total_di_bits + 34)/35;
total_di_bits += total_di_bits/35; /* Allow for embedded busy bits */
microslot = 0; word = 0;
for ( i=0; i<36*total_microslots; i++ )
{
    data[microslot][word] = data[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        data[microslot][word] |= ss;
    else if ( i < 24 ) /* Frame Sync */
        data[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        data[microslot][word] |= extract_dibit(4,16,nid);
    else if ( i < total_di_bits ) /* Data Blocks */
    {
        data[microslot][word] |= extract_dibit(14,14,data_block);
        data_block_di_bit--;
    }
    else /* trailing nulls */
        data[microslot][word] |= 0;
    if ( data_block_di_bit <= 0 )
    {
        for ( j=0; j<16; j++ )
            sarq_block[j+2] = data_octets[16*data_block_number + j];
        sarq_block[1] = sarq_crc[data_block_number] & 0xff;
        sarq_block[0] = (serial_number[data_block_number] << 1) |
            (sarq_crc[data_block_number] >> 8);
        trellis_3_4_encode ( sarq_block, data_block );
        data_block_di_bit = 98;
        data_block_number++;
    }
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if (feof(in) == 0)
{
    fprintf(out, "\n\nNAC      = %03x", nac);
    fprintf(out, "\n\nSSymbol= %01x", ss);
    fprintf(out, "\n\nI/O      = %01x", io_bit);
    fprintf(out, "\n\nSAP      = %02x", sap);
    fprintf(out, "\n\nMFID     = %02x", mfid);
    fprintf(out, "\n\nLLID     = %06x", logical_link_id);
    fprintf(out, "\n\nFMF      = %01x", fmf);
    fprintf(out, "\n\nblocks  = %02x", blocks_to_follow);
    fprintf(out, "\n\npads    = %02x", pad_octet_count);
    fprintf(out, "\n\nsync    = %01x", sync);
    fprintf(out, "\n\nN(S)    = %01x", n_s);
    fprintf(out, "\n\nFSNF    = %01x", fsnf);
    fprintf(out, "\n\noffset  = %02x", header_offset);
    fprintf(out, "\n\nHdr CRC= %04x", header_crc);
}

```



```

        fprintf(out,
            "\nData :
SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15");
        for ( i=0; i<16*blocks_to_follow; i++ )
        {
            if ( i % 16 == 0 )
                fprintf(out, "\n%4i    %02x %03x",
                    i/16, serial_number[i/16], sarq_crc[i/16]);
            fprintf(out, " %02x", data_octets[i]);
        }
        fprintf(out, "\nData Packet:" );
        fprintf(out,
            "\nMicroslot      0      1      ");
        for ( i=0; i<total_microslots; i++ )
        {
            if ( i % 2 == 0 ) fprintf(out, "\n      %3i", i);
            fprintf(out, " ");
            for ( j=0; j<6; j++ ) fprintf(out, " %03x", data[i][j]);
        }
    }
}

fclose(in);
fclose(out);
return (0);
}

```

A.4.2 Response Data Packet

This is a program to encode Response Data Packets.

```
/******
```

```
    Name:  data_response.c
    Author: Al Wilson
    Date:   9/5/93
    History: 8/6/93  New
            9/5/93  Fixed exception for zero data blocks.
```

This program is for testing the voice formats for the CAI document. It accepts as input a file of values, in hexadecimal, in the following order:

```
    Network Access Code ( 1   3 digits)
    Status Symbols      ( 1   1 digit )   to be repeated as required
    io_bit              ( 1   1 digit )
    Class               ( 1   1 digit )
    Type                ( 1   1 digit )
    Status              ( 1   1 digit )
    Manufacturer ID     ( 1   2 digits)
    Logical Link Addr   ( 1   6 digits)
    Blocks_to_follow    ( 1   2 digits)    = B
    Data_octets         ( N   2 digits)    N = 12 x B - 4
```

The program then generates a header block and B data blocks. The output is formatted in hexadecimal, with 3 digit groups.

```
*****/
```

```
#include "bch.h"
#include "crc.h"
#include "trellis.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"
```

```
static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};
```

```
#define DUID          0xc /* Data Unit ID for Data Packet */
```

```
int main(void)
{
```

```
    int nac, ss, io_bit, d_class, d_type, d_status, mfid, blocks_to_follow;
    long int logical_link_id, data_crc;
    int data_length, header_crc;
    int data_octets[24]; /* enough for 2 blocks of data */
    int data_block[15]; /* enough for 1 block + 1 extra */
    int header_block[12];
    int data[10][6]; /* 10 microslots, 6 ints of 12 bits per slot */
    int error_state, i, j, k;
    int fs[4]; /* Frame Sync */
    int nid[5]; /* Network ID code word */
    int microslot, word;
    int total_di_bits; /* The number of dibits, not counting busy bits */
    int total_microslots; /* The number of status symbols. */
    int data_block_di_bit;
    int data_block_number;
```

```
    error_state = get_file_name();
    if(error_state == 0)
        return (1);
```

```
    fprintf(out, "Response Data Packet");
    while(feof(in) == 0)
    {
        nac = get_file_word(3);
```

```

ss                = get_file_word(1) & 0x3;
io_bit            = get_file_word(1) & 0x1;
d_class           = get_file_word(1) & 0x3;
d_type            = get_file_word(1) & 0x7;
d_status          = get_file_word(1) & 0x7;
mfid              = get_file_word(2);
logical_link_id   = get_file_word_long(6);
blocks_to_follow  = get_file_word(2) & 0x7f;
data_length       = 12 * blocks_to_follow - 4;
if ( feof(in) != 0 ) break;
for ( i=0; i<data_length; i++ ) data_octets[i] = get_file_word(2);
if ( feof(in) != 0 ) break;

for ( i=0; i<10; i++ ) for ( j=0; j<6; j++ ) data[i][j] = 0;

/* Do the Frame Sync */
for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

/* Do the Network ID */
for ( i=0; i<5; i++ ) nid[i] = 0;
bch_64_encode ( (nac << 4) | DUID, nid );

/* Do the header block */
header_block[0] = 0x03 | (io_bit << 5);
header_block[1] = (d_class << 6) | (d_type << 3) | d_status;
header_block[2] = mfid;
header_block[3] = 0xff & (logical_link_id >> 16);
header_block[4] = 0xff & (logical_link_id >> 8);
header_block[5] = 0xff & logical_link_id;
header_block[6] = 0x80 | blocks_to_follow;
header_block[7] = 0;
header_block[8] = 0;
header_block[9] = 0;
header_crc = crc_ccitt ( header_block );
header_block[10] = 0xff & (header_crc >> 8);
header_block[11] = 0xff & header_crc;
trellis_1_2_encode ( header_block, data_block );
data_block_di_bit = 98; /* number of dibits in array */

/* Get data ready to turn into blocks. */
if ( blocks_to_follow > 0 )
{
    data_crc = crc_32 ( data_length, data_octets );
    for ( i=data_length+3; i>=data_length; i-- )
    {
        data_octets[i] = 0xff & data_crc;
        data_crc = data_crc >> 8;
    }
}
data_block_number = 0;

/* There are 24 frame sync dibits
   32 NID dibits
   98 dibits for each block (add 1 for header block)
*/
total_di_bits = 24 + 32 + 98 * (1 + blocks_to_follow);
total_microslots = (total_di_bits + 34)/35;
total_di_bits += total_di_bits/35; /* Allow for embedded busy bits */
microslot = 0; word = 0;
for ( i=0; i<36*total_microslots; i++ )
{
    data[microslot][word] = data[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        data[microslot][word] |= ss;
    else if ( i < 24 ) /* Frame Sync */
        data[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        data[microslot][word] |= extract_dibit(4,16,nid);
    else if ( i < total_di_bits ) /* Data Blocks */
    {
        data[microslot][word] |= extract_dibit(14,14,data_block);
        data_block_di_bit--;
    }
}

```

```

    }
    else /* trailing nulls */
        data[microslot][word] |= 0;
    if ( data_block_di_bit <= 0 )
    {
        trellis_1_2_encode ( &data_octets[12*data_block_number],
                             data_block );
        data_block_di_bit = 98;
        data_block_number++;
    }
    if ( i%6 == 5 ) word++;
    if ( word == 6 ) { microslot++; word = 0; }
}

if (feof(in) == 0)
{
    fprintf(out, "\n\nNAC      = %03x", nac);
    fprintf(out, "\nSSymbol= %01x", ss);
    fprintf(out, "\nI/O      = %01x", io_bit);
    fprintf(out, "\nClass    = %01x", d_class);
    fprintf(out, "\nType     = %01x", d_type);
    fprintf(out, "\nStatus   = %01x", d_status);
    fprintf(out, "\nMFID     = %02x", mfid);
    fprintf(out, "\nLLID     = %06x", logical_link_id);
    fprintf(out, "\nblocks   = %02x", blocks_to_follow);
    fprintf(out, "\nHdr CRC= %04x", header_crc);
    if ( blocks_to_follow > 0 )
        fprintf(out, "\nData : 0_1_2_3_4_5_6_7_8_9_10_11");
    for ( i=0; i<12*blocks_to_follow; i++ )
    {
        if ( i % 12 == 0 ) fprintf(out, "\n%4i ", i/12);
        fprintf(out, " %02x", data_octets[i]);
    }
    fprintf(out, "\nData Packet:");
    fprintf(out,
        "\nMicroslot 0 _____ 1 _____");
    for ( i=0; i<total_microslots; i++ )
    {
        if ( i % 2 == 0 ) fprintf(out, "\n      %3i", i);
        fprintf(out, " ");
        for ( j=0; j<6; j++ ) fprintf(out, " %03x", data[i][j]);
    }
}

fclose(in);
fclose(out);
return (0);
}

```

A.4.3 Unconfirmed Data Packet

This is a program to encode Unconfirmed Data Packets.

```

/*****
    Name:  data_unconfirmed.c
    Author: Al Wilson
    Date:  9/5/93
    History: 8/5/93  New
            9/5/93  Fixed exception for zero data blocks.

```

This program is for testing the voice formats for the CAI document. It accepts as input a file of values, in hexadecimal, in the following order:

```

Network Access Code ( 1   3 digits)
Status Symbols      ( 1   1 digit )  to be repeated as required
io_bit              ( 1   1 digit )
SAP                 ( 1   2 digits)
Manufacturer ID     ( 1   2 digits)
Logical Link Addr   ( 1   6 digits)
Blocks_to_follow    ( 1   2 digits)  = B
Pad_octets          ( 1   2 digits)  = P
Data_header_offset  ( 1   2 digits)
Data_octets         ( N   2 digits)  N = 12 x B - 4 - P

```

The program then generates a header block and B data blocks. The output is formatted in hexadecimal, with 3 digit groups.

```

*****/

#include "bch.h"
#include "crc.h"
#include "trellis.h"
#include <stdio.h>
#include "get_file_io.h"
#include "vector_operations.h"

static int frame_sync_const[4] = {0x5575, 0xf5ff, 0x77ff, 0};

#define DUID      0xc /* Data Unit ID for Data Packet */

int main(void)
{
    int nac, ss, io_bit, sap, mfid, blocks_to_follow, pad_octet_count;
    int header_offset;
    long int logical_link_id, data_crc;
    int data_length, header_crc;
    int data_octets[516];
    int data_block[15]; /* enough for 1 block + 1 extra */
    int header_block[12];
    int data[116][6]; /* 116 microslots, 6 ints of 12 bits per slot */
    int error_state, i, j, k;
    int fs[4]; /* Frame Sync */
    int nid[5]; /* Network ID code word */
    int microslot, word;
    int total_di_bits; /* The number of dibits, not counting busy bits */
    int total_microslots; /* The number of status symbols. */
    int data_block_di_bit;
    int data_block_number;

    error_state = get_file_name();
    if(error_state == 0)
        return (1);

    fprintf(out, "Unconfirmed Data Packet");
    while(!feof(in) == 0)
    {

```

```

nac          = get_file_word(3);
ss           = get_file_word(1) & 0x3;
io_bit       = get_file_word(1) & 0x1;
sap          = get_file_word(2) & 0x3f;
mfid         = get_file_word(2);
logical_link_id = get_file_word_long(6);
blocks_to_follow = get_file_word(2) & 0x7f;
pad_octet_count = get_file_word(2) & 0x1f;
header_offset = get_file_word(2) & 0x3f;
data_length   = 12 * blocks_to_follow - 4 - pad_octet_count;
if ( feof(in) != 0 ) break;
for ( i=0; i<data_length; i++ ) data_octets[i] = get_file_word(2);
if ( feof(in) != 0 ) break;

for ( i=0; i<116; i++ ) for ( j=0; j<6; j++ ) data[i][j] = 0;

/* Do the Frame Sync */
for ( i=0; i<4; i++ ) fs[i] = frame_sync_const[i];

/* Do the Network ID */
for ( i=0; i<5; i++ ) nid[i] = 0;
bch_64_encode ( (nac << 4) | DUID, nid );

/* Do the header block */
header_block[0] = 0x15 | (io_bit << 5);
header_block[1] = 0xc0 | sap;
header_block[2] = mfid;
header_block[3] = 0xff & (logical_link_id >> 16);
header_block[4] = 0xff & (logical_link_id >> 8);
header_block[5] = 0xff & logical_link_id;
header_block[6] = 0x80 | blocks_to_follow;
header_block[7] = pad_octet_count;
header_block[8] = 0;
header_block[9] = header_offset;
header_crc = crc_ccitt ( header_block );
header_block[10] = 0xff & (header_crc >> 8);
header_block[11] = 0xff & header_crc;
trellis_1_2_encode ( header_block, data_block );
data_block_di_bit = 98; /* number of dibits in array */

/* Get data ready to turn into blocks. */
if ( blocks_to_follow > 0 )
{
    for ( i=data_length; i<(data_length+pad_octet_count); i++ )
        data_octets[i] = 0;
    data_crc = crc_32 ( data_length+pad_octet_count, data_octets );
    for ( i=data_length+pad_octet_count+3;
          i>=(data_length+pad_octet_count); i-- )
    {
        data_octets[i] = 0xff & data_crc;
        data_crc = data_crc >> 8;
    }
}
data_block_number = 0;

/* There are 24 frame sync dibits
   32 NID dibits
   98 dibits for each block (add 1 for header block)
*/
total_di_bits = 24 + 32 + 98 * (1 + blocks_to_follow);
total_microslots = (total_di_bits + 34)/35;
total_di_bits += total_di_bits/35; /* Allow for embedded busy bits */
microslot = 0; word = 0;
for ( i=0; i<36*total_microslots; i++ )
{
    data[microslot][word] = data[microslot][word] << 2;
    if ( i%36 == 35 ) /* Status Symbol */
        data[microslot][word] |= ss;
    else if ( i < 24 ) /* Frame Sync */
        data[microslot][word] |= extract_dibit(3,16,fs);
    else if ( i < 57 ) /* Network ID */
        data[microslot][word] |= extract_dibit(4,16,nid);
}

```

```

else if ( i < total_di_bits ) /* Data Blocks */
{
    data[microslot][word] |= extract_dibit(14,14,data_block);
    data_block_di_bit--;
}
else /* trailing nulls */
    data[microslot][word] |= 0;
if ( data_block_di_bit <= 0 )
{
    trellis_1_2_encode ( &data_octets[12*data_block_number],
                        data_block );
    data_block_di_bit = 98;
    data_block_number++;
}
if ( i%6 == 5 ) word++;
if ( word == 6 ) { microslot++; word = 0; }
}

if(feof(in) == 0)
{
    fprintf(out, "\n\nNAC = %03x", nac);
    fprintf(out, "\nSSymbol= %01x", ss);
    fprintf(out, "\nI/O = %01x", io_bit);
    fprintf(out, "\nSAP = %02x", sap);
    fprintf(out, "\nMFID = %02x", mfid);
    fprintf(out, "\nLLID = %06x", logical_link_id);
    fprintf(out, "\nblocks = %02x", blocks_to_follow);
    fprintf(out, "\npads = %02x", pad_octet_count);
    fprintf(out, "\noffset = %02x", header_offset);
    fprintf(out, "\nHdr CRC= %04x", header_crc);
    fprintf(out, "\nData : _0_1_2_3_4_5_6_7_8_9_10_11");
    for ( i=0; i<12*blocks_to_follow; i++ )
    {
        if ( i % 12 == 0 ) fprintf(out, "\n%4i ", i/12);
        fprintf(out, " %02x", data_octets[i]);
    }
    fprintf(out, "\nData Packet:");
    fprintf(out,
        "\nMicroslot _____0_____1_____");
    for ( i=0; i<total_microslots; i++ )
    {
        if ( i % 2 == 0 ) fprintf(out, "\n      %3i", i);
        fprintf(out, " ");
        for ( j=0; j<6; j++ ) fprintf(out, " %03x", data[i][j]);
    }
}

fclose(in);
fclose(out);
return (0);
}

```

This page intentionally left blank

Limited Use Only

Annex B. Test Vectors

This annex contains a list of test vectors created by the test routines given in Annex A. The test vectors are representative of actual signals conforming to the Common Air Interface standard. They are referred to in the tests given in the normative part of this document.

B.1 Network Access Code Test Vectors

Case 1: Subscriber Radio

NAC = 023
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 023 338 3b1 526 d5c 600 000 000

NAC = 031
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 031 374 7c1 aa8 976 f00 000 000

NAC = 032
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 032 324 dfa ca6 1ef 400 000 000

NAC = 043
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 043 3dc 27e c39 7c5 c00 000 000

NAC = 114
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 114 390 94a 1d6 f7b a80 000 000

NAC = 205
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 205 358 784 2ff 9b9 b00 000 000

NAC = 40b
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 40b 3d8 24f 462 02c 200 000 000

NAC = 411
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 411 380 a11 1d1 19b 680 000 000

NAC = 464
 SSym = 0 0
 Terminator Data Unit:
 Microslot 01
 0 557 5f5 ff7 7ff 464 3bc dce 6a7 993 a80 000 000

NAC = 516
 SSym = 0 0
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 516 358 80d 9ae ef0 700 000 000

NAC = 585
 SSym = 0 0
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 585 3cc 419 de1 d56 a80 000 000

NAC = 811
 SSym = 0 0
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 811 3e4 137 0aa bed 380 000 000

Case 2: Repeater Radio

NAC = 023
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 023 339 3b1 526 d5c 600 000 001

NAC = 031
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 031 375 7c1 aa8 976 f00 000 001

NAC = 032
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 032 325 dfa ca6 1ef 400 000 001

NAC = 043
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 043 3dd 27e c39 7c5 c00 000 001

NAC = 114
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 114 391 94a 1d6 f7b a80 000 001

NAC = 205
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 205 359 784 2ff 9b9 b00 000 001

NAC = 40b
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 40b 3d9 24f 462 02c 200 000 001

NAC = 411
 SSym = 1 1
 Terminator Data Unit:
 Microslot 0 557 5f5 ff7 7ff 411 381 a11 1d1 19b 680 000 001

NAC = 464

```

SSym = 1 1
Terminator Data Unit:
Microslot 0 557 5f5 ff7 7ff 464 3bd dce 6a7 993 a80 000 001

```

```

NAC = 516
SSym = 1 1
Terminator Data Unit:
Microslot 0 557 5f5 ff7 7ff 516 359 80d 9ae ef0 700 000 001

```

```

NAC = 585
SSym = 1 1
Terminator Data Unit:
Microslot 0 557 5f5 ff7 7ff 585 3cd 419 de1 d56 a80 000 001

```

```

NAC = 811
SSym = 1 1
Terminator Data Unit:
Microslot 0 557 5f5 ff7 7ff 811 3e5 137 0aa bed 380 000 001

```

Limited Use Only

B.2 Status Symbol Test Vectors

Case 1: Subscriber Radio

```

NAC   = 293
SSym  = 0 0 0 0 0 0 0 0 0 0 0 0
MI    = 00 0000 0000 0000 0000
MFID  = 00
ALGID = 80
KID   = 0000
TGID  = 0001
Header Data Unit:
Microslot
0      557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2      000 000 000 000 000 000 000 000 000 000 000 000
4      000 000 000 000 000 000 000 000 000 000 000 000
6      000 000 000 18e b5e 094 527 c2a 235 40d 9f3 47c
8      f1d 41e 421 ea2 358 bfc 312 a5e 371 84a 965 1b8
10     30a 4f8 0c7 54c 751 000

```

```

SSym = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
LCF  = 00
MFID = 00
Emerg = 0
TGID = 0001
Sourc = 000001
Dest  = 000001
LSD   = 00 00

```

```

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 00
2: 09b 088 0cc 621 7b4 02a 04c 01
3: 09b 088 0cc 621 7b4 02a 04c 00
4: 09b 088 0cc 621 7b4 02a 04c 01
5: 09b 088 0cc 621 7b4 02a 04c 00
6: 09b 088 0cc 621 7b4 02a 04c 01
7: 09b 088 0cc 621 7b4 02a 04c 00
8: 09b 088 0cc 621 7b4 02a 04c 01
9: 09b 088 0cc 621 7b4 02a 04c 00

```

Link Data Unit 1:

```

Microslot
0      557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2      0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4      90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6      000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8      89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10     184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12     d43 3c0 be1 b91 844 ff0 162 962 760 e6d e5d 548
14     ade 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16     d83 ba1 41c 2d2 ba3 890 a12 435 0cf 02f 86e 460
18     44f f05 8a5 89d 839 4c8 fb0 235 a4e 24a 124 350
20     33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22     892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

NAC   = 293
SSym  = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
MI    = 00 0000 0000 0000 0000
ALGID = 80
KID   = 0000
LSD   = 00 00

```

```

IMBE :  u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1:  09b 088 0cc 621 7b4 02a 04c 01
2:  09b 088 0cc 621 7b4 02a 04c 00
3:  09b 088 0cc 621 7b4 02a 04c 01
4:  09b 088 0cc 621 7b4 02a 04c 00
5:  09b 088 0cc 621 7b4 02a 04c 01
6:  09b 088 0cc 621 7b4 02a 04c 00
7:  09b 088 0cc 621 7b4 02a 04c 01
8:  09b 088 0cc 621 7b4 02a 04c 00
9:  09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	ace	24a	124				
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0ec	e24	a10				
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	400	000				
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294				
8	89d	83b	000	000	000	038	24a	124	350	cf0	2f8	6e4				
10	184	4ff	058	a58	9d8	390	000	000	000	0e2	4a1	240				
12	d43	3c0	be1	b91	844	ff0	162	962	760	ee0	e00	000				
14	000	389	284	90d	433	c08	f86	e46	113	fc1	629	624				
16	d83	9ae	8b4	8b6	493	890	a12	435	0cf	02f	86e	460				
18	44f	f05	8a5	89d	83b	9a8	f4f	1fd	60e	24a	124	350				
20	33c	0be	1b9	184	4ff	058	296	276	0e4	000	000	00c				
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0ec				

NAC = 293

SSym = 0 0

Terminator Data Unit:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	338	5dd	ca3	3b5	b80	000	000				

NAC = 293

SSym = 2 2 2 2 2 2 2 2 2 2 2 2

MI = 80 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0001

Header Data Unit:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	00e	ced	77e	007	0a0	6cd	002				
2	000	000	000	000	000	002	000	000	000	000	000	002				
4	000	000	000	000	000	00a	371	800	000	000	000	002				
6	000	000	000	18e	bd5	f5a	952	c78	86c	544	6f0	5aa				
8	949	f03	da1	04a	97f	cca	e3b	9bf	ebc	593	16a	bae				
10	7a5	4b1	0a4	faa	235	002										

NAC = 293

SSym = 2

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 000001

LSD = 00 00

```

IMBE :  u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1:  09b 088 0cc 621 7b4 02a 04c 00
2:  09b 088 0cc 621 7b4 02a 04c 01
3:  09b 088 0cc 621 7b4 02a 04c 00
4:  09b 088 0cc 621 7b4 02a 04c 01
5:  09b 088 0cc 621 7b4 02a 04c 00
6:  09b 088 0cc 621 7b4 02a 04c 01
7:  09b 088 0cc 621 7b4 02a 04c 00
8:  09b 088 0cc 621 7b4 02a 04c 01
9:  09b 088 0cc 621 7b4 02a 04c 00

```

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	556	7bc	b19	4d0	dce	24a	126				
2	0d4	33c	0be	1b9	184	4fe	c16	296	276	0e4	e24	a12				
4	90d	433	c0b	e1b	918	44e	fc1	629	627	60e	c00	002				
6	000	003	892	849	0d4	33e	02f	86e	461	13f	c16	296				

```

      8  89d 839 000 000 001 c3a 24a 124 350 cf0 2f8 6e6
     10 184 4ff 058 a58 9d8 3b2 000 000 007 0e2 4a1 242
     12 d43 3c0 be1 b91 844 ff2 162 962 760 e6d e5d 54a
     14 ade 389 284 90d 433 c0a f86 e46 113 fc1 629 626
     16 d83 ba1 41c 2d2 ba3 892 a12 435 0cf 02f 86e 462
     18 44f f05 8a5 89d 839 4ca fb0 235 a4e 24a 124 352
     20 33c 0be 1b9 184 4ff 05a 296 276 0ec 000 000 00e
     22 892 849 0d4 33c 0be 1ba 461 13f c16 296 276 0e6

```

```

NAC = 293
SSym = 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
MI = 00 0000 0000 0000 0000
ALGID= 80
KID = 0000
LSD = 00 00
IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7
      1: 09b 088 0cc 621 7b4 02a 04c 01
      2: 09b 088 0cc 621 7b4 02a 04c 00
      3: 09b 088 0cc 621 7b4 02a 04c 01
      4: 09b 088 0cc 621 7b4 02a 04c 00
      5: 09b 088 0cc 621 7b4 02a 04c 01
      6: 09b 088 0cc 621 7b4 02a 04c 00
      7: 09b 088 0cc 621 7b4 02a 04c 01
      8: 09b 088 0cc 621 7b4 02a 04c 00
      9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

```

Microslot      0      1
      0 557 5f5 ff7 7ff 293 aba a4e fb0 9a8 ace 24a 126
      2 0d4 33c 0be 1b9 184 4fe c16 296 276 0ec e24 a12
      4 90d 433 c0b e1b 918 44e fc1 629 627 60e 400 002
      6 000 003 892 849 0d4 33e 02f 86e 461 13f c16 296
      8 89d 83b 000 000 000 03a 24a 124 350 cf0 2f8 6e6
     10 184 4ff 058 a58 9d8 392 000 000 000 0e2 4a1 242
     12 d43 3c0 be1 b91 844 ff2 162 962 760 ee0 e00 002
     14 000 389 284 90d 433 c0a f86 e46 113 fc1 629 626
     16 d83 9ae 8b4 8b6 493 892 a12 435 0cf 02f 86e 462
     18 44f f05 8a5 89d 83b 9aa f4f 1fd 60e 24a 124 352
     20 33c 0be 1b9 184 4ff 05a 296 276 0e4 000 000 00e
     22 892 849 0d4 33c 0be 1ba 461 13f c16 296 276 0ee

```

```

NAC = 293
SSym = 2 2
Terminator Data Unit:
Microslot      0      1
      0 557 5f5 ff7 7ff 293 33a 5dd ca3 3b5 b80 000 002

```

Case 2: Repeater Radio

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0001
Header Data Unit:
Microslot      0      1
      0 557 5f5 ff7 7ff 293 00d ced 77e 007 080 000 001
      2 000 000 000 000 000 001 000 000 000 000 000 001
      4 000 000 000 000 000 009 371 800 000 000 000 001
      6 000 000 000 18e b5e 095 527 c2a 235 40d 9f3 47d
      8 f1d 41e 421 ea2 358 bfd 312 a5e 371 84a 965 1b9
     10 30a 4f8 0c7 54c 751 001

```

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
LCF = 00
MFID = 00
Emerg= 0
TGID = 0001

```

```

Source= 000001
Dest = 000001
LSD = 00 00
IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 00
2: 09b 088 0cc 621 7b4 02a 04c 01
3: 09b 088 0cc 621 7b4 02a 04c 00
4: 09b 088 0cc 621 7b4 02a 04c 01
5: 09b 088 0cc 621 7b4 02a 04c 00
6: 09b 088 0cc 621 7b4 02a 04c 01
7: 09b 088 0cc 621 7b4 02a 04c 00
8: 09b 088 0cc 621 7b4 02a 04c 01
9: 09b 088 0cc 621 7b4 02a 04c 00
Link Data Unit 1:
Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 000 007 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e6d e5d 549
14 ade 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 ba1 41c 2d2 ba3 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 4c9 fb0 235 a4e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
ALGID= 80
KID = 0000
LSD = 00 00
IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01
Link Data Unit 2:
Microslot      0      1
0 557 5f5 ff7 7ff 293 ab9 a4e fb0 9a8 ace 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0ec e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e 400 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 83b 000 000 000 039 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 391 000 000 000 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 ee0 e00 001
14 000 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 9ae 8b4 8b6 493 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 83b 9a9 f4f 1fd 60e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0e4 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0ed

NAC = 293
SSym = 1 1
Terminator Data Unit:
Microslot      0      1
0 557 5f5 ff7 7ff 293 339 5dd ca3 3b5 b80 000 001

NAC = 293
SSym = 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
MI = 08 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0001

```

TIA-102.BAAB

Header Data Unit:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	00f	ced	77e	007	082	93e	003
2	000	000	000	000	000	003	000	000	000	000	000	003
4	000	000	000	000	000	00b	371	800	000	000	000	003
6	000	000	000	18e	bb0	673	a88	d52	a59	710	db7	eeb
8	3f3	2ec	b25	018	eb3	b9b	f71	0d8	f68	465	928	a4f
10	e1c	436	faf	146	3a9	003						

NAC = 293

SSym = 3

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 000001

LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 00
 2: 09b 088 0cc 621 7b4 02a 04c 01
 3: 09b 088 0cc 621 7b4 02a 04c 00
 4: 09b 088 0cc 621 7b4 02a 04c 01
 5: 09b 088 0cc 621 7b4 02a 04c 00
 6: 09b 088 0cc 621 7b4 02a 04c 01
 7: 09b 088 0cc 621 7b4 02a 04c 00
 8: 09b 088 0cc 621 7b4 02a 04c 01
 9: 09b 088 0cc 621 7b4 02a 04c 00

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	557	7bc	b19	4d0	dce	24a	127
2	0d4	33c	0be	1b9	184	4ff	c16	296	276	0e4	e24	a13
4	90d	433	c0b	e1b	918	44f	fc1	629	627	60e	c00	003
6	000	003	892	849	0d4	33f	02f	86e	461	13f	c16	297
8	89d	839	000	000	001	c3b	24a	124	350	cf0	2f8	6e7
10	184	4ff	058	a58	9d8	3b3	000	000	007	0e2	4a1	243
12	d43	3c0	be1	b91	844	ff3	162	962	760	e6d	e5d	54b
14	ade	389	284	90d	433	c0b	f86	e46	113	fc1	629	627
16	d83	ba1	41c	2d2	ba3	893	a12	435	0cf	02f	86e	463
18	44f	f05	8a5	89d	839	4cb	fb0	235	a4e	24a	124	353
20	33c	0be	1b9	184	4ff	05b	296	276	0ec	000	000	00f
22	892	849	0d4	33c	0be	1bb	461	13f	c16	296	276	0e7

NAC = 293

SSym = 3

MI = 00 0000 0000 0000 0000

ALGID= 80

KID = 0000

LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 01
 2: 09b 088 0cc 621 7b4 02a 04c 00
 3: 09b 088 0cc 621 7b4 02a 04c 01
 4: 09b 088 0cc 621 7b4 02a 04c 00
 5: 09b 088 0cc 621 7b4 02a 04c 01
 6: 09b 088 0cc 621 7b4 02a 04c 00
 7: 09b 088 0cc 621 7b4 02a 04c 01
 8: 09b 088 0cc 621 7b4 02a 04c 00
 9: 09b 088 0cc 621 7b4 02a 04c 01

Link Data Unit 2:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	abb	a4e	fb0	9a8	ace	24a	127
2	0d4	33c	0be	1b9	184	4ff	c16	296	276	0ec	e24	a13
4	90d	433	c0b	e1b	918	44f	fc1	629	627	60e	400	003
6	000	003	892	849	0d4	33f	02f	86e	461	13f	c16	297
8	89d	83b	000	000	000	03b	24a	124	350	cf0	2f8	6e7
10	184	4ff	058	a58	9d8	393	000	000	000	0e2	4a1	243
12	d43	3c0	be1	b91	844	ff3	162	962	760	ee0	e00	003
14	000	389	284	90d	433	c0b	f86	e46	113	fc1	629	627
16	d83	9ae	8b4	8b6	493	893	a12	435	0cf	02f	86e	463
18	44f	f05	8a5	89d	83b	9ab	f4f	1fd	60e	24a	124	353
20	33c	0be	1b9	184	4ff	05b	296	276	0e4	000	000	00f


```

      22  892 849 0d4 33c 0be 1bb  461 13f c16 296 276 0ef
NAC  = 293
SSym = 3 3
Terminator Data Unit:
Microslot      0      1
               0 557 5f5 ff7 7ff 293 33b  5dd ca3 3b5 b80 000 003

```

B.3 Unencrypted Voice Header Test Vectors

Case 1: Subscriber Radio

NAC = 293
 SSym = 0 0 0 0 0 0 0 0 0 0
 MI = 00 0000 0000 0000 0000
 MFID = 00
 ALGID= 80
 KID = 0000
 TGID = 0001
 Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 000 000 18e b5e 094	527 c2a 235 40d 9f3 47c
8	f1d 41e 421 ea2 358 bfc	312 a5e 371 84a 965 1b8
10	30a 4f8 0c7 54c 751 000	

NAC = 293
 SSym = 0 0 0 0 0 0 0 0 0 0
 MI = 00 0000 0000 0000 0000
 MFID = 00
 ALGID= 80
 KID = 0000
 TGID = 0002
 Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 000 000 293 ecd 1fc	cd1 fec 19c 952 c40 000
8	08d c63 da1 07b 42f 684	087 89a c19 c71 0da 93c
10	823 718 37e e9f 5e3 000	

NAC = 293
 SSym = 0 0 0 0 0 0 0 0 0 0
 MI = 00 0000 0000 0000 0000
 MFID = 00
 ALGID= 80
 KID = 0000
 TGID = 0004
 Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 000 000 4a9 7e2 1b0	faf 160 6cd 31d 465 4b0
8	790 878 1b3 5ed 080 c74	5a2 c2f 2c9 400 001 a78
10	7f2 c94 46e 339 087 000	

NAC = 293
 SSym = 0 0 0 0 0 0 0 0 0 0
 MI = 00 0000 0000 0000 0000
 MFID = 00
 ALGID= 80
 KID = 0000
 TGID = 0008
 Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 000 000 8dc 6ba 288	931 6b8 86c 773 76c 19c
8	18e a1f 684 2c1 9cf 0bc	be8 548 ee6 fe2 1b3 f30
10	b90 367 a4f 836 603 000	

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0010
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 000 000 001 036 70a 4f8 40d 9c8 dc6 fcc bbe bc4
8 596 4a1 ed0 8bc 130 638 b7b 423 57d 62f 04f 660
10 0d7 825 69e 7e8 b0b 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0020
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 000 000 002 06c d65 928 ee6 e6b ade e21 b1a 79c
8 dbf 74c 16a 863 a9e 218 c54 46c 000 0ba 28a 4c4
10 699 64a fcc b94 9f0 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0040
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 000 006 3ac 000 025 4b4 237 183 1d5 fcc ba0 6cc
8 6a2 350 63a cdf ba7 10c 625 4b7 2c9 469 e7f 57c
10 58e e6f 4a9 667 d8f 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0080
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 000 00a 4f8 000 03d a10 2f0 4fd a10 e21 b26 564
8 3eb c5f 0be ca4 f80 a4c eee 6e4 ee6 f31 d46 2fc
10 ceb ade bc1 24a 4f8 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0100

```

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 012 a5c 000 00c 754	37e e82 93e d98 0ea 234
8	560 ce1 49f 04a 97f af0	571 odb 57d 681 b34 c74
10	462 ff3 583 393 2b2 000	

NAC = 293

SSym = 0 0 0 0 0 0 0 0 0 0 0

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0200

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 023 718 000 06f dd0	063 aff 32e a88 d72 c94
8	052 7cd 3cf 58e a11 49c	c46 e30 000 0ee 6e5 328
10	cb0 5aa 9f6 3e1 e26 000	

NAC = 293

SSym = 0 0 0 0 0 0 0 0 0 0 0

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0400

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 040 d9c 000 0ae b78	659 286 3a9 4a9 642 93c
8	a37 185 446 e06 cdc 168	a2f 04e 93e 037 eea d94
10	dd4 9f0 18e a46 3a9 000	

NAC = 293

SSym = 0 0 0 0 0 0 0 0 0 0 0

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0800

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 081 b34 000 023 718	a4f 837 ee8 878 9a1 e24
8	ac1 9c5 27c 132 b26 590	afc cbb 886 c8b fcd 2a4
10	65f 5e3 1ed 08b c13 000	

NAC = 293

SSym = 0 0 0 0 0 0 0 0 0 0 0

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 1000

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	18e b00 000 000 0cb 250	63a 850 367 9f6 3cc 750
8	62f f3c d1f cdf ba4 a94	9d5 f59 f5e 346 e30 bc0
10	4c9 52d 583 39f 5e3 000	

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 2000
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 293 e00 000 000 0e8 548 aeb 79b f74 237 1bd a10
8 2e8 a26 fdd 0a4 f87 b40 800 001 782 569 e7d dcc
10 765 4b1 9f6 3f9 087 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 4000
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 4a9 700 000 000 0a8 8d4 3b9 bcd fba 544 6dc 434
8 b66 032 371 84a 971 8e8 9a4 f80 7b4 231 d47 1d4
10 07e bc5 18e a76 603 000

```

```

NAC = 293
SSym = 0 0 0 0 0 0 0 0 0 0
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 8000
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00c ced 77e 007 080 000 000
2 000 000 000 000 000 000 000 000 000 000 000 000
4 000 000 000 000 000 000 008 371 800 000 000 000 000
6 8dc 600 000 000 029 3e0 1ed 0a1 e26 bc1 25d cdc
8 463 a9b a28 98e a1d 980 3e2 1b2 7d8 f81 b36 a20
10 d48 dc6 1ed 0a8 b0b 000

```

Case 2: Repeater Radio

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0001
Header Data Unit:
Microslot
0 557 5f5 ff7 7ff 293 00d ced 77e 007 080 000 001
2 000 000 000 000 000 000 001 000 000 000 000 001
4 000 000 000 000 000 000 009 371 800 000 000 001
6 000 000 000 18e b5e 095 527 c2a 235 40d 9f3 47d
8 f1d 41e 421 ea2 358 bfd 312 a5e 371 84a 965 1b9
10 30a 4f8 0c7 54c 751 001

```

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000

```

TIA-102.BAAB

MFID = 00

ALGID= 80

KID = 0000

TGID = 0002

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 000 000 293 ecd 1fd	cd1 fec 19c 952 c40 001
8	08d c63 da1 07b 42f 685	087 89a c19 c71 0da 93d
10	823 718 37e e9f 5e3 001	

NAC = 293

SSym = 1 1 1 1 1 1 1 1 1 1

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0004

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 000 000 4a9 7e2 1b1	faf 160 6cd 31d 465 4b1
8	790 878 1b3 5ed 080 c75	5a2 c2f 2c9 400 001 a79
10	7f2 c94 46e 339 087 001	

NAC = 293

SSym = 1 1 1 1 1 1 1 1 1 1

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0008

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 000 000 8dc 6ba 289	931 6b8 86c 773 76c 19d
8	18e a1f 684 2c1 9cf 0bd	be8 548 ee6 fe2 1b3 f31
10	b90 367 a4f 836 603 001	

NAC = 293

SSym = 1 1 1 1 1 1 1 1 1 1

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0010

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 000 001 036 70a 4f9	40d 9c8 dc6 fcc bbe bc5
8	596 4a1 ed0 8bc 130 639	b7b 423 57d 62f 04f 661
10	0d7 825 69e 7e8 b0b 001	

NAC = 293

SSym = 1 1 1 1 1 1 1 1 1 1

MI = 00 0000 0000 0000 0000

MFID = 00

ALGID= 80

KID = 0000

TGID = 0020

Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001

```

      4 000 000 000 000 000 009 371 800 000 000 000 001
      6 000 000 002 06c d65 929 ee6 e6b ade e21 b1a 79d
      8 dbf 74c 16a 863 a9e 219 c54 46c 000 0ba 28a 4c5
     10 699 64a fcc b94 9f0 001

```

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0040
Header Data Unit:

```

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 006 3ac 000 025 4b5	237 183 1d5 fcc ba0 6cd
8	6a2 350 63a cdf ba7 10d	625 4b7 2c9 469 e7f 57d
10	58e e6f 4a9 667 d8f 001	

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0080
Header Data Unit:

```

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 00a 4f8 000 03d a11	2f0 4fd a10 e21 b26 565
8	3eb c5f 0be ca4 f80 a4d	eee 6e4 ee6 f31 d46 2fd
10	ceb ade bc1 24a 4f8 001	

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0100
Header Data Unit:

```

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 012 a5c 000 00c 755	37e e82 93e d98 0ea 235
8	560 ce1 49f 04a 97f af1	571 odb 57d 681 b34 c75
10	462 ff3 583 393 2b2 001	

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0200
Header Data Unit:

```

Microslot	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 023 718 000 06f dd1	063 aff 32e a88 d72 c95
8	052 7cd 3cf 58e a11 49d	c46 e30 000 0ee 6e5 329
10	cb0 5aa 9f6 3e1 e26 001	

```

NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000

```

TIA-102.BAAB

```
MFID = 00
ALGID= 80
KID = 0000
TGID = 0400
Header Data Unit:
Microslot
```

	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 040 d9c 000 0ae b79	659 286 3a9 4a9 642 93d
8	a37 185 446 e06 cdc 169	a2f 04e 93e 037 eea d95
10	dd4 9f0 18e a46 3a9 001	

```
NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 0800
Header Data Unit:
```

```
Microslot
```

	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	000 081 b34 000 023 719	a4f 837 ee8 878 9a1 e25
8	ac1 9c5 27c 132 b26 591	afc cbb 886 c8b fcd 2a5
10	65f 5e3 1ed 08b c13 001	

```
NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 1000
Header Data Unit:
```

```
Microslot
```

	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	18e b00 000 000 0cb 251	63a 850 367 9f6 3cc 751
8	62f f3c d1f cdf ba4 a95	9d5 f59 f5e 346 e30 bc1
10	4c9 52d 583 39f 5e3 001	

```
NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 2000
Header Data Unit:
```

```
Microslot
```

	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001
4	000 000 000 000 000 009	371 800 000 000 000 001
6	293 e00 000 000 0e8 549	aeb 79b f74 237 1bd a11
8	2e8 a26 fdd 0a4 f87 b41	800 001 782 569 e7d dcd
10	765 4b1 9f6 3f9 087 001	

```
NAC = 293
SSym = 1 1 1 1 1 1 1 1 1 1
MI = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID = 0000
TGID = 4000
Header Data Unit:
```

```
Microslot
```

	0	1
0	557 5f5 ff7 7ff 293 00d	ced 77e 007 080 000 001
2	000 000 000 000 000 001	000 000 000 000 000 001


```

      4  000 000 000 000 000 009 371 800 000 000 000 001
      6  4a9 700 000 000 0a8 8d5 3b9 bcd fba 544 6dc 435
      8  b66 032 371 84a 971 8e9 9a4 f80 7b4 231 d47 1d5
     10  07e bc5 18e a76 603 001

```

```

NAC  = 293
SSym = 1 1 1 1 1 1 1 1 1 1 1
MI   = 00 0000 0000 0000 0000
MFID = 00
ALGID= 80
KID  = 0000
TGID = 8000
Header Data Unit:

```

```

Microslot      0      1
0  557 5f5 ff7 7ff 293 00d ced 77e 007 080 000 001
2  000 000 000 000 000 001 000 000 000 000 000 001
4  000 000 000 000 000 009 371 800 000 000 000 001
6  8dc 600 000 000 029 3e1 1ed 0a1 e26 bc1 25d cdd
8  463 a9b a28 98e a1d 981 3e2 1b2 7d8 f81 b36 a21
10 d48 dc6 1ed 0a8 b0b 001

```

Limited Use Only

B.4 Unencrypted Link Data Unit 1 Test Vectors

For the sake of brevity, the full list of test inputs, which includes the IMBE information, is only given for the first test vector in each case. The remaining test vectors only have the Link Control information portion listed.

Case 1: Subscriber Radio

```

NAC      = 293
SSym     = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
LCF      = 00
MFID     = 00
Emerg    = 0
TGID     = 0001
Sourc    = 000001
Dest     = 000001
LSD      = 00 00
IMBE :   u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7
1:  09b 088 0cc 621 7b4 02a 04c 00
2:  09b 088 0cc 621 7b4 02a 04c 01
3:  09b 088 0cc 621 7b4 02a 04c 00
4:  09b 088 0cc 621 7b4 02a 04c 01
5:  09b 088 0cc 621 7b4 02a 04c 00
6:  09b 088 0cc 621 7b4 02a 04c 01
7:  09b 088 0cc 621 7b4 02a 04c 00
8:  09b 088 0cc 621 7b4 02a 04c 01
9:  09b 088 0cc 621 7b4 02a 04c 00

Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e6d e5d 548
14 ade 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 ba1 41c 2d2 ba3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 4c8 fb0 235 a4e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0002
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 002 338 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e51 184 9c0
14 5c1 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b6e 66a 16c ef3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 848 e52 495 d4e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0004
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 004 738 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e6a 63b ccc
14 3d3 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bfb 2ba 668 783 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 190 dee ba5 58e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0008
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 008 b38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e5f e4c ac0
14 bcf 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bde f1f 849 563 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 2d0 aeb 584 44e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0010
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot      0      1

```

TIA-102.BAAB

```

0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 010 d38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e76 7a6 6fc
14 7ec 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b95 45b 4bb 1f3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 444 4c8 976 68e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0020
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	020	e38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e67	678	a98
14	7ab	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b07	2e1	def	ab3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	954	83b	202	5ce	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0040
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	700	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e48	bed	fcc
14	ee1	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b1e	1dd	0fd	4a3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	924	25d	564	34e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0080
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	8c0	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e58	6ea	cf0
14	da5	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b19	1e2	55b	3c3	890	a12	435	0cf	02f	86e	460

```

18  44f f05 8a5 89d 839 330 528 a86 e4e 24a 124 350
20  33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22  892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 00

MFID = 00

Emerg= 0

TGID = 0100

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	001	1c0	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e78	8e2	2a4
14	b1f	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b11	d86	e53	ec3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	7f8	b87	443	40e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 0200

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	002	2c0	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e7b	7f3	e30
14	649	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b00	14a	8b6	493	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	e50	668	b48	48e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 0400

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	004	340	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e7d	3d1	31c
14	cef	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b22	ccc	529	0d3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	d9c	d13	67e	ace	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 0800

Sourc= 000001

TIA-102.BAAB

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	008	380	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e71	f92	554
14	99a	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b61	bcf	edf	b73	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	a14	b78	f33	30e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 1000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	1c0	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e78	892	7a8
14	ede	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b1e	285	84a	2d3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	e20	5d6	0e8	b4e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 2000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	230	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e7b	711	c3c
14	dc1	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b19	37b	4b9	c13	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	d10	a16	31f	3ce	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 4000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	470	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240

```

12 d43 3c0 be1 b91 844 ff0 162 962 760 e7d 31e 320
14 3d3 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b11 c97 dee 313 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 b08 55a 550 00e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 00

MFID = 00

Emerg= 0

TGID = 8000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	8b0	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e71	f07	130
14	bcf	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b00	367	fb1	e23	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	704	b0a	99e	dce	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 1

TGID = 0001

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	08c	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e6b	a55	9e0
14	94a	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	bc0	cd0	aea	0e3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	f3c	c83	1fe	50e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000002

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	008	ce2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e65	561	a0c
14	a85	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b83	a55	61b	033	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	528	e22	2db	78e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00

MFID = 00

TIA-102.BAAB

```

Emerg= 0
TGID = 0001
Sourc= 000004
Dest = 000001
Link Data Unit 1:
Microslot
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 011 ce2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e75 819 0b4
14 22d 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bc0 ee1 fb0 643 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 694 d12 126 18e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000008
Dest = 000001
Link Data Unit 1:
Microslot
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 022 ce2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e55 6ea de0
14 b67 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b43 788 c7e 993 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 190 b0a 76c 7ce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000010
Dest = 000001
Link Data Unit 1:
Microslot
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 043 4e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e56 907 344
14 1e2 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b4c 94a bf7 443 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 fb0 706 ba8 38e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000020
Dest = 000001
Link Data Unit 1:
Microslot
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000

```



```

6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 083 8e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e50 dd1 230
14 4d0 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b55 8f3 4c8 ef3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 340 fc3 3c0 8ce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000040
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 001 c00 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e6f d9d a74
14 9fe 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bcf 0f3 d12 6a3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 2d0 b0b 67e dce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000080
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 002 300 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e61 2ed c58
14 cf3 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b5a 794 956 a63 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 924 704 97d 10e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000100
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 004 700 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e7d 30f c20
14 efd 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b7f 969 1e3 203 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 e50 fc3 44a 98e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

TIA-102.BAAB

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000200
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 008 b00 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e45 bc0 ccc
14 2c2 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b34 0b4 000 233 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 0fc eac d05 a4e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000400
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 010 d00 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e76 755 90c
14 6a6 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 ba6 71f 3be 123 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 d10 c0f cfb 78e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000800
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2  0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4  90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6  000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8  89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 020 e00 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e53 270 684
14 66a 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b8b 464 4ba 493 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 668 925 dd6 18e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 001000
Dest = 000001
Link Data Unit 1:
Microslot      0      1

```

```

0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 700 000 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e6e 178 8cc
14 023 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b33 08b 696 c23 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 8b4 705 a5a 14e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 002000
Dest = 000001
Link Data Unit 1:

```

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 8c0 000 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e63 12a 10c
14 758 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bae b7b eaf ec3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 d60 fc1 2e4 b8e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 004000
Dest = 000001
Link Data Unit 1:

```

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 001 1c0 000 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e79 48b 684
14 586 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b9a 8a8 e51 a53 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 6e4 eac 239 54e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 008000
Dest = 000001
Link Data Unit 1:

```

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 002 2c0 000 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e4d 0c7 db8
14 41c 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bf3 f3c fc1 113 890 a12 435 0cf 02f 86e 460

```

TIA-102.BAAB

```

18  44f f05 8a5 89d 839 11c  c0e 2d2 2ce 24a 124 350
20  33c 0be 1b9 184 4ff 058  296 276 0ec 000 000 00c
22  892 849 0d4 33c 0be 1b8  461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 010000
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	004	340	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e66	a5d	7ec
14	33c	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b22	c23	cf0	783	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	eac	926	124	44e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 020000
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	008	380	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e73	c61	b78
14	d4a	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b8c	631	aea	ba3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	16c	332	768	c4e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 040000
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	1c0	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e6e	1c0	fcc
14	fec	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	bc8	3ab	838	cc3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	b84	c82	850	00e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 080000

```

Dest = 000001
Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	230	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e63	155	b3c
14	0ef	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b52	90d	343	d33	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	b78	83b	580	70e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 100000
Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	470	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e79	470	6dc
14	ade	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b6e	478	529	c13	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	ae8	11c	f30	fce	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 200000
Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	8b0	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e4d	03b	d10
14	685	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b16	e99	9a9	dd3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	9d8	3cf	881	e0e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 400000
Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	000	000	001	c38	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b4	0d0	000	000	0e2	4a1	240

TIA-102.BAAB

```

12 d43 3c0 be1 b91 844 ff0 162 962 760 e66 aa9 aa4
14 a2d 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 be5 367 071 fe3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 fc0 669 563 cce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 800000
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 000
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b8 0e0 000 000 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e73 c84 9fc
14 b67 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b08 c8b 331 b93 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 3cc d60 cc7 f8e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 c0c
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 000 000 001 c38 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e57 5fb 2e0
14 575 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b11 f88 84b cf3 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 8c4 b87 03c f0e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 010000
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 c0c
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 001 0d0 000 038 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e53 292 684
14 447 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b92 720 193 033 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 3cc 340 3fa e8e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

```

LCF = 03
MFID = 00

```

```

Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 020000
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	c0c
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	002	0e0	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e63	1f3	ea4
14	800	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	bd1	0b4	8b4	1c3	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	2a0	b0b	3c0	8ce	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 040000
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	c0c
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	070	000	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e5b	9fb	148
14	847	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b61	a85	071	323	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	954	447	889	54e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 080000
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	c0c
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	839	08c	000	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	3b0	000	000	007	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	e73	c22	d10
14	400	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	b3c	ff0	b08	473	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	839	774	528	647	88e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0ec	000	000	00c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 100000
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	dce	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0e4	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	c00	c0c

TIA-102.BAAB

```

6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 11c 000 000 038 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e60 e9d 9b8
14 48b 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b84 911 d62 a63 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 b84 78b 94a e8e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 200000
Link Data Unit 1:

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 c0c
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 22c 000 000 038 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e46 4ea cf0
14 d86 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bfb 0ef 1e3 583 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 2d0 2a0 470 8ce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 400000
Link Data Unit 1:

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 c0c
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 434 000 000 038 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e49 700 274
14 b88 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 b00 303 838 b43 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 08c 8b7 d34 c8e 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```

LCF = 03
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000001
Dest = 800000
Link Data Unit 1:

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 554 7bc b19 4d0 dce 24a 124
2 0d4 33c 0be 1b9 184 4fc c16 296 276 0e4 e24 a10
4 90d 433 c0b e1b 918 44c fc1 629 627 60e c00 c0c
6 000 003 892 849 0d4 33c 02f 86e 461 13f c16 294
8 89d 839 838 000 000 038 24a 124 350 cf0 2f8 6e4
10 184 4ff 058 a58 9d8 3b0 000 000 007 0e2 4a1 240
12 d43 3c0 be1 b91 844 ff0 162 962 760 e57 5de f44
14 3b7 389 284 90d 433 c08 f86 e46 113 fc1 629 624
16 d83 bfc 0f3 b7b 443 890 a12 435 0cf 02f 86e 460
18 44f f05 8a5 89d 839 434 c7c ccc 7ce 24a 124 350
20 33c 0be 1b9 184 4ff 058 296 276 0ec 000 000 00c
22 892 849 0d4 33c 0be 1b8 461 13f c16 296 276 0e4

```


Case 2: Repeater Radio

NAC = 293
 SSym = 1
 LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0001
 Sourc= 000001
 Dest = 000001
 LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 00
 2: 09b 088 0cc 621 7b4 02a 04c 01
 3: 09b 088 0cc 621 7b4 02a 04c 00
 4: 09b 088 0cc 621 7b4 02a 04c 01
 5: 09b 088 0cc 621 7b4 02a 04c 00
 6: 09b 088 0cc 621 7b4 02a 04c 01
 7: 09b 088 0cc 621 7b4 02a 04c 00
 8: 09b 088 0cc 621 7b4 02a 04c 01
 9: 09b 088 0cc 621 7b4 02a 04c 00

Link Data Unit 1:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125		
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11		
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001		
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295		
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5		
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241		
12	d43	3c0	be1	b91	844	ff1	162	962	760	e6d	e5d	549		
14	ade	389	284	90d	433	c09	f86	e46	113	fc1	629	625		
16	d83	ba1	41c	2d2	ba3	891	a12	435	0cf	02f	86e	461		
18	44f	f05	8a5	89d	839	4c9	fb0	235	a4e	24a	124	351		
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d		
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5		

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0002
 Sourc= 000001
 Dest = 000001

Link Data Unit 1:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125		
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11		
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001		
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295		
8	89d	839	000	000	002	339	24a	124	350	cf0	2f8	6e5		
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241		
12	d43	3c0	be1	b91	844	ff1	162	962	760	e51	184	9c1		
14	5c1	389	284	90d	433	c09	f86	e46	113	fc1	629	625		
16	d83	b6e	66a	16c	ef3	891	a12	435	0cf	02f	86e	461		
18	44f	f05	8a5	89d	839	849	e52	495	d4e	24a	124	351		
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d		
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5		

TIA-102.BAAB

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0004
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0												1											
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125												
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11												
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001												
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295												
8	89d	839	000	000	004	739	24a	124	350	cf0	2f8	6e5												
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241												
12	d43	3c0	be1	b91	844	ff1	162	962	760	e6a	63b	ccd												
14	3d3	389	284	90d	433	c09	f86	e46	113	fc1	629	625												
16	d83	bfb	2ba	668	783	891	a12	435	0cf	02f	86e	461												
18	44f	f05	8a5	89d	839	191	dee	ba5	58e	24a	124	351												
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d												
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5												

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0008
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0												1											
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125												
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11												
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001												
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295												
8	89d	839	000	000	008	b39	24a	124	350	cf0	2f8	6e5												
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241												
12	d43	3c0	be1	b91	844	ff1	162	962	760	e5f	e4c	ac1												
14	bcf	389	284	90d	433	c09	f86	e46	113	fc1	629	625												
16	d83	bde	f1f	849	563	891	a12	435	0cf	02f	86e	461												
18	44f	f05	8a5	89d	839	2d1	aeb	584	44e	24a	124	351												
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d												
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5												

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0010
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0												1											
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125												
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11												
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001												
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295												
8	89d	839	000	000	010	d39	24a	124	350	cf0	2f8	6e5												
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241												
12	d43	3c0	be1	b91	844	ff1	162	962	760	e76	7a6	6fd												
14	7ec	389	284	90d	433	c09	f86	e46	113	fc1	629	625												
16	d83	b95	45b	4bb	1f3	891	a12	435	0cf	02f	86e	461												
18	44f	f05	8a5	89d	839	445	4c8	976	68e	24a	124	351												
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d												
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5												

LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0020
 Sourc= 000001
 Dest = 000001
 Link Data Unit 1:

Microslot	0												1											
-----------	---	--	--	--	--	--	--	--	--	--	--	--	---	--	--	--	--	--	--	--	--	--	--	--

0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	020	e39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e67	678	a99
14	7ab	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b07	2e1	def	ab3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	955	83b	202	5ce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0040

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	700	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e48	bed	fcd
14	ee1	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b1e	1dd	0fd	4a3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	925	25d	564	34e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0080

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	8c0	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e58	6ea	cf1
14	da5	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b19	1e2	55b	3c3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	331	528	a86	e4e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0100

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	001	1c0	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e78	8e2	2a5
14	b1f	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b11	d86	e53	ec3	891	a12	435	0cf	02f	86e	461

TIA-102.BAAB

```

18  44f f05 8a5 89d 839 7f9 b87 443 40e 24a 124 351
20  33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22  892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0200
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	002	2c0	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e7b	7f3	e31
14	649	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b00	14a	8b6	493	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	e51	668	b48	48e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0400
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	004	340	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e7d	3d1	31d
14	cef	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b22	ccc	529	0d3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	d9d	d13	67e	ace	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0800
Sourc= 000001
Dest = 000001
Link Data Unit 1:
Microslot

```

	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	008	380	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e71	f92	555
14	99a	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b61	bcf	edf	b73	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	a15	b78	f33	30e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 1000
Sourc= 000001

```

Dest = 000001
Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	1c0	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e78	892	7a9
14	ede	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b1e	285	84a	2d3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	e21	5d6	0e8	b4e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 2000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	230	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e7b	711	c3d
14	dc1	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b19	37b	4b9	c13	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	d11	a16	31f	3ce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 4000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	470	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e7d	31e	321
14	3d3	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b11	c97	dee	313	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	b09	55a	550	00e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 8000

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	8b0	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241

TIA-102.BAAB

```

12 d43 3c0 be1 b91 844 ff1 162 962 760 e71 f07 131
14 bcf 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b00 367 fb1 e23 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 705 b0a 99e dce 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 1
TGID = 0001
Sourc= 000001
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 08c 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 000 007 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e6b a55 9e1
14 94a 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 bc0 cd0 aea 0e3 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 f3d c83 1fe 50e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000002
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 000 008 ce2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e65 561 a0d
14 a85 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b83 a55 61b 033 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 529 e22 2db 78e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000004
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 000 011 ce2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e75 819 0b5
14 22d 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 bc0 ee1 fb0 643 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 695 d12 126 18e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00

```

```

Emerg= 0
TGID = 0001
Sourc= 000008
Dest = 000001
Link Data Unit 1:
Microslot
0      557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2      0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4      90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6      000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8      89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10     184 4ff 058 a58 9d8 3b1 000 000 022 ce2 4a1 241
12     d43 3c0 be1 b91 844 ff1 162 962 760 e55 6ea de1
14     b67 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16     d83 b43 788 c7e 993 891 a12 435 0cf 02f 86e 461
18     44f f05 8a5 89d 839 191 b0a 76c 7ce 24a 124 351
20     33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22     892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000010
Dest = 000001
Link Data Unit 1:
Microslot
0      557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2      0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4      90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6      000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8      89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10     184 4ff 058 a58 9d8 3b1 000 000 043 4e2 4a1 241
12     d43 3c0 be1 b91 844 ff1 162 962 760 e56 907 345
14     1e2 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16     d83 b4c 94a bf7 443 891 a12 435 0cf 02f 86e 461
18     44f f05 8a5 89d 839 fb1 706 ba8 38e 24a 124 351
20     33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22     892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000020
Dest = 000001
Link Data Unit 1:
Microslot
0      557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2      0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4      90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6      000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8      89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10     184 4ff 058 a58 9d8 3b1 000 000 083 8e2 4a1 241
12     d43 3c0 be1 b91 844 ff1 162 962 760 e50 dd1 231
14     4d0 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16     d83 b55 8f3 4c8 ef3 891 a12 435 0cf 02f 86e 461
18     44f f05 8a5 89d 839 341 fc3 3c0 8ce 24a 124 351
20     33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22     892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000040
Dest = 000001
Link Data Unit 1:
Microslot
0      557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2      0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4      90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001

```

TIA-102.BAAB

```

6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 001 c00 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e6f d9d a75
14 9fe 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 bcf 0f3 d12 6a3 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 2d1 b0b 67e dce 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000080
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 002 300 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e61 2ed c59
14 cf3 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b5a 794 956 a63 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 925 704 97d 10e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000100
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 004 700 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e7d 30f c21
14 efd 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b7f 969 1e3 203 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 e51 fc3 44a 98e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000200
Dest = 000001
Link Data Unit 1:

```

```

Microslot      0      1
0 557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2 0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4 90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 008 b00 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e45 bc0 ccd
14 2c2 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b34 0b4 000 233 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 0fd eac d05 a4e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```



```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000400
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2  0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4  90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6  000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8  89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 010 d00 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e76 755 90d
14 6a6 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 ba6 71f 3be 123 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 d11 c0f cfb 78e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 000800
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2  0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4  90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6  000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8  89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 020 e00 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e53 270 685
14 66a 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b8b 464 4ba 493 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 669 925 dd6 18e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 001000
Dest = 000001
Link Data Unit 1:
Microslot      0      1
0  557 5f5 ff7 7ff 293 555 7bc b19 4d0 dce 24a 125
2  0d4 33c 0be 1b9 184 4fd c16 296 276 0e4 e24 a11
4  90d 433 c0b e1b 918 44d fc1 629 627 60e c00 001
6  000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8  89d 839 000 000 001 c39 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 700 000 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e6e 178 8cd
14 023 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b33 08b 696 c23 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 8b5 705 a5a 14e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

```

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 002000
Dest = 000001
Link Data Unit 1:
Microslot      0      1

```

TIA-102.BAAB

0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	8c0	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e63	12a	10d
14	758	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	bae	b7b	eaf	ec3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	d61	fc1	2e4	b8e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 004000
Dest = 000001
Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	001	1c0	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e79	48b	685
14	586	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b9a	8a8	e51	a53	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	6e5	eac	239	54e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 008000
Dest = 000001
Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	002	2c0	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e4d	0c7	db9
14	41c	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	bf3	f3c	fc1	113	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	11d	c0e	2d2	2ce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00
MFID = 00
Emerg= 0
TGID = 0001
Sourc= 010000
Dest = 000001
Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	004	340	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e66	a5d	7ed
14	33c	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b22	c23	cf0	783	891	a12	435	0cf	02f	86e	461

```

18  44f f05 8a5 89d 839 ead 926 124 44e 24a 124 351
20  33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22  892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 020000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	008	380	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e73	c61	b79
14	d4a	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b8c	631	aea	ba3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	16d	332	768	c4e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 040000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	1c0	000	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e6e	1c0	fed
14	fec	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	bc8	3ab	838	cc3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	b85	c82	850	00e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 080000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	230	000	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e63	155	b3d
14	0ef	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b52	90d	343	d33	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	b79	83b	580	70e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 100000

TIA-102.BAAB

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	470	000	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e79	470	6dd
14	ade	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b6e	478	529	c13	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	ae9	11c	f30	fce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 200000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	8b0	000	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e4d	03b	d11
14	685	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b16	e99	9a9	dd3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	9d9	3cf	881	e0e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 400000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b5	0d0	000	000	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e66	aa9	aa5
14	a2d	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	be5	367	071	fe3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	fc1	669	563	cce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 00

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 800000

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	001
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b9	0e0	000	000	0e2	4a1	241

```

12 d43 3c0 be1 b91 844 ff1 162 962 760 e73 c84 9fd
14 b67 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 b08 c8b 331 b93 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 3cd d60 cc7 f8e 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 000001

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	000	000	001	c39	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e57	5fb	2e1
14	575	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b11	f88	84b	cf3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	8c5	b87	03c	f0e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 010000

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	001	0d0	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e53	292	685
14	447	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b92	720	193	033	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	3cd	340	3fa	e8e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 020000

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	002	0e0	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e63	1f3	ea5
14	800	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	bd1	0b4	8b4	1c3	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	2a1	b0b	3c0	8ce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 03

MFID = 00

TIA-102.BAAB

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 040000

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	839	070	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	e5b	9fb	149				
14	847	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	b61	a85	071	323	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	839	955	447	889	54e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5				

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 080000

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	839	08c	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	e73	c22	d11				
14	400	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	b3c	ff0	b08	473	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	839	775	528	647	88e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5				

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 100000

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	839	11c	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	e60	e9d	9b9				
14	48b	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	b84	911	d62	a63	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	839	b85	78b	94a	e8e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5				

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 200000

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d				

```

6 000 003 892 849 0d4 33d 02f 86e 461 13f c16 295
8 89d 839 22c 000 000 039 24a 124 350 cf0 2f8 6e5
10 184 4ff 058 a58 9d8 3b1 000 000 007 0e2 4a1 241
12 d43 3c0 be1 b91 844 ff1 162 962 760 e46 4ea cf1
14 d86 389 284 90d 433 c09 f86 e46 113 fc1 629 625
16 d83 bfb 0ef 1e3 583 891 a12 435 0cf 02f 86e 461
18 44f f05 8a5 89d 839 2d1 2a0 470 8ce 24a 124 351
20 33c 0be 1b9 184 4ff 059 296 276 0ec 000 000 00d
22 892 849 0d4 33c 0be 1b9 461 13f c16 296 276 0e5

```

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 400000

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	434	000	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e49	700	275
14	b88	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	b00	303	838	b43	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	08d	8b7	d34	c8e	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

LCF = 03

MFID = 00

Emerg= 0

TGID = 0001

Sourc= 000001

Dest = 800000

Link Data Unit 1:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	555	7bc	b19	4d0	dce	24a	125
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0e4	e24	a11
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	c00	c0d
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295
8	89d	839	838	000	000	039	24a	124	350	cf0	2f8	6e5
10	184	4ff	058	a58	9d8	3b1	000	000	007	0e2	4a1	241
12	d43	3c0	be1	b91	844	ff1	162	962	760	e57	5de	f45
14	3b7	389	284	90d	433	c09	f86	e46	113	fc1	629	625
16	d83	bfc	0f3	b7b	443	891	a12	435	0cf	02f	86e	461
18	44f	f05	8a5	89d	839	435	c7c	ccc	7ce	24a	124	351
20	33c	0be	1b9	184	4ff	059	296	276	0ec	000	000	00d
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0e5

B.5 Unencrypted Link Data Unit 2 Test Vectors

Case 1: Subscriber Radio

NAC = 293
 SSym = 0
 MI = 00 0000 0000 0000 0000
 ALGID= 80
 KID = 0000
 LSD = 40 41

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 01
 2: 09b 088 0cc 621 7b4 02a 04c 00
 3: 09b 088 0cc 621 7b4 02a 04c 01
 4: 09b 088 0cc 621 7b4 02a 04c 00
 5: 09b 088 0cc 621 7b4 02a 04c 01
 6: 09b 088 0cc 621 7b4 02a 04c 00
 7: 09b 088 0cc 621 7b4 02a 04c 01
 8: 09b 088 0cc 621 7b4 02a 04c 00
 9: 09b 088 0cc 621 7b4 02a 04c 01

Link Data Unit 2:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	ace	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0ec	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	400	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	83b	000	000	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	390	000	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	ee0	e00	000
14	000	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	9ae	8b4	8b6	493	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	83b	9a8	f4f	1fd	60e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0e5	009	d04	78c
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0ec

NAC = 293
 SSym = 0
 MI = 00 0000 0000 0000 0000
 ALGID= 80
 KID = 0000
 LSD = 42 44

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 01
 2: 09b 088 0cc 621 7b4 02a 04c 00
 3: 09b 088 0cc 621 7b4 02a 04c 01
 4: 09b 088 0cc 621 7b4 02a 04c 00
 5: 09b 088 0cc 621 7b4 02a 04c 01
 6: 09b 088 0cc 621 7b4 02a 04c 00
 7: 09b 088 0cc 621 7b4 02a 04c 01
 8: 09b 088 0cc 621 7b4 02a 04c 00
 9: 09b 088 0cc 621 7b4 02a 04c 01

Link Data Unit 2:

Microslot	0						1					
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	ace	24a	124
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0ec	e24	a10
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	400	000
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294
8	89d	83b	000	000	000	038	24a	124	350	cf0	2f8	6e4
10	184	4ff	058	a58	9d8	390	000	000	000	0e2	4a1	240
12	d43	3c0	be1	b91	844	ff0	162	962	760	ee0	e00	000
14	000	389	284	90d	433	c08	f86	e46	113	fc1	629	624
16	d83	9ae	8b4	8b6	493	890	a12	435	0cf	02f	86e	460
18	44f	f05	8a5	89d	83b	9a8	f4f	1fd	60e	24a	124	350
20	33c	0be	1b9	184	4ff	058	296	276	0e5	095	513	0cc
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0ec

NAC = 293


```

SSym = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
MI    = 00 0000 0000 0000 0000
ALGID= 80
KID   = 0000
LSD   = 48 50

```

```

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	ace	24a	124				
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0ec	e24	a10				
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	400	000				
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294				
8	89d	83b	000	000	000	038	24a	124	350	cf0	2f8	6e4				
10	184	4ff	058	a58	9d8	390	000	000	000	0e2	4a1	240				
12	d43	3c0	be1	b91	844	ff0	162	962	760	ee0	e00	000				
14	000	389	284	90d	433	c08	f86	e46	113	fc1	629	624				
16	d83	9ae	8b4	8b6	493	890	a12	435	0cf	02f	86e	460				
18	44f	f05	8a5	89d	83b	9a8	f4f	1fd	60e	24a	124	350				
20	33c	0be	1b9	184	4ff	058	296	276	0e5	235	943	f0c				
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0ec				

```

NAC   = 293
SSym  = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
MI    = 00 0000 0000 0000 0000
ALGID= 80
KID   = 0000
LSD   = 70 80

```

```

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	ace	24a	124				
2	0d4	33c	0be	1b9	184	4fc	c16	296	276	0ec	e24	a10				
4	90d	433	c0b	e1b	918	44c	fc1	629	627	60e	400	000				
6	000	003	892	849	0d4	33c	02f	86e	461	13f	c16	294				
8	89d	83b	000	000	000	038	24a	124	350	cf0	2f8	6e4				
10	184	4ff	058	a58	9d8	390	000	000	000	0e2	4a1	240				
12	d43	3c0	be1	b91	844	ff0	162	962	760	ee0	e00	000				
14	000	389	284	90d	433	c08	f86	e46	113	fc1	629	624				
16	d83	9ae	8b4	8b6	493	890	a12	435	0cf	02f	86e	460				
18	44f	f05	8a5	89d	83b	9a8	f4f	1fd	60e	24a	124	350				
20	33c	0be	1b9	184	4ff	058	296	276	0e5	c1c	e01	38c				
22	892	849	0d4	33c	0be	1b8	461	13f	c16	296	276	0ec				

Case 2: Repeater Radio

```

NAC   = 293
SSym  = 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
MI    = 00 0000 0000 0000 0000
ALGID= 80
KID   = 0000
LSD   = 40 41
IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7

```

TIA-102.BAAB

```

1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab9	a4e	fb0	9a8	ace	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0ec	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	400	001				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	83b	000	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	391	000	000	000	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	ee0	e00	001				
14	000	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	9ae	8b4	8b6	493	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	83b	9a9	f4f	1fd	60e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0e5	009	d04	78d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0ed				

NAC = 293

SSym = 1

MI = 00 0000 0000 0000 0000

ALGID= 80

KID = 0000

LSD = 42 44

```

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab9	a4e	fb0	9a8	ace	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0ec	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	400	001				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	83b	000	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	391	000	000	000	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	ee0	e00	001				
14	000	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	9ae	8b4	8b6	493	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	83b	9a9	f4f	1fd	60e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0e5	095	513	0cd				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0ed				

NAC = 293

SSym = 1

MI = 00 0000 0000 0000 0000

ALGID= 80

KID = 0000

LSD = 48 50

```

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
1: 09b 088 0cc 621 7b4 02a 04c 01
2: 09b 088 0cc 621 7b4 02a 04c 00
3: 09b 088 0cc 621 7b4 02a 04c 01
4: 09b 088 0cc 621 7b4 02a 04c 00
5: 09b 088 0cc 621 7b4 02a 04c 01
6: 09b 088 0cc 621 7b4 02a 04c 00
7: 09b 088 0cc 621 7b4 02a 04c 01
8: 09b 088 0cc 621 7b4 02a 04c 00
9: 09b 088 0cc 621 7b4 02a 04c 01

```

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab9	a4e	fb0	9a8	ace	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0ec	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	400	001				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	83b	000	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	391	000	000	000	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	ee0	e00	001				
14	000	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	9ae	8b4	8b6	493	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	83b	9a9	f4f	1fd	60e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0e5	235	943	f0d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0ed				

NAC = 293

SSym = 1

MI = 00 0000 0000 0000 0000

ALGID= 80

KID = 0000

LSD = 70 80

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7

1: 09b 088 0cc 621 7b4 02a 04c 01

2: 09b 088 0cc 621 7b4 02a 04c 00

3: 09b 088 0cc 621 7b4 02a 04c 01

4: 09b 088 0cc 621 7b4 02a 04c 00

5: 09b 088 0cc 621 7b4 02a 04c 01

6: 09b 088 0cc 621 7b4 02a 04c 00

7: 09b 088 0cc 621 7b4 02a 04c 01

8: 09b 088 0cc 621 7b4 02a 04c 00

9: 09b 088 0cc 621 7b4 02a 04c 01

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab9	a4e	fb0	9a8	ace	24a	125				
2	0d4	33c	0be	1b9	184	4fd	c16	296	276	0ec	e24	a11				
4	90d	433	c0b	e1b	918	44d	fc1	629	627	60e	400	001				
6	000	003	892	849	0d4	33d	02f	86e	461	13f	c16	295				
8	89d	83b	000	000	000	039	24a	124	350	cf0	2f8	6e5				
10	184	4ff	058	a58	9d8	391	000	000	000	0e2	4a1	241				
12	d43	3c0	be1	b91	844	ff1	162	962	760	ee0	e00	001				
14	000	389	284	90d	433	c09	f86	e46	113	fc1	629	625				
16	d83	9ae	8b4	8b6	493	891	a12	435	0cf	02f	86e	461				
18	44f	f05	8a5	89d	83b	9a9	f4f	1fd	60e	24a	124	351				
20	33c	0be	1b9	184	4ff	059	296	276	0e5	c1c	e01	38d				
22	892	849	0d4	33c	0be	1b9	461	13f	c16	296	276	0ed				

B.6 Silent Audio Test Vectors

Case 1: Subscriber Radio

NAC = 293
 SSym = 0
 LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0001
 Sourc= 000001
 Dest = 000001
 LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7
 1: 040 cfd 7bf b7d 793 6cf 33c 44
 2: 040 cfd 7bf b7d 793 6cf 33c 45
 3: 040 cfd 7bf b7d 793 6cf 33c 44
 4: 040 cfd 7bf b7d 793 6cf 33c 45
 5: 040 cfd 7bf b7d 793 6cf 33c 44
 6: 040 cfd 7bf b7d 793 6cf 33c 45
 7: 040 cfd 7bf b7d 793 6cf 33c 44
 8: 040 cfd 7bf b7d 793 6cf 33c 45
 9: 040 cfd 7bf b7d 793 6cf 33c 44

Link Data Unit 1:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	554	7bc	b19	4d0	ddb	10b	a14				
2	de2	e82	693	63d	981	f98	f88	ec6	2b8	011	b10	ba0				
4	5de	2e8	269	363	d98	1f8	6f8	8ec	62b	801	800	000				
6	000	006	c42	e85	de2	e80	9a4	d8f	660	7e6	f88	ec4				
8	8ae	004	000	000	001	c6c	10b	a17	78b	a09	a4d	8f4				
10	981	f9b	e23	b18	ae0	060	000	000	007	1b1	0ba	174				
12	e2e	826	936	3d9	81f	9bc	88e	c62	b80	12d	e5d	548				
14	ade	6c4	2e8	5de	2e8	268	4d8	f66	07e	6f8	8ec	628				
16	e00	6a1	41c	2d2	ba6	c40	ba1	778	ba0	9a4	d8f	660				
18	1f9	be2	3b1	8ae	004	4c8	fb0	235	a5b	10b	a17	788				
20	e82	693	63d	981	f9b	e20	ec6	2b8	018	000	000	018				
22	c42	e85	de2	e82	693	63c	660	7e6	f88	ec6	2b8	010				

NAC = 293
 SSym = 0
 MI = 00 0000 0000 0000 0000
 ALGID= 80
 KID = 0000
 LSD = 40 40

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7
 1: 040 cfd 7bf b7d 793 6cf 33c 45
 2: 040 cfd 7bf b7d 793 6cf 33c 44
 3: 040 cfd 7bf b7d 793 6cf 33c 45
 4: 040 cfd 7bf b7d 793 6cf 33c 44
 5: 040 cfd 7bf b7d 793 6cf 33c 45
 6: 040 cfd 7bf b7d 793 6cf 33c 44
 7: 040 cfd 7bf b7d 793 6cf 33c 45
 8: 040 cfd 7bf b7d 793 6cf 33c 44
 9: 040 cfd 7bf b7d 793 6cf 33c 45

Link Data Unit 2:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab8	a4e	fb0	9a8	adb	10b	a14				
2	de2	e82	693	63d	981	f98	f88	ec6	2b8	019	b10	ba0				
4	5de	2e8	269	363	d98	1f8	6f8	8ec	62b	801	000	000				
6	000	006	c42	e85	de2	e80	9a4	d8f	660	7e6	f88	ec4				
8	8ae	006	000	000	000	06c	10b	a17	78b	a09	a4d	8f4				
10	981	f9b	e23	b18	ae0	040	000	000	000	1b1	0ba	174				
12	e2e	826	936	3d9	81f	9bc	88e	c62	b80	1a0	e00	000				
14	000	6c4	2e8	5de	2e8	268	4d8	f66	07e	6f8	8ec	628				
16	e00	4ae	8b4	8b6	496	c40	ba1	778	ba0	9a4	d8f	660				
18	1f9	be2	3b1	8ae	006	9a8	f4f	1fd	61b	10b	a17	788				
20	e82	693	63d	981	f9b	e20	ec6	2b8	011	009	d00	9d8				
22	c42	e85	de2	e82	693	63c	660	7e6	f88	ec6	2b8	018				

Case 2: Repeater Radio

NAC = 293
 SSym = 1
 LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0001
 Sourc= 000001
 Dest = 000001
 LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 040 cfd 7bf b7d 793 6cf 33c 44
 2: 040 cfd 7bf b7d 793 6cf 33c 45
 3: 040 cfd 7bf b7d 793 6cf 33c 44
 4: 040 cfd 7bf b7d 793 6cf 33c 45
 5: 040 cfd 7bf b7d 793 6cf 33c 44
 6: 040 cfd 7bf b7d 793 6cf 33c 45
 7: 040 cfd 7bf b7d 793 6cf 33c 44
 8: 040 cfd 7bf b7d 793 6cf 33c 45
 9: 040 cfd 7bf b7d 793 6cf 33c 44

Link Data Unit 1:

Microslot	0	1
0	557 5f5 ff7 7ff 293 555	7bc b19 4d0 ddb 10b a15
2	de2 e82 693 63d 981 f99	f88 ec6 2b8 011 b10 ba1
4	5de 2e8 269 363 d98 1f9	6f8 8ec 62b 801 800 001
6	000 006 c42 e85 de2 e81	9a4 d8f 660 7e6 f88 ec5
8	8ae 004 000 000 001 c6d	10b a17 78b a09 a4d 8f5
10	981 f9b e23 b18 ae0 061	000 000 007 1b1 0ba 175
12	e2e 826 936 3d9 81f 9bd	88e c62 b80 12d e5d 549
14	ade 6c4 2e8 5de 2e8 269	4d8 f66 07e 6f8 8ec 629
16	e00 6a1 41c 2d2 ba6 c41	ba1 778 ba0 9a4 d8f 661
18	1f9 be2 3b1 8ae 004 4c9	fb0 235 a5b 10b a17 789
20	e82 693 63d 981 f9b e21	ec6 2b8 018 000 000 019
22	c42 e85 de2 e82 693 63d	660 7e6 f88 ec6 2b8 011

NAC = 293
 SSym = 1
 MI = 00 0000 0000 0000 0000
 ALGID= 80
 KID = 0000
 LSD = 40 40

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 040 cfd 7bf b7d 793 6cf 33c 45
 2: 040 cfd 7bf b7d 793 6cf 33c 44
 3: 040 cfd 7bf b7d 793 6cf 33c 45
 4: 040 cfd 7bf b7d 793 6cf 33c 44
 5: 040 cfd 7bf b7d 793 6cf 33c 45
 6: 040 cfd 7bf b7d 793 6cf 33c 44
 7: 040 cfd 7bf b7d 793 6cf 33c 45
 8: 040 cfd 7bf b7d 793 6cf 33c 44
 9: 040 cfd 7bf b7d 793 6cf 33c 45

Link Data Unit 2:

Microslot	0	1
0	557 5f5 ff7 7ff 293 ab9	a4e fb0 9a8 adb 10b a15
2	de2 e82 693 63d 981 f99	f88 ec6 2b8 019 b10 ba1
4	5de 2e8 269 363 d98 1f9	6f8 8ec 62b 801 000 001
6	000 006 c42 e85 de2 e81	9a4 d8f 660 7e6 f88 ec5
8	8ae 006 000 000 000 06d	10b a17 78b a09 a4d 8f5
10	981 f9b e23 b18 ae0 041	000 000 000 1b1 0ba 175
12	e2e 826 936 3d9 81f 9bd	88e c62 b80 1a0 e00 001
14	000 6c4 2e8 5de 2e8 269	4d8 f66 07e 6f8 8ec 629
16	e00 4ae 8b4 8b6 496 c41	ba1 778 ba0 9a4 d8f 661
18	1f9 be2 3b1 8ae 006 9a9	f4f 1fd 61b 10b a17 789
20	e82 693 63d 981 f9b e21	ec6 2b8 011 009 d00 9d9
22	c42 e85 de2 e82 693 63d	660 7e6 f88 ec6 2b8 019

B.7 Confirmed Data Packet Test Vectors

Case 1: Subscriber Radio

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 01
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 02d3
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
      0  00 1fb 01 23 45 67 89 ab cd ef 01 23 45 67 5f 91 af 1d
Data Packet:
Microslot
      0          0          1
      0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 088 8b4
      2  e22 245 052 222 922 228 c08 888 888 48b b48 888
      4  cc2 921 f21 13e 002 f4c b0c 9b8 0c3 5d7 2b8 7ec
      6  694 aa7 5cc 5d2 9f4 7c0 d7f 000 000 000 000 000

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 990f
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
      0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
      1  01 1f8 01 23 45 67 89 ab cd ef 01 23 45 67 a6 74 83 04
Data Packet:
Microslot
      0          0          1
      0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 088 8b4
      2  922 1e5 052 222 922 270 808 888 888 4b7 f48 888
      4  c12 927 823 13e 002 f44 1ac bf8 0c3 5d6 a84 bec
      6  694 aa7 8c1 7d2 9f4 7c0 c78 0b8 4f8 00b d13 7c8
      8  fe0 30d 75a cb4 2b6 948 a9c b3e 34a 7d1 f0d b9c

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 0
blocks   = 01
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= dfeb
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
      0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
Data Packet:

```

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd4	82f	80a	ecd	cb1	088	8b8		
2	e22	f85	052	222	222	2b4	808	888	888	48e	b48	888		
4	cc2	928	423	13e	002	f44	1ac	bf8	0c3	5d6	a84	bec		
6	694	aa7	8c1	7d2	9f4	7c0	c78	000	000	000	000	000		

NAC = 293
 SSymbol= 0
 I/O = 0
 SAP = 00
 MFID = 00
 LLID = 000001
 FMF = 1
 blocks = 03
 pads = 00
 sync = 0
 N(S) = 0
 FSNF = 8
 offset = 00
 Hdr CRC= efbb

Data : SN	CRC	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	00	045	01	23	45	67	89	ab	cd	ef	01	23	45	67	89	ab	cd
1	01	12e	12	34	56	78	9a	bc	de	f0	12	34	56	78	9a	bc	de
2	02	0e0	23	45	67	89	ab	cd	ef	01	23	45	67	89	63	8b	97

Data Packet:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd4	82f	80a	ecd	cb1	088	8b4		
2	522	f65	052	222	922	264	008	888	888	491	b48	888		
4	cf2	928	423	13e	002	f44	1ac	bf8	0c3	5d6	a84	bec		
6	694	aa7	8c1	7d2	9f4	7c0	c78	0b1	55e	844	f80	088		
8	b49	64e	e03	0db	e3d	d0c	eda	52a	da2	d6b	4a7	d1c		
10	27f	446	b55	738	ed2	5a8	849	254	e85	de3	078	f58		
12	cb2	c31	e06	8bc	450	000								

NAC = 293
 SSymbol= 0
 I/O = 0
 SAP = 00
 MFID = 00
 LLID = 000001
 FMF = 1
 blocks = 04
 pads = 00
 sync = 0
 N(S) = 0
 FSNF = 8
 offset = 00
 Hdr CRC= be96

Data : SN	CRC	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	00	045	01	23	45	67	89	ab	cd	ef	01	23	45	67	89	ab	cd
1	01	12e	12	34	56	78	9a	bc	de	f0	12	34	56	78	9a	bc	de
2	02	001	23	45	67	89	ab	cd	ef	01	23	45	67	89	ab	cd	ef
3	03	1bd	34	56	78	9a	bc	de	f0	12	34	56	78	9a	c4	c5	65

Data Packet:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd4	82f	80a	ecd	cb1	088	8b4		
2	222	1a9	052	222	922	244	c08	888	b08	4a0	348	888		
4	ce2	926	d23	13e	002	f44	1ac	bf8	0c3	5d6	a84	bec		
6	694	aa7	8c1	7d2	9f4	7c0	c78	0b1	55e	844	f80	088		
8	b49	64e	e03	0db	e3d	d0c	eda	52a	da2	d6b	4a7	d1c		
10	22f	446	b55	7a1	ed7	5a8	849	259	3ba	9e3	078	f74		
12	0f2	c31	e06	8b5	ad2	a3c	800	bd1	1ac	f48	0c3	5d4		
14	a58	cd6	694	aa7	867	2d0	a7d	1f0	c5c	c00	000	000		

NAC = 293
 SSymbol= 0
 I/O = 0
 SAP = 00
 MFID = 00
 LLID = 000001
 FMF = 1
 blocks = 05

TIA-102.BAAB

```

pads      = 00
sync      = 0
N(S)      = 0
FSNF      = 8
offset    = 00
Hdr CRC= c822
Data : SN_CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 00 08b 00 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0 45 24 d4
1 01 1c6 00 51 55 97 61 96 9b 71 d7 9f 82 18 a3 92 59 a7
2 02 0f5 00 a2 9a ab b2 db af c3 1c b3 d3 5d b7 e3 9e bb
3 03 0e3 00 f3 df bf 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0
4 04 1d6 00 45 24 d4 51 55 97 61 96 9b 71 d7 3c 91 ee c0

```

Data Packet:

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 088 8b4
2 e22 f29 052 222 922 250 408 888 b08 486 748 888
4 c02 929 128 2af 7d5 08c 9bc 9b4 610 613 567 7f8
6 634 19c 27f e2e 7be 7b0 a9b cb3 ab6 017 acd ac8
8 6f3 6f3 af3 a64 23b 19c 11f e8b 883 243 143 164
10 228 4c9 72a f8d 6df 3a4 bde cba c87 e01 76c db8
12 4cd 2c5 908 4d1 4d2 840 2eb df5 420 b6f 9f4 610
14 184 d4f 91e 634 19c 2f0 863 9ef 9ec a89 798 eb4
16 805 a12 e65 9f3 6f3 3ac 649 fce c67 271 b09 bc0
18 c90 cea 200 000 000 000

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 0f
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= b53e

```

```

Data : SN_CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 00 117 01 23 45 67 89 ab cd ef 01 23 45 67 89 00 00 00
1 01 1a6 00 00 00 00 00 00 00 00 00 00 24 c4 de e5

```

Data Packet:

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 088 8b4
2 952 1e9 052 222 922 24c c08 888 8bc 4ae 348 888
4 c18 920 628 13e 002 f4c 888 8f8 0c3 5d6 88b bec
6 694 aa7 822 5d2 9f4 7c0 c48 8a4 888 888 898 0c8
8 622 222 222 8d4 222 220 889 c4c 888 888 88a 914

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 0f
sync     = 0
N(S)     = 7
FSNF     = 8
offset   = 00
Hdr CRC= bd67

```

```

Data : SN_CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
0 00 117 01 23 45 67 89 ab cd ef 01 23 45 67 89 00 00 00
1 01 1a6 00 00 00 00 00 00 00 00 00 00 24 c4 de e5

```

Data Packet:

```

Microslot 0 1
0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 088 8b4
2 9b2 105 052 222 923 24c 408 888 8bd 8a1 f48 888
4 c18 92b 328 13e 002 f4c 888 8f8 0c3 5d6 88b bec

```



```

        6 694 aa7 822 5d2 9f4 7c0 c48 8a4 888 888 898 0c8
        8 622 222 222 8d4 222 220 889 c4c 888 888 88a 914

NAC      = 293
SSymbol= 0
I/O      = 1
SAP      = 1f
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 0c
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 1ec9
Data : SN CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
        0 00 0ed 00 00 02 c0 01 23 45 67 89 ab cd ef 01 23 45 67
        1 01 0ab 00 00 00 00 00 00 00 00 00 00 00 14 fc b4 b4
Data Packet:
Microslot
        0
        0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd cb1 148 8b4
        2 922 24e 3b2 222 922 2c4 6cc 888 8bc 48c 760 888
        4 c15 926 72e 2d2 557 a10 4ec 88a 092 593 b83 7c8
        6 6e3 dd0 fb6 222 268 b58 b4a 480 888 888 8a7 fc8
        8 622 222 222 868 f22 220 88b 90e 888 888 88b e08

```

Case 2: Base Station Radio

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 01
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 3bde
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
        0  00 1fb 01 23 45 67 89 ab cd ef 01 23 45 67 5f 91 af 1d
Data Packet:
Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
2  e22 289 352 222 922 2f9 ec8 888 888 498 f48 889
4  cc2 924 621 13e 002 f4d b0c 9b8 0c3 5d7 2b8 7ed
6  694 aa7 5cc 5d2 9f4 7c1 d7f 000 000 000 000 001

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= a002
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
        0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
        1  01 1f8 01 23 45 67 89 ab cd ef 01 23 45 67 a6 74 83 04
Data Packet:
Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
2  922 129 352 222 922 2a1 ac8 888 888 4a4 b48 889
4  c12 922 123 13e 002 f45 1ac bf8 0c3 5d6 a84 bed
6  694 aa7 8c1 7d2 9f4 7c1 c78 0b8 4f8 00b d13 7c9
8  fe0 30d 75a cb4 2b6 949 a9c b3e 34a 7d1 f0d b9d

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 0
blocks   = 01
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= e6e6
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
        0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
Data Packet:
Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b9

```

```

      2  e22 f49 352 222 222 265  ac8 888 888 49d f48 889
      4  cc2 92d d23 13e 002 f45  lac bf8 0c3 5d6 a84 bed
      6  694 aa7 8c1 7d2 9f4 7c1  c78 000 000 000 000 001

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 03
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= d6b6

```

```

Data : SN_CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
      0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
      1  01 12e 12 34 56 78 9a bc de f0 12 34 56 78 9a bc de f0
      2  02 0e0 23 45 67 89 ab cd ef 01 23 45 67 89 63 8b 97 91

```

Data Packet:

```

Microslot      0      1
      0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
      2  522 fa9 352 222 922 2b5 2c8 888 888 482 f48 889
      4  cf2 92d d23 13e 002 f45 lac bf8 0c3 5d6 a84 bed
      6  694 aa7 8c1 7d2 9f4 7c1 c78 0b1 55e 844 f80 089
      8  b49 64e e03 0db e3d d0d eda 52a da2 d6b 4a7 d1d
     10  27f 446 b55 738 ed2 5a9 849 254 e85 de3 078 f59
     12  cb2 c31 e06 8bc 450 001

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 04
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 879b

```

```

Data : SN_CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
      0  00 045 01 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef
      1  01 12e 12 34 56 78 9a bc de f0 12 34 56 78 9a bc de f0
      2  02 001 23 45 67 89 ab cd ef 01 23 45 67 89 ab cd ef 01
      3  03 1bd 34 56 78 9a bc de f0 12 34 56 78 9a c4 c5 65 d5

```

Data Packet:

```

Microslot      0      1
      0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
      2  222 165 352 222 922 295 ec8 888 b08 4b3 748 889
      4  ce2 923 423 13e 002 f45 lac bf8 0c3 5d6 a84 bed
      6  694 aa7 8c1 7d2 9f4 7c1 c78 0b1 55e 844 f80 089
      8  b49 64e e03 0db e3d d0d eda 52a da2 d6b 4a7 d1d
     10  22f 446 b55 7a1 ed7 5a9 849 259 3ba 9e3 078 f75
     12  0f2 c31 e06 8b5 ad2 a3d 800 bd1 lac f48 0c3 5d5
     14  a58 cd6 694 aa7 867 2d1 a7d 1f0 c5c c00 000 001

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 05
pads     = 00
sync     = 0

```

TIA-102.BAAB

```

N(S)      = 0
FSNF      = 8
offset    = 00
Hdr CRC= f12f
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
0  00 08b 00 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0 45 24 d4
1  01 1c6 00 51 55 97 61 96 9b 71 d7 9f 82 18 a3 92 59 a7
2  02 0f5 00 a2 9a ab b2 db af c3 1c b3 d3 5d b7 e3 9e bb
3  03 0e3 00 f3 df bf 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0
4  04 1d6 00 45 24 d4 51 55 97 61 96 9b 71 d7 3c 91 ee c0

```

Data Packet:

```

Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
2  e22 fe5 352 222 922 281 6c8 888 b08 495 348 889
4  c02 92c 828 2af 7d5 08d 9bc 9b4 610 613 567 7f9
6  634 19c 27f e2e 7be 7b1 a9b cb3 ab6 017 acd ac9
8  6f3 6f3 af3 a64 23b 19d 11f e8b 883 243 143 165
10 228 4c9 72a f8d 6df 3a5 bde cba c87 e01 76c db9
12 4cd 2c5 908 4d1 4d2 841 2eb df5 420 b6f 9f4 611
14 184 d4f 91e 634 19c 2f1 863 9ef 9ec a89 798 eb5
16 805 a12 e65 9f3 6f3 3ad 649 fce c67 271 b09 bc1
18 c90 cea 200 000 000 001

```

```

NAC        = 293
SSymbol= 1
I/O        = 1
SAP        = 00
MFID       = 00
LLID       = 000001
FMF        = 1
blocks     = 02
pads       = 0f
sync       = 0
N(S)       = 0
FSNF       = 8
offset     = 00
Hdr CRC= 8c33

```

```

Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
0  00 117 01 23 45 67 89 ab cd ef 01 23 45 67 89 00 00 00
1  01 1a6 00 00 00 00 00 00 00 00 00 00 24 c4 de e5

```

Data Packet:

```

Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
2  952 125 352 222 922 29d ec8 888 8bc 4bd 748 889
4  c18 925 f28 13e 002 f4d 888 8f8 0c3 5d6 88b bed
6  694 aa7 822 5d2 9f4 7c1 c48 8a4 888 888 898 0c9
8  622 222 222 8d4 222 221 889 c4c 888 888 88a 915

```

```

NAC        = 293
SSymbol= 1
I/O        = 1
SAP        = 00
MFID       = 00
LLID       = 000001
FMF        = 1
blocks     = 02
pads       = 0f
sync       = 0
N(S)       = 7
FSNF       = 8
offset     = 00
Hdr CRC= 846a

```

```

Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
0  00 117 01 23 45 67 89 ab cd ef 01 23 45 67 89 00 00 00
1  01 1a6 00 00 00 00 00 00 00 00 00 00 24 c4 de e5

```

Data Packet:

```

Microslot      0      1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 088 8b5
2  9b2 1c9 352 222 923 29d 6c8 888 8bd 8b2 b48 889
4  c18 92e a28 13e 002 f4d 888 8f8 0c3 5d6 88b bed
6  694 aa7 822 5d2 9f4 7c1 c48 8a4 888 888 898 0c9
8  622 222 222 8d4 222 221 889 c4c 888 888 88a 915

```

```

NAC      = 293
SSymbol  = 1
I/O      = 1
SAP      = 1f
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 02
pads     = 0c
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= 1ec9
Data : SN CRC 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
      0 00 0ed 00 00 02 c0 01 23 45 67 89 ab cd ef 01 23 45 67
      1 01 0ab 00 00 00 00 00 00 00 00 00 00 00 14 fc b4 b4
Data Packet:
Microslot      0      1
0 557 5f5 ff7 7ff 293 cd5 82f 80a ecd cb1 148 8b5
2 922 24e 3b2 222 922 2c5 6cc 888 8bc 48c 760 889
4 c15 926 72e 2d2 557 a11 4ec 88a 092 593 b83 7c9
6 6e3 dd0 fb6 222 268 b59 b4a 480 888 888 8a7 fc9
8 622 222 222 868 f22 221 88b 90e 888 888 88b e09

```

B.8 Response Packet Test Vectors

Case 1: Subscriber Radio

```

NAC      = 293
SSymbol= 0
I/O      = 0
Class    = 0
Type     = 1
Status   = 0
MFID     = 00
LLID     = 000001
blocks   = 00
Hdr CRC= 10a6
Data Packet:
Microslot
0          0          1
0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd c89 488 8b4
2  222 219 222 222 922 2c8 884 888 888 8b9 fe4 888
4  c22 222 d00 000 000 000

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
Class    = 1
Type     = 5
Status   = 7
MFID     = 00
LLID     = 000001
blocks   = 00
Hdr CRC= 5928
Data Packet:
Microslot
0          0          1
0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd c8a d48 8b4
2  222 ce2 2d2 222 922 200 490 888 888 8b6 be0 888
4  c22 227 900 000 000 000

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
Class    = 2
Type     = 0
Status   = 0
MFID     = 00
LLID     = 000001
blocks   = 01
Hdr CRC= f05f
Data : 0_1_2_3_4_5_6_7_8_9_10_11
0  f8 ff ff ff ff ff ff ff d7 5b 92 1c
Data Packet:
Microslot
0          0          1
0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd c89 888 8b4
2  e22 fc5 292 222 922 280 088 888 888 894 fc8 888
4  cc2 222 8ff 888 888 8b4 a4a 222 222 22c 1f1 a20
6  888 880 de3 988 888 880 d05 400 000 000 000 000

```

Case 2: Base Station Radio

NAC = 293
 SSymbol= 1
 I/O = 1
 Class = 0
 Type = 1
 Status = 0
 MFID = 00
 LLID = 000001
 blocks = 00
 Hdr CRC= 29ab
 Data Packet:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd5	82f	80a	ecd	c89	488	8b5		
2	222	2d5	122	222	922	219	a44	888	888	8aa	be4	889		
4	c22	227	400	000	000	001								

NAC = 293
 SSymbol= 1
 I/O = 1
 Class = 1
 Type = 5
 Status = 7
 MFID = 00
 LLID = 000001
 blocks = 00
 Hdr CRC= 6025
 Data Packet:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd5	82f	80a	ecd	c8a	d48	8b5		
2	222	c2e	1d2	222	922	2d1	650	888	888	8a5	fe0	889		
4	c22	222	000	000	000	001								

NAC = 293
 SSymbol= 1
 I/O = 1
 Class = 2
 Type = 0
 Status = 0
 MFID = 00
 LLID = 000001
 blocks = 01
 Hdr CRC= c952
 Data : 0 1 2 3 4 5 6 7 8 9 10 11
 0 f8 ff ff ff ff ff ff d7 5b 92 1c

Data Packet:

Microslot	0							1						
0	557	5f5	ff7	7ff	293	cd5	82f	80a	ecd	c89	888	8b5		
2	e22	f09	192	222	922	251	248	888	888	887	bc8	889		
4	cc2	227	1ff	888	888	8b5	a4a	222	222	22c	1f1	a21		
6	888	880	de3	988	888	881	d05	400	000	000	000	001		

B.9 Unconfirmed Data Packet Test Vectors

Case 1: Subscriber Radio

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 01
pads     = 00
offset   = 00
Hdr CRC= 48af
Data : 0 1 2 3 4 5 6 7 8 9 10 11
       0 01 23 45 67 89 ab cd ef 9f 15 9a a0
Data Packet:
Microslot 0 1
          0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd c88 c88 8b4
            2 e22 c15 c52 222 922 2e8 808 888 888 885 008 888
            4 cc2 229 82e b06 d83 65c 688 87b 669 59f 1e8 a70
            6 71a f43 0d9 cf0 374 b88 028 800 000 000 000 000

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 02
pads     = 00
offset   = 00
Hdr CRC= d373
Data : 0 1 2 3 4 5 6 7 8 9 10 11
       0 01 23 45 67 89 ab cd ef 01 23 45 67
       1 89 ab cd ef 01 23 45 67 f4 eb cb 85
Data Packet:
Microslot 0 1
          0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd c88 c88 8b4
            2 922 fb5 c52 222 922 2b0 c08 888 888 8b9 408 888
            4 c12 22f f2e b06 d83 5e8 c14 87b 669 588 7b4 a70
            6 71a f42 9c7 cf0 374 b8c 3c0 c76 0d7 ac2 3e1 ba4
            8 a56 21e d86 591 af4 29c 1ee 871 d2e 33c 0f9 100

```

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 02
pads     = 0b
offset   = 00
Hdr CRC= 2382
Data : 0 1 2 3 4 5 6 7 8 9 10 11
       0 01 23 45 67 89 ab cd ef 01 00 00 00
       1 00 00 00 00 00 00 00 00 53 73 78 61
Data Packet:
Microslot 0 1
          0 557 5f5 ff7 7ff 293 cd4 82f 80a ecd c88 c88 8b4
            2 952 269 c52 222 922 218 408 888 884 8a4 808 888
            4 c14 22f 12e b06 d83 5e0 888 87b 669 588 888 a70
            6 71a f42 222 cf0 374 b8c 088 888 888 88b 2ef 388
            8 222 222 203 3d2 222 220 8b9 5a4 888 888 8bf e70

```

```

NAC      = 293
SSymbol= 0

```



```

I/O      = 1
SAP      = 1f
MFID     = 00
LLID     = 000001
blocks   = 02
pads     = 00
offset   = 00
Hdr CRC= 21d4
Data :   0  1  2  3  4  5  6  7  8  9 10 11
         0 15 c0 00 00 00 02 00 00 00 0d f1
         1 01 23 45 67 89 ab cd ef f8 eb 1c ed
Data Packet:
Microslot      0                                1
0  557 5f5 ff7 7ff 293 cd4 82f 80a ecd c88 d48 8b4
2  922 232 fb2 222 922 218 ecc 888 888 8a4 020 888
4  c12 22c e23 222 292 220 8fb 148 888 888 8a0 088
6  222 222 2f5 022 221 220 8af 0ba c1b 60e 3d7 f88
8  1ed 9a5 686 c62 9c7 1ac d1a 8d3 3c0 dd2 e25 16c

```

Case 2: Base Station Radio

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 01
pads     = 00
offset   = 00
Hdr CRC= 71a2
Data :   0  1  2  3  4  5  6  7  8  9 10 11
         0 01 23 45 67 89 ab cd ef 9f 15 9a a0
Data Packet:
Microslot      0                                1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd c88 c88 8b5
2  e22 cd9 f52 222 922 239 ac8 888 888 896 408 889
4  cc2 22c 12e b06 d83 65d 688 87b 669 59f 1e8 a71
6  71a f43 0d9 cf0 374 b89 028 800 000 000 000 001

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 02
pads     = 00
offset   = 00
Hdr CRC= ea7e
Data :   0  1  2  3  4  5  6  7  8  9 10 11
         0 01 23 45 67 89 ab cd ef 01 23 45 67
         1 89 ab cd ef 01 23 45 67 f4 eb cb 85
Data Packet:
Microslot      0                                1
0  557 5f5 ff7 7ff 293 cd5 82f 80a ecd c88 c88 8b5
2  922 f79 f52 222 922 261 ec8 888 888 8aa 008 889
4  c12 22a 62e b06 d83 5e9 c14 87b 669 588 7b4 a71
6  71a f42 9c7 cf0 374 b8d 3c0 c76 0d7 ac2 3e1 ba5
8  a56 21e d86 591 af4 29d 1ee 871 d2e 33c 0f9 101

```

```

NAC      = 293
SSymbol= 1
I/O      = 1
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 02
pads     = 0b
offset   = 00

```

TIA-102.BAAB

Hdr CRC= 1a8f

Data : 0 1 2 3 4 5 6 7 8 9 10 11
 0 01 23 45 67 89 ab cd ef 01 00 00 00
 1 00 00 00 00 00 00 00 00 53 73 78 61

Data Packet:

Microslot	0												1											
0	557	5f5	ff7	7ff	293	cd5	82f	80a	ecd	c88	c88	8b5												
2	952	2a5	f52	222	922	2c9	6c8	888	884	8b7	c08	889												
4	c14	22a	82e	b06	d83	5e1	888	87b	669	588	888	a71												
6	71a	f42	222	cf0	374	b8d	088	888	888	88b	2ef	389												
8	222	222	203	3d2	222	221	8b9	5a4	888	888	8bf	e71												

NAC = 293

SSymbol= 1

I/O = 1

SAP = 1f

MFID = 00

LLID = 000001

blocks = 02

pads = 00

offset = 00

Hdr CRC= 21d4

Data : 0 1 2 3 4 5 6 7 8 9 10 11
 0 15 c0 00 00 00 02 00 00 00 0d f1
 1 01 23 45 67 89 ab cd ef f8 eb 1c ed

Data Packet:

Microslot	0												1											
0	557	5f5	ff7	7ff	293	cd5	82f	80a	ecd	c88	d48	8b5												
2	922	232	fb2	222	922	219	ecc	888	888	8a4	020	889												
4	c12	22c	e23	222	292	221	8fb	148	888	888	8a0	089												
6	222	222	2f5	022	221	221	8af	0ba	c1b	60e	3d7	f89												
8	1ed	9a5	686	c62	9c7	1ad	d1a	8d3	3c0	dd2	e25	16d												

B.10 Voice Receiver Test Vectors

NAC = 293
 SSym = 0 0 0 0 0 0 0 0 0 0 0 0
 MI = 00 0000 0000 0000 0000
 MFID = 00
 ALGID= 80
 KID = 0000
 TGID = 0001
 Header Data Unit:

Microslot	0	1
0	557 5f5 ff7 7ff 293 00c	ced 77e 007 080 000 000
2	000 000 000 000 000 000	000 000 000 000 000 000
4	000 000 000 000 000 008	371 800 000 000 000 000
6	000 000 000 18e b5e 094	527 c2a 235 40d 9f3 47c
8	f1d 41e 421 ea2 358 bfc	312 a5e 371 84a 965 1b8
10	30a 4f8 0c7 54c 751 000	

NAC = 293
 SSym = 0
 LCF = 00
 MFID = 00
 Emerg= 0
 TGID = 0001
 Sourc= 000001
 Dest = 000001
 LSD = 00 00

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 00
 2: 09b 088 0cc 621 7b4 02a 04c 01
 3: 09b 088 0cc 621 7b4 02a 04c 00
 4: 09b 088 0cc 621 7b4 02a 04c 01
 5: 09b 088 0cc 621 7b4 02a 04c 00
 6: 09b 088 0cc 621 7b4 02a 04c 01
 7: 09b 088 0cc 621 7b4 02a 04c 00
 8: 09b 088 0cc 621 7b4 02a 04c 01
 9: 09b 088 0cc 621 7b4 02a 04c 00

Link Data Unit 1:

Microslot	0	1
0	557 5f5 ff7 7ff 293 554	7bc b19 4d0 dce 24a 124
2	0d4 33c 0be 1b9 184 4fc	c16 296 276 0e4 e24 a10
4	90d 433 c0b e1b 918 44c	fc1 629 627 60e c00 000
6	000 003 892 849 0d4 33c	02f 86e 461 13f c16 294
8	89d 839 000 000 001 c38	24a 124 350 cf0 2f8 6e4
10	184 4ff 058 a58 9d8 3b0	000 000 007 0e2 4a1 240
12	d43 3c0 be1 b91 844 ff0	162 962 760 e6d e5d 548
14	ade 389 284 90d 433 c08	f86 e46 113 fc1 629 624
16	d83 ba1 41c 2d2 ba3 890	a12 435 0cf 02f 86e 460
18	44f f05 8a5 89d 839 4c8	fb0 235 a4e 24a 124 350
20	33c 0be 1b9 184 4ff 058	296 276 0ec 000 000 00c
22	892 849 0d4 33c 0be 1b8	461 13f c16 296 276 0e4

NAC = 293
 SSym = 0
 MI = 00 0000 0000 0000 0000
 ALGID= 80
 KID = 0000
 LSD = 40 41

IMBE : u_0 u_1 u_2 u_3 u_4 u_5 u_6 u7
 1: 09b 088 0cc 621 7b4 02a 04c 01
 2: 09b 088 0cc 621 7b4 02a 04c 00
 3: 09b 088 0cc 621 7b4 02a 04c 01
 4: 09b 088 0cc 621 7b4 02a 04c 00
 5: 09b 088 0cc 621 7b4 02a 04c 01
 6: 09b 088 0cc 621 7b4 02a 04c 00
 7: 09b 088 0cc 621 7b4 02a 04c 01
 8: 09b 088 0cc 621 7b4 02a 04c 00
 9: 09b 088 0cc 621 7b4 02a 04c 01

Link Data Unit 2:

TIA-102.BAAB

Microslot	0								1							
0	557	5f5	ff7	7ff	293	ab8			a4e	fb0	9a8	ace	24a	124		
2	0d4	33c	0be	1b9	184	4fc			c16	296	276	0ec	e24	a10		
4	90d	433	c0b	e1b	918	44c			fc1	629	627	60e	400	000		
6	000	003	892	849	0d4	33c			02f	86e	461	13f	c16	294		
8	89d	83b	000	000	000	038			24a	124	350	cf0	2f8	6e4		
10	184	4ff	058	a58	9d8	390			000	000	000	0e2	4a1	240		
12	d43	3c0	be1	b91	844	ff0			162	962	760	ee0	e00	000		
14	000	389	284	90d	433	c08			f86	e46	113	fc1	629	624		
16	d83	9ae	8b4	8b6	493	890			a12	435	0cf	02f	86e	460		
18	44f	f05	8a5	89d	83b	9a8			f4f	1fd	60e	24a	124	350		
20	33c	0be	1b9	184	4ff	058			296	276	0e5	009	d04	78c		
22	892	849	0d4	33c	0be	1b8			461	13f	c16	296	276	0ec		

NAC = 293

SSym = 0 0

Terminator Data Unit:

Microslot	0								1							
0	557	5f5	ff7	7ff	293	338			5dd	ca3	3b5	b80	000	000		

Limited Use Only

B.11 Data Receiver Test Vectors

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
FMF      = 1
blocks   = 05
pads     = 00
sync     = 0
N(S)     = 0
FSNF     = 8
offset   = 00
Hdr CRC= c822
Data : SN_CRC_0_1_2_3_4_5_6_7_8_9_10_11_12_13_14_15
      0  00 08b 00 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0 45 24 d4
      1  01 1c6 00 51 55 97 61 96 9b 71 d7 9f 82 18 a3 92 59 a7
      2  02 0f5 00 a2 9a ab b2 db af c3 1c b3 d3 5d b7 e3 9e bb
      3  03 0e3 00 f3 df bf 04 20 c4 14 61 c8 24 a2 cc 34 e3 d0
      4  04 1d6 00 45 24 d4 51 55 97 61 96 9b 71 d7 3c 91 ee c0

```

Data Packet:

Microslot	0	1
0	557 5f5 ff7 7ff 293 cd4	82f 80a ecd cb1 088 8b4
2	e22 f29 052 222 922 250	408 888 b08 486 748 888
4	c02 929 128 2af 7d5 08c	9bc 9b4 610 613 567 7f8
6	634 19c 27f e2e 7be 7b0	a9b cb3 ab6 017 acd ac8
8	6f3 6f3 af3 a64 23b 19c	11f e8b 883 243 143 164
10	228 4c9 72a f8d 6df 3a4	bde cba c87 e01 76c db8
12	4cd 2c5 908 4d1 4d2 840	2eb df5 420 b6f 9f4 610
14	184 d4f 91e 634 19c 2f0	863 9ef 9ec a89 798 eb4
16	805 a12 e65 9f3 6f3 3ac	649 fce c67 271 b09 bc0
18	c90 cea 200 000 000 000	

```

NAC      = 293
SSymbol= 0
I/O      = 0
SAP      = 00
MFID     = 00
LLID     = 000001
blocks   = 01
pads     = 00
offset   = 00
Hdr CRC= 48af
Data : 0_1_2_3_4_5_6_7_8_9_10_11
      0  01 23 45 67 89 ab cd ef 9f 15 9a a0

```

Data Packet:

Microslot	0	1
0	557 5f5 ff7 7ff 293 cd4	82f 80a ecd c88 c88 8b4
2	e22 c15 c52 222 922 2e8	808 888 888 885 008 888
4	cc2 229 82e b06 d83 65c	688 87b 669 59f 1e8 a70
6	71a f43 0d9 cf0 374 b88	028 800 000 000 000 000

Limited Use Only

Limited Use Only

THE TELECOMMUNICATIONS INDUSTRY ASSOCIATION

TIA represents the global information and communications technology (ICT) industry through standards development, advocacy, tradeshow, business opportunities, market intelligence and world-wide environmental regulatory analysis. Since 1924, TIA has been enhancing the business environment for broadband, wireless, information technology, cable, satellite, and unified communications.

TIA members' products and services empower communications in every industry and market, including healthcare, education, security, public safety, transportation, government, the utilities. TIA is accredited by the American National Standards Institute (ANSI).

